



Desarrollo de una arquitectura escalable de Big Data en AWS

alimentada mediante un sistema de IoT

Aplicación en monitoreo de generación eólica de pequeña escala

Carlos Andrés Gómez Flórez

Trabajo de grado presentado como requisito para optar al título de:

Especialista en Tecnologías de la Información y las Comunicaciones

Director:

Alexander Molina Cabrera

Codirector:

César Augusto Jaramillo Acevedo

Universidad Tecnológica de Pereira

Facultad de Ingenierías

Programa de Especialización en Tecnologías de la Información y las Comunicaciones

Pereira, Risaralda

2026

Nota de Aceptación

Nombre del director

Director

Jurado

Jurado

Desarrollo de una arquitectura escalable de Big Data en AWS alimentada mediante un sistema de IoT: Aplicación en monitoreo de generación eólica de pequeña escala

© Carlos Andrés Gómez Flórez

Pereira, 2026

Programa de Especialización en Tecnologías de la Información y las Comunicaciones

Universidad Tecnológica de Pereira

Carrera 27 No. 10-02 Barrio Álamos - Risaralda - Colombia

Tel. (+57)(6)3137122

www.utp.edu.co

Resumen

En este trabajo se propone, desarrolla y valida una arquitectura para adquirir, transmitir, almacenar, procesar y visualizar datos de un generador eólico de pequeña escala. La solución integra un subsistema embebido distribuido basado en tarjetas ESP32-C6, comunicación local entre nodos, datos en formato json, publicación segura mediante MQTTs hacia AWS IoT Core y una capa de almacenamiento y consulta soportada en servicios serverless de AWS.

El sistema de adquisición se organiza en tres etapas: un nodo de campo que mide variables del generador, un nodo integrador local que consolida variables ambientales y operativas y estructura el mensaje de telemetría, y un gateway IoT encargado de la conectividad Wi-Fi, la sesión MQTT segura y la publicación de datos hacia la nube.

En la capa cloud, la arquitectura usa AWS IoT Core para la ingesta MQTTs, DynamoDB para conservar el último dato por dispositivo, S3 como repositorio histórico, Glue y Athena para catalogar y consultar series temporales, Lambda como API serverless y un dashboard React/Vite servido mediante S3 y CloudFront. Esta separación distingue la ruta de datos casi en tiempo real de la ruta histórica, reduciendo costos y facilitando la escalabilidad.

La contribución principal es una metodología reproducible para instrumentar, transmitir y analizar datos de pequeños sistemas eólicos integrando IoT, cloud computing y Big Data, con trazabilidad desde la sensórica y el firmware hasta la visualización web y la validación experimental.

Palabras clave: ESP32-C6, IoT, MQTT, TLS, AWS IoT Core, Big Data, DynamoDB, Amazon S3, Athena, Lambda, generación eólica, sistemas embebidos, dashboard web.

Abstract

This work proposes, develops and validates an architecture to acquire, transmit, store, process and visualize data from a small-scale wind generator. The solution integrates a distributed embedded subsystem based on ESP32-C6 boards, local communication between nodes, data in json format, secure MQTTs publication to AWS IoT Core, and a storage and query layer supported by AWS serverless services.

The acquisition system is organized into three stages: a field node that measures generator variables, a local integration node that consolidates environmental and operational variables and structures the telemetry message, and an IoT gateway responsible for Wi-Fi connectivity, the secure MQTT session and data publication to the cloud.

In the cloud layer, the architecture uses AWS IoT Core for MQTTs ingestion, DynamoDB to keep the latest value per device, S3 as the historical repository, Glue and Athena to catalog and query time series, Lambda as a serverless API, and a React/Vite dashboard served through S3 and CloudFront. This separation distinguishes the near-real-time data path from the historical path, reducing costs and facilitating scalability.

The main contribution is a reproducible methodology to instrument, transmit and analyze data from small wind systems by integrating IoT, cloud computing and Big Data, with traceability from sensors and firmware to web visualization and experimental validation.

Keywords: ESP32-C6, IoT, MQTT, TLS, AWS IoT Core, Big Data, DynamoDB, Amazon S3, Athena, Lambda, wind generation, embedded systems, web dashboard.

Tabla de contenidos

Resumen	III
Abstract	IV
1. Introducción	1
1.1. Planteamiento del problema	2
1.2. Justificación	3
1.3. Objetivos	4
1.3.1. Objetivo general	4
1.3.2. Objetivos específicos	4
1.4. Estructura del documento	4
2. Marco de referencia	5
2.1. Internet de las Cosas aplicado a sistemas energéticos	7
2.2. Arquitectura de referencia del sistema propuesto	9
2.3. Modelos OSI y TCP/IP como marco de lectura de la arquitectura	11
2.4. Protocolos de comunicación local: ESP-NOW y UART	13
2.5. MQTT , telemetría y contrato de datos	14
2.6. Seguridad: TLS , certificados X.509 y principio de mínimo privilegio	15
2.7. Arquitecturas serverless y Big Data para telemetría IoT	17
2.8. Computación en el borde y gestión de concurrencia	18
2.9. Síntesis del marco de referencia	20
3. Diseño e implementación de la arquitectura de hardware IoT Edge	22
3.1. Propósito de la capa IoT Edge	24

3.2.	Requisitos funcionales de la arquitectura embebida	25
3.3.	Trazabilidad del código fuente usado para el capítulo	27
3.4.	Arquitectura física y asignación de responsabilidades	27
3.5.	Modelo de capas aplicado al hardware IoT Edge	29
3.6.	ESP1: nodo de adquisición de campo	30
3.6.1.	Adquisición GPS en ESP1	31
3.6.2.	Medición de voltaje en ESP1	32
3.6.3.	Estimación de RPM en ESP1	34
3.6.4.	Envío ESP-NOW desde ESP1	35
3.7.	ESP2: nodo integrador, almacenamiento local y serialización json	35
3.7.1.	Recepción ESP-NOW en ESP2	37
3.7.2.	Medición de velocidad del viento en ESP2	37
3.7.3.	Bus I2C compartido: BME680, OLED y DS3231	38
3.7.4.	Almacenamiento local en microSD	39
3.7.5.	Generación del json en ESP2	40
3.7.6.	Envío UART de ESP2 hacia ESP3	40
3.7.7.	Tareas concurrentes en ESP2	41
3.8.	ESP3: gateway UART, Wi-Fi y MQTT/TLS hacia AWS IoT Core	42
3.8.1.	Conectividad Wi-Fi en ESP3	43
3.8.2.	MQTT sobre TLS y certificados X.509	43
3.8.3.	Recepción y validación json en ESP3	44
3.9.	Contrato de datos Edge–Cloud	44
3.10.	Gestión temporal y sincronización	46
3.11.	Seguridad de la capa hardware Edge	46
3.12.	Integridad, calidad y pérdida de datos	47
3.13.	Estrategia de validación de la capa IoT Edge	47
3.14.	Síntesis del capítulo	49
4.	Diseño e implementación de la arquitectura de software en AWS	50
4.1.	Propósito de la capa cloud	52
4.2.	Requisitos funcionales y no funcionales	53
4.3.	Flujo general de datos	54
4.4.	Ingesta segura con AWS IoT Core	55

4.5. Ruta de último dato: DynamoDB	56
4.6. Ruta histórica: Amazon S3	57
4.7. Catálogo y transformación con AWS Glue	58
4.8. Consulta histórica con Amazon Athena	58
4.9. Capa API con Lambda Function URLs	59
4.10. Roles IAM y mínimo privilegio	60
4.11. Dashboard web con React, S3 y CloudFront	61
4.12. Evidencias de implementación	62
4.13. Síntesis del capítulo	63
5. Pruebas de despliegue y validación	64
5.1. Estrategia general de validación	64
5.2. Validación de la ESP1: adquisición de campo	66
5.2.1. Evidencias (Prueba ESP1)	67
5.2.2. Evidencias (Prueba ESP2)	69
5.2.3. Evidencias (Prueba ESP3)	69
5.2.4. Evidencias (Prueba AWS IoT Core)	69
5.2.5. Evidencias (Prueba DynamoDB)	69
5.2.6. Evidencias (Prueba S3 + Glue + Athena)	75
5.2.7. Evidencias (Prueba Lambda)	76
5.2.8. Evidencias (Prueba Dashboard)	76
6. Análisis de costos, resultados y escalabilidad	80
6.1. Resultados técnicos consolidados	80
6.2. Variables del modelo de costos	82
6.3. Escenario base: una turbina	83
6.3.1. Aplicación numérica del modelo para el escenario base	84
6.3.2. Reglas operativas del dashboard para controlar costos	87
6.3.3. Prueba prolongada de 24 horas para contraste con Cost Explorer	88
6.4. Escalamiento a n turbinas	89
6.5. Discusión de resultados	90
6.6. Síntesis del capítulo	91
7. Conclusiones y trabajos futuros	92

7.1. Conclusiones	92
7.2. Trabajos futuros	94
7.3. Cierre	96
 Apéndices	 98
 A. Anexo A — Código ESP1	 98
 B. Anexo B — Código ESP2	 99
 C. Anexo C — Código ESP3	 100
 D. Anexo D — Lambdas y frontend	 101

Listado de Tablas

2.1. Lectura por capas de la arquitectura IoT propuesta.	12
3.1. Requisitos funcionales de la arquitectura embebida IoT Edge.	25
3.2. Trazabilidad de archivos fuente analizados para la arquitectura IoT Edge.	27
3.3. Distribución física y lógica de responsabilidades entre las tres ESP32-C6.	28
3.4. Modelo por capas aplicado a los tramos del sistema IoT Edge.	30
3.5. Configuración de hardware identificada en ESP1.	31
3.6. Subsistemas integrados en ESP2 según el firmware activo.	36
3.7. Tareas concurrentes identificadas en ESP2.	41
3.8. Configuración funcional del gateway ESP3.	42
3.9. Especificación propuesta del contrato json Edge–Cloud.	45
3.10. Riesgos y controles de seguridad para la capa IoT Edge.	46
3.11. Casos de prueba recomendados para validar la arquitectura IoT Edge.	48
4.1. Requisitos funcionales de la arquitectura AWS.	53
4.2. Diseño lógico de la ruta de último dato en DynamoDB.	56
4.3. Capas de datos previstas para el histórico IoT.	58
4.6. Evidencias recomendadas para documentación y validación.	62
5.1. Niveles de validación propuestos para el sistema extremo a extremo.	65
5.2. Casos de validación del nodo ESP1.	66
6.1. Resultados técnicos consolidados de la arquitectura.	81
6.2. Variables del modelo paramétrico de costos.	82
6.3. Comportamiento cualitativo de costos para una turbina.	83
6.4. Reglas de juego para el escenario base de costos.	84

6.5. Estimación mensual aproximada para una turbina con dashboard histórico bajo demanda.	86
6.6. Reglas de operación del dashboard para el escenario base.	87
6.7. Plan de consultas puntuales para la prueba controlada de 24 horas.	88
6.8. Ajustes de arquitectura para escalar a diez turbinas.	89
6.9. Comparación entre almacenamiento JSON y Parquet para histórico IoT.	90

Listado de Figuras

2.1. Arquitectura de referencia rediseñada del sistema IoT para monitoreo de un generador eólico axial en AWS. Fuente: elaboración propia con apoyo de ChatGPT, modelo GPT-5.5 Thinking de OpenAI, 2026.	6
3.1. Arquitectura IoT Edge basada en tres ESP32-C6: nodo de campo, nodo integrador y gateway AWS. Fuente: elaboración propia con apoyo de ChatGPT, modelo GPT-5.5 Thinking de OpenAI, 2026.	23
4.1. Arquitectura de software en AWS para el sistema IoT de monitoreo eólico. Fuente: elaboración propia con apoyo de ChatGPT, modelo GPT-5.5 Thinking de OpenAI, 2026.	51
5.1. ESP1 desenergizada: validación de condición de pérdida de enlace y valores controlados para voltaje y RPM.	67
5.2. Cliente MQTT - esp_now_connected false	68
5.3. Consola esp-idf - flujo ESP1 a ESP2	68
5.4. ESP2 desenergizada	70
5.5. Cliente MQTT - sin mensajes ESP2	71
5.6. Consola esp-idf - UART ESP2	71
5.7. ESP3 desenergizada	72
5.8. Cliente MQTT - sin mensajes ESP3	73
5.9. Consola esp-idf - MQTT ESP3	73
5.10. IoT Core - recibiendo mensajes	74
5.11. Política IoT	74
5.12. DynamoDB turbina_latest	74
5.13. S3 processed	75

5.14. S3 objeto json	75
5.15. Glue table	75
5.16. Athena consulta	76
5.17. Lambda GetLatest	77
5.18. Lambda GetTimeseries	78
5.19. Dashboard en S3	78
5.20. Dashboard 15 min	79

Introducción

La transición energética requiere de grandes esfuerzos para suplir la demanda energética, esto, implica avanzar hacia el aprovechamiento energético en todas las escalas. Lo anterior, ha generado que la energía eólica a pequeña escala gane interés en diferentes áreas de la investigación, la formación académica y en la implementación de soluciones energéticas distribuidas. Para conocer el estado de la operación constantemente, se requiere implementar sistemas con la capacidad de medir de manera continua las diferentes variables operativas y ambientales que influyen en la generación eólica. En este trabajo, se propone una arquitectura de datos de extremo a extremo que integra sistemas embebidos, comunicación IoT con MQTTs, integración con servicios serverless de AWS y sistema de visualización web reactivo de las diferentes variables del sistema [UPME, 2024, Wang et al., 2023, Ullah et al., 2024, Calautit and Johnstone, 2023, Foley et al., 2012].

El sistema desarrollado en este trabajo se estructura alrededor de un generador eólico de eje vertical tipo Savonius instrumentado con tarjetas ESP32-C6. Debido a la capacidad de las tarjetas, naturaleza de los procesos y funciones implementadas, se segmentó en una arquitectura que distribuye sus funciones en 3 nodos o ESPs. La ESP1 mide variables del generador como voltaje AC, RPM de la turbina, coordenadas y transmite a ESP2 a través de ESP-NOW; la ESP2 consolida variables de la ESP1 con variables ambientales (temperatura, presión atmosférica, humedad, altitud), velocidad del viento, almacenamiento local, timestamp, visualización local y transmite todo en paquete json a través de UART; y ESP3 actúa como gateway MQTT/TLS hacia AWS IoT Core. Esta separación facilita la depuración, reduce la complejidad por nodo, permite aislar fallas de sensórica, facilita la comunicación local con la conectividad cloud y trata de aprovechar al máximo las prestaciones de cada ESP32-C6 a través de la programación con ANSI/C en su

entorno nativo esp-idf versión 5.4.1 [Gómez Flórez, 2024, Bhutta et al., 2012, Eriksson et al., 2008, Tjiu et al., 2015, Wekesa et al., 2020].

Para el almacenamiento y la visualización de datos, se propone una arquitectura soportada en servicios serverless administrados de AWS dado que previamente el autor presentó una arquitectura con almacenamiento y visualización local tradicional. Para usar el servicio que provee tópicos y otras funciones que facilitan la conexión con MQTT, Amazon exige soportar la comunicación con cifrado SSL/TLS (MQTTs). Soportado en esta implementación, se usa AWS IoT Core, el cual recibe la telemetría y a partir de políticas y gestión de eventos consume DynamoDB para conservar el último paquete de datos recibido y Amazon S3 para almacenar los datos históricos, los cuáles se estructuran en formato YYYY/MM/DD/HH/mm. A partir de AWS Glue y amazon Athena se habilitan consultas temporales (ETL), AWS Lambda gestiona las funciones para la base de datos y expone servicios HTTP para el consumo de estos y un dashboard React/Vibe desplegado mediante S3 y CloudFront se encargan de visualizar los datos instantáneos y las series históricas de las variables disponibles bajo demanda. El resultado de esta arquitectura es una plataforma reproducible y escalable para validación experimental, monitoreo remoto y análisis de los datos que pueden influir en estos esquemas de generación en sistemas de pequeña escala [Gubbi et al., 2013, Botta et al., 2016, Hashem et al., 2015, Marjani et al., 2017].

1.1. Planteamiento del problema

Los sistemas energéticos de pequeña escala o para implementación en lugares off-grid suelen enfrentar múltiples desafíos relacionados con el monitoreo y analítica de los datos relacionados con el proceso debido principalmente a limitaciones en infraestructura. Entender cómo se comporta la radiación solar o la velocidad del viento puede ser determinante para la viabilidad y el sostenimiento de un proyecto, por esta razón, es indispensable pensar en implementar sistemas para la medición de datos que permitan consulta histórica, trazabilidad y escalamiento a múltiples dispositivos. Entender la relación entre las variables atmosféricas, la respuesta mecánica y la generación eléctrica es importante por viabilidad e implementación de modelos de desempeño, mantenimiento o pronóstico del recurso [Akyuz and demircan, 2019, Tchakoua et al., 2014, Liu et al., 2025, Emexidis and Gkonis, 2024, Jung and Broadwater, 2014].

El problema en este tipo de sistemas no consiste únicamente en medir variables, sino en diseñar una topología completa que permita la interconexión entre los diferentes sistemas, asegurando

la adquisición, preprocesamiento, transmisión, almacenamiento, procesamiento, visualización de datos y los procesos adicionales que se implementen para el aprovechamiento de los datos obtenidos. Lo anterior, debe garantizar la integridad, trazabilidad, seguridad y escalabilidad del proceso, permitiendo avanzar en la resolución de preguntas como: qué se mide, cómo se transmite, dónde se almacena y cómo se consulta [Al-Fuqaha et al., 2015, Atzori et al., 2010, Sisinni et al., 2018, Sasi et al., 2024].

Con base en el planteamiento anterior, se plantea como pregunta orientadora del trabajo:

¿Cómo desarrollar una arquitectura soportada en IoT y Big Data, a partir de ESP32-C6 y servicios de AWS, que permita el monitoreo de forma segura y escalable de las variables operativas y ambientales de un generador eólico de eje vertical de pequeña escala?

1.2. Justificación

La transición energética va ligada a su vez a una gran transición digital, y esto, implica que el sector eléctrico se vea mediado por una profundización en el desarrollo de sistemas digitales complejos que faciliten la medición en escalas temporales lo más cercanas al tiempo real, permitiendo el seguimiento a los mercados y sistemas de estimación de energía, intercambio de excedentes, perfiles de consumo de cargas específicas, entre muchos otros elementos. La integración entre sistemas embebidos con protocolos de IoT y servicios de Cloud Computing permiten desplegar arquitecturas que como se demuestra en este trabajo, pueden ser accesibles, escalables, seguras, trazables y proveen ventajas con respecto a sistemas soportados en computación tradicional. En este proyecto, se aprovecha el potencial de las tarjetas ESP32-C6, las cuales soportan múltiples protocolos de comunicación a muy buenas velocidades y con excelentes herramientas que son utilizadas para la captación y el procesamiento de las variables medidas. Entre esos protocolos, se destacan el manejo de múltiples dispositivos I2C, SPI, UART, WIFI, ESP-NOW, MQTTs, desplegando MQTT/TLS usando las funcionalidades nativas de ESP-IDF, lo que permite optimizar el rendimiento de los procesadores, además del uso de FreeRTOS para gestionar de mejor manera múltiples procesos en cada etapa. Adicionalmente, el desarrollo de la arquitectura de almacenamiento, ETL y visualización en AWS permite separar responsabilidades, a bajo costo, sin administrar servidores dedicados y mejorando la disponibilidad y presentación de la información [International Energy Agency (IEA), 2023, Malhotra et al., 2021, Botta et al., 2016, Sisinni et al., 2018, Quincozes et al., 2019].

El enfoque propuesto en este trabajo es importante por tres razones fundamentales:

Primero, se busca el fortalecimiento de la instrumentación y monitoreo de un sistema eólico previamente construido, mediante arquitecturas de hardware y software económicas, accesibles, escalables y seguras que permiten el almacenamiento masivo de datos. Segundo, el sistema permite analizar los datos a nivel histórico e instantáneo para buscar patrones de interés con base en las variables que se relacionan con la potencia eólica y la generación eléctrica, sentando bases importantes para escalar el nivel de analítica del sistema mediante la implementación de algoritmos más complejos basados en análisis de series temporales, machine learning, deep learning u otras herramientas. Tercero, permite documentar buenas prácticas de seguridad en transmisión y almacenamiento de sistemas de datos con IoT, la cual no es un área que se haya desarrollado mucho en nuestra región, en el país y le aporta a la generación de conocimiento local y propio [Gómez Flórez, 2024, Gong and Chen, 2024, Liu et al., 2025, Emexidis and Gkonis, 2024, Marjani et al., 2017, Alwahedi et al., 2024].

1.3. Objetivos

1.3.1. Objetivo general

Desarrollar una arquitectura escalable de Big Data en AWS alimentada mediante dispositivos IoT para el monitoreo de variables físicas de un generador eólico de pequeña escala.

1.3.2. Objetivos específicos

1. Desarrollar un sistema de adquisición de datos para un pequeño generador eólico de eje vertical a partir del SoC ESP32-C6.
2. Desarrollar una arquitectura para la recepción, almacenamiento y procesamiento de datos soportado en AWS IoT Core y otros servicios de AWS.
3. Desarrollar una interfaz de visualización para los datos de interés almacenados en AWS.
4. Validar el funcionamiento del prototipo, documentar los desafíos y trabajos potenciales

1.4. Estructura del documento

El documento se organiza en siete capítulos. El capítulo 1 presenta la introducción, el problema, la justificación y los objetivos. El capítulo 2 desarrolla el marco de referencia sobre IoT, modelos de comunicación, protocolos, seguridad y serverless. El capítulo 3 describe la arquitectura de hardware IoT Edge basada en ESP32-C6. El capítulo 4 presenta la arquitectura de software en AWS. El capítulo 5 documenta las pruebas y validación del sistema. El capítulo 6 analiza costos, resultados y escalabilidad. Finalmente, el capítulo 7 presenta conclusiones y trabajos futuros.

Marco de referencia

En este capítulo se desarrolla el marco de referencia que soporta la arquitectura propuesta para el monitoreo de un generador eólico de pequeña escala soportado en IoT y procesamiento en la nube de AWS. Para este análisis, el foco se centra en el desarrollo de la arquitectura desde la capa física hasta la capa de visualización, examinando los diferentes protocolos requeridos para la implementación en cada etapa desde la perspectiva de los modelos OSI y TCP/IP. El National Institute of Standards and Technology (NIST, 2016) plantea que las redes de cosas integran sensores, agregadores, canales de comunicación, servicios externos y procesos de decisión, por lo cual esta perspectiva permite analizar la arquitectura por capas funcionales. De forma complementaria, AWS (2026g) presenta el enfoque **serverless** como una manera de diseñar y operar aplicaciones en la nube delegando la administración de infraestructura al proveedor, lo que resulta pertinente para el pipeline de adquisición, almacenamiento, procesamiento y visualización que se plantea en este trabajo [National Institute of Standards and Technology, 2016, AWS, 2026n].

La Figura nro. 2.1, arquitectura de referencia, resume el sistema como una cadena de adquisición, comunicación, recepción, almacenamiento, consulta y visualización de datos. Se propone de esta manera porque se proyectan dos arquitecturas comparables, local frente a nube, y permite determinar de qué manera se pueden aprovechar las oportunidades que ofrecen y evaluar sus limitaciones. En esta línea, Gubbi et al. (2013) explican que una visión cloud-centric del IoT facilita integrar sensores, redes y plataformas de procesamiento para explotar datos provenientes del mundo físico. Por su parte, AWS (2026g) sostiene que las arquitecturas **serverless** permiten reducir la carga operativa asociada a servidores dedicados. A partir de lo anterior, la nube ofrece



Figura 1. Arquitectura lógica del sistema (desde sensorica/ESP32 hasta visualización web).

Figura nro. 2.1: Arquitectura de referencia rediseñada del sistema IoT para monitoreo de un generador eólico axial en AWS. Fuente: elaboración propia con apoyo de ChatGPT, modelo GPT-5.5 Thinking de OpenAI, 2026.

la flexibilidad de no administrar uno o múltiples servidores locales para implementar todo el pipeline mencionado, por lo que se profundiza en una arquitectura **serverless** donde se paga por uso y se mejora la flexibilidad y la escalabilidad según la demanda del sistema; la seguridad, en cambio, se aborda mediante mecanismos específicos de comunicación segura, certificados, políticas y control de acceso que se desarrollan en apartados posteriores [Gubbi et al., 2013, AWS, 2026n].

2.1. Internet de las Cosas aplicado a sistemas energéticos

El Internet of Things (IoT), o Internet de las Cosas, puede entenderse como un ecosistema de tecnologías donde se relacionan diversas áreas del conocimiento. Desde la electrónica, se integran dispositivos de hardware con capacidad de sensado de variables físicas; desde los sistemas, se incorporan redes, procesamiento local, protocolos de comunicación y servicios de información que permiten transformar observaciones del mundo físico en datos explorables. El NIST (2016) explica que las redes de cosas articulan objetos físicos y componentes computacionales para observar el entorno y generar información útil. En el caso de pequeños sistemas eólicos, Wang et al. (2023) señalan que las turbinas eólicas de pequeña escala tienen potencial para integrarse con aplicaciones IoT, especialmente cuando se requiere monitorear variables ambientales, eléctricas y mecánicas. En el generador eólico de prueba de este trabajo, el sistema IoT interconecta los sensores de capa física con una infraestructura que permite la estructuración, transporte, almacenamiento y procesamiento de esa información en servicios de AWS, convirtiéndolo en un sistema ciberfísico que ofrece trazabilidad de la información, diagnóstico operativo y bases de datos históricas para soportar análisis posteriores [National Institute of Standards and Technology, 2016, Wang et al., 2023] [Atzori et al., 2010, Botta et al., 2016, Sisinni et al., 2018].

Si se analiza un sistema IoT desde una perspectiva de funcionamiento, se podría dividir en tres dominios de operación diferentes: el dominio físico, donde se ubican las variables a medir, como temperatura, voltaje y velocidad del viento; el dominio embebido o Edge, donde se capturan, filtran y estructuran los datos medidos; y el dominio de plataforma, donde se almacenan, consultan y visualizan los datos. Al-Fuqaha et al. (2015) describen el IoT como un conjunto de tecnologías habilitadoras, protocolos y aplicaciones que permiten conectar objetos, redes y servicios; esta interpretación permite organizar el sistema de este trabajo en capas. En nuestro caso, el planteamiento anterior se materializa a través de la sensórica para medir variables del

proceso eólico y estructurarlas en dimensión temporal, como el **BME680**, el anemómetro, el **DS3231** y los sensores de pulsos. También se materializa en tres tarjetas **ESP32-C6**, las cuales cuentan con múltiples protocolos de comunicación que operan en varias capas, como **I2C**, **SPI**, **UART**, **MQTT**, **SSL/TLS**, **HTTP**, **ESP-NOW**, Zigbee y Thread. Finalmente, se implementa una plataforma en AWS soportada en servicios como **AWS IoT Core**, **DynamoDB**, **S3**, **Glue**, **Athena**, **Lambda**, **CloudFront**, **CloudTrail** y **CloudWatch**. AWS (2026a) permite sustentar esta selección al documentar que **AWS IoT Core** soporta protocolos como **MQTT** sobre **TLS** para conectar dispositivos con servicios cloud [Al-Fuqaha et al., 2015, AWS, 2026g].

En esta investigación, la arquitectura puede interpretarse como la integración entre una capa de hardware/sensado como servicio (HaaS) y una capa de software como servicio (SaaS). La primera convierte variables físicas del generador eólico en telemetría estructurada; la segunda permite que esos datos sean almacenados, consultados y visualizados mediante servicios cloud, sin que el usuario final administre servidores o bases de datos locales [Mell and Grance, 2011, Perera et al., 2014, Botta et al., 2016, Marjani et al., 2017].

En sistemas energéticos, la importancia del IoT y la modernización de los sistemas de comunicación surge porque se requiere monitorear variables de diversas naturalezas para conocer la condición operativa y establecer relaciones o patrones con otro tipo de variables. Wang et al. (2023) muestran que los sistemas eólicos pequeños pueden beneficiarse de aplicaciones IoT porque estas permiten capturar datos de operación y del entorno para análisis posteriores. Tchakoua et al. (2014) también destacan que el monitoreo de condición en turbinas eólicas requiere observar variables mecánicas, ambientales y eléctricas para apoyar diagnóstico y mantenimiento. Por ejemplo, la potencia eólica depende de la velocidad del viento y de la densidad del aire, siendo la densidad del aire una variable relacionada con la presión atmosférica y la temperatura. A su vez, la potencia eléctrica depende de la velocidad de rotación del rotor de la turbina y de las condiciones de generación. De allí que se requieran sistemas electrónicos que permitan la observabilidad permanente de estos sistemas para evaluar desempeño, detectar anomalías, identificar fallas y construir modelos de operación y predicción que contribuyan al mejoramiento de la gestión del proceso. El NIST (2019) complementa este argumento al indicar que en sistemas IoT deben considerarse la seguridad, la confiabilidad y la integridad de los datos durante el ciclo de vida del dispositivo [Wang et al., 2023, Tchakoua et al., 2014] [Akyuz and demircan, 2019, Liu et al., 2025, Emexidis and Gkonis, 2024, Pryor et al., 2020, Foley et al., 2012].

La arquitectura IoT propuesta en este trabajo se plantea como una solución con nodos distribuidos. Esta decisión se entiende mejor al compararla con el trabajo desarrollado previamente por Gómez Flórez (2024), donde se trabajaba con una tarjeta BeagleBone Black para la medición de variables, un Arduino de protección, almacenamiento en MySQL, procesamiento en R y visualización con Shiny. A partir de esa tesis raíz, la nueva arquitectura propone una evolución hacia tres tarjetas **ESP32-C6**, las cuales poseen menor capacidad de procesamiento que una BeagleBone Black, pero son más económicas, tienen menor consumo y permiten distribuir funciones en nodos especializados. En lugar de concentrar todas las funciones en un único nodo, se separan las tareas en adquisición de variables de generación en la ESP1, adquisición de variables ambientales e integración local en la ESP2, y gateway hacia AWS en la ESP3. Esta separación se sustenta en la lectura distribuida de los sistemas IoT planteada por el NIST (2016) y en la documentación de Espressif Systems (2026a), que presenta **ESP-NOW** como una alternativa de comunicación local entre dispositivos ESP. Esta organización reduce complejidad por módulo, permite aprovechar las capacidades de cada ESP, mejora la depuración por etapas y permite reemplazar o ampliar sensores sin rediseñar toda la cadena de comunicación [Gómez Flórez, 2024, Espressif Systems, 2026d] [Liu et al., 2017, Gong and Chen, 2024, Malhotra et al., 2021, Bakhshi et al., 2024].

2.2. Arquitectura de referencia del sistema propuesto

Debido a la naturaleza del sistema y a la división por nodos adecuados a la distribución física y espacial del sistema eólico, se plantea una arquitectura de referencia con cuatro capas: física y sensórica, Edge o comunicación local, capa cloud para ETL y capa cloud para visualización. El NIST (2016) permite justificar esta división al considerar que las redes de cosas se componen de sensores, canales de comunicación, agregadores y servicios externos que permiten convertir eventos físicos en información digital. AWS (2026e), por su parte, permite sustentar el uso de funciones **Lambda** como mecanismo para exponer servicios HTTP ligeros dentro de una arquitectura cloud. Esta estructura permite explicar con mayor detalle cuál es el rol que cumple cada componente, como sensores, **DS3231**, pantalla **OLED** y memoria **SD**; cuál protocolo se usa en cada tramo; por qué se usa; y cuáles riesgos surgen cuando se pasa de señales físicas a información consultable desde un dashboard expuesto a la web [National Institute of Standards and Technology, 2016, AWS, 2026k] [Gubbi et al., 2013, Al-Fuqaha et al., 2015, Atzori et al., 2010, Botta et al., 2016].

La primera capa corresponde al entorno físico del generador eólico de eje vertical y sus variables asociadas. En esta capa se ubican sensores y dispositivos necesarios para el funcionamiento adecuado de la arquitectura. En la ESP1, ubicada a la altura del generador, se mide la velocidad de la turbina contando sus RPM con un sensor de efecto Hall, se mide el voltaje a partir del acondicionamiento de la señal sinusoidal y del uso del conversor análogo-digital, y se integra un módulo **GPS** para obtener información de ubicación del dispositivo. En la ESP2, ubicada a una altura media o alta de la turbina, se leen los datos del sensor **BME680**, el cual provee datos de presión atmosférica, temperatura, humedad y altitud; también se toma la fecha y hora del reloj de tiempo real **DS3231**, la velocidad del viento del anemómetro mediante pulsos, interrupciones y temporizadores, el uso de **memoria SD** para almacenamiento temporal de muestras y la pantalla **OLED** para visualización local de datos. Esta capa se relaciona con Gómez Flórez (2024), quien ya había trabajado la adquisición de variables físicas en turbinas eólicas de eje vertical, aunque con una arquitectura diferente basada en BeagleBone Black y servidor físico remoto [Gómez Flórez, 2024, Bosch Sensortec, 2024] [Wang et al., 2023, Tchakoua et al., 2014, Bhutta et al., 2012, Eriksson et al., 2008, Calautit and Johnstone, 2023].

La segunda capa corresponde al procesamiento Edge en el sistema embebido, para este caso el SoC **ESP32-C6**. Espressif Systems (2026a) documenta **ESP-NOW** como un protocolo de comunicación **Wi-Fi** sin conexión que permite enviar datos entre dispositivos ESP sin construir una red IP completa, lo cual justifica su uso entre la ESP1 y la ESP2. En esta capa, la ESP1 envía los datos medidos y estructurados a la ESP2 mediante **ESP-NOW**. La ESP2 consolida todos los datos en una estructura **json** y la transmite por **UART** a la ESP3, que actúa como gateway. La ESP3 recibe el paquete de datos, lo reconstruye, gestiona la conexión a la red Wi-Fi y la comunicación con **AWS IoT Core** mediante **MQTT** sobre **SSL/TLS**, y se encarga de enviar los datos una vez se conecta al tópico seguro. El NIST (2019) advierte que los sistemas IoT deben gestionar riesgos de seguridad, privacidad e integridad de los datos; por eso, esta etapa es clave, ya que los errores que aquí ocurren pueden propagarse a través del sistema hasta alojarse en la base de datos y generar análisis erróneos [Espressif Systems, 2026d, National Institute of Standards and Technology, 2019] [Sisinni et al., 2018, Quincozes et al., 2019, Al-Fuqaha et al., 2015, Malhotra et al., 2021].

La tercera capa corresponde a la recepción de datos en **AWS IoT Core** mediante **MQTT** seguro, el almacenamiento del último dato en **DynamoDB**, el almacenamiento histórico en **Amazon S3**, la gestión de ETL histórico mediante **Athena**, Glue y **S3**, y la exposición web mediante

Lambda. AWS (2026g) presenta **AWS IoT Core** como un servicio orientado a la comunicación segura entre dispositivos y la nube mediante protocolos como **MQTT**, mientras que AWS (2026l) describe **Lambda Function URLs** como una alternativa para exponer funciones mediante HTTPS sin desplegar un servidor propio. Para la capa de datos, AWS (2026c) permite sustentar el uso de **DynamoDB** con claves primarias para identificar ítems, AWS (2026m) documenta el uso de prefijos en **S3** para organizar objetos, y AWS (2026a) explica que **Athena** utiliza **AWS Glue Data Catalog** para almacenar metadatos de tablas, columnas y datos ubicados en **S3**. La división entre “último dato” e “histórico” permite generar un monitoreo más constante y responder a una visualización más extensa de los datos cuando se requiera. Esta separación también permite que el sistema no consulte permanentemente el histórico para mostrar información operativa reciente, lo cual ayuda a controlar costos y tiempos de respuesta [AWS, 2026g,c,m,a] [Hashem et al., 2015, Marjani et al., 2017, Botta et al., 2016].

La cuarta capa corresponde a la exposición de los datos al usuario. Las funciones **Lambda** con URL pública controlada actúan como API entre el almacenamiento y el dashboard web, mientras que **React/Vite**, **S3** y **CloudFront** permiten presentar una actualización de los datos cada 30 segundos y series históricas temporales de cada variable registrada. AWS (2026k) explica que las configuraciones **CORS** son necesarias cuando una aplicación web consulta recursos desde un origen distinto, y AWS (2026l) permite sustentar el uso de **Lambda Function URLs** como mecanismo ligero de exposición HTTP. Esta capa se debe diseñar teniendo en cuenta los tiempos de respuesta de cada consulta, el control de errores identificados y las restricciones de consulta para evitar la elevación de los costos de ETL o el aumento de latencias injustificadas [AWS, 2026k,l].

2.3. Modelos OSI y TCP/IP como marco de lectura de la arquitectura

Este trabajo se aborda desde la mirada de los modelos OSI y TCP/IP, siendo TCP/IP el modelo que se implementa en la práctica. El modelo OSI es útil porque explica de manera ordenada y detallada en qué nivel se gestiona cada función de la comunicación. IEEE Standards Association (2016) permite comprender la operación de redes inalámbricas en términos de funciones de capa física y de enlace, mientras que Eddy (2022) especifica TCP como protocolo de transporte orientado a conexión y entrega confiable. En el sistema de prueba que se implementa, la lectura

por capas facilita la distinción entre señales a nivel de capa física, transmisión en capa de enlace, red IP, transporte seguro mediante **TLS** y aplicación de telemetría en las capas superiores [IEEE Standards Association, 2018, Eddy, 2022a].

En la implementación real se hace uso del estándar TCP/IP, el cual simplifica el modelo OSI en cuatro capas: acceso a la red, Internet, transporte y aplicación. Espressif Systems (2026d) permite sustentar que **ESP-NOW** opera como un protocolo **Wi-Fi** sin conexión entre dispositivos ESP, principalmente sin requerir direccionamiento IP. En cambio, AWS (2026g) muestra que la salida hacia **AWS IoT Core** requiere conexión Wi-Fi, asignación de dirección IP, establecimiento de sesión TCP, negociación **TLS** y publicación mediante **MQTT** hacia un broker administrado por AWS. Por esta razón, el sistema combina tramos no IP en la comunicación local con un tramo IP seguro hacia la nube [Espressif Systems, 2026d, AWS, 2026g, Rescorla, 2018].

Tabla nro. 2.1: Lectura por capas de la arquitectura IoT propuesta.

Capa conceptual	Implementación en el prototipo	Función técnica
Física	Sensores, ADC, GPIO, UART, radio 2.4 GHz	Captura y transporte físico de señales
Enlace	ESP-NOW, Wi-Fi/IEEE 802.11, direccionamiento MAC	Comunicación local y acceso inalámbrico
Red	IP sobre Wi-Fi en ESP3	Conectividad hacia Internet
Transporte	TCP para MQTT/TLS	Entrega confiable extremo a extremo
Seguridad	TLS, certificados X.509, políticas IoT	Autenticación, cifrado e integridad
Aplicación	MQTT, json, Lambda URL, dashboard	Telemetría, consulta y visualización

La Tabla nro. 2.1 muestra que no todas las comunicaciones son iguales en el sistema. El tramo ESP1–ESP2, aunque se comunica a través de radiofrecuencia de 2.4 GHz mediante **ESP-NOW**, no requiere dirección IP, ya que se soporta en la dirección MAC única de cada ESP. La comunicación entre ESP2 y ESP3 se hace a través de **UART** a 9600 baudios. Mientras tanto, la comunicación

ESP3–AWS sí depende de conexión Wi-Fi, direccionamiento IP, TCP y **TLS**. Eddy (2022) permite sustentar el papel de TCP en la entrega confiable de datos, mientras que Rescorla (2018) fundamenta el uso de **TLS** como protocolo de seguridad para proteger la comunicación. Esta aclaración es importante porque ayuda a identificar las diferentes etapas del sistema y los criterios de decisión que permiten aprovechar mejor los recursos disponibles, combinando comunicación local de baja complejidad con publicación cloud segura, eficiente y económica [Eddy, 2022a, Rescorla, 2018].

La lectura por capas también permite identificar y clasificar de mejor manera los riesgos. El NIST (2019) explica que los sistemas IoT presentan riesgos asociados con la seguridad, privacidad, disponibilidad, integridad y confiabilidad de los datos. En la capa física pueden predominar errores de medición, ruido, falta de mantenimiento, daños en los sensores o problemas de cableado. En la capa de enlace pueden generarse pérdidas de paquetes, interferencia, colisiones o pérdida de comunicación en buses como **I2C**. En la capa de transporte se deben tener en cuenta las reconexiones y las latencias. En sesión y presentación se evalúan aspectos de seguridad, certificados, claves y permisos. Finalmente, en la capa de aplicación se revisa la consistencia del **json**, la validación de campos, los errores HTTP y las consultas históricas. AWS (2026f) complementa esta lectura al señalar que las políticas de **AWS IoT Core** permiten autorizar acciones específicas sobre recursos determinados, lo cual resulta necesario cuando los dispositivos publican datos hacia la nube [National Institute of Standards and Technology, 2019, AWS, 2026f] [Sasi et al., 2024, Bakhshi et al., 2024, Liu et al., 2024, Alwahedi et al., 2024].

2.4. Protocolos de comunicación local: ESP-NOW y UART

ESP-NOW es un protocolo propio de los dispositivos ESP soportado por Espressif para el entorno local de **esp-idf**. Espressif Systems (2026d) describe **ESP-NOW** como un protocolo de comunicación **Wi-Fi** sin conexión que permite comunicación directa entre dispositivos ESP mediante tramas de acción específicas, incluyendo registro de pares, callbacks de envío o recepción y transmisión de datos entre dispositivos. En este caso, se usa **ESP-NOW** porque es un protocolo flexible que permite intercomunicar dos o más ESP32 a través de sus direcciones MAC en la banda de 2.4 GHz, evitando construir una red IP completa. Esto es conveniente cuando se deben transmitir datos a un nodo integrador y se quiere reducir la complejidad de red. No obstante, su uso exige registrar pares de tarjetas, controlar direcciones MAC y configurar seguridad para

garantizar cifrado en la transmisión cuando el sistema lo requiera [Espressif Systems, 2026d, IEEE Standards Association, 2018].

Dentro de la arquitectura, **ESP-NOW** se ubica en un nivel diferente a **MQTT**. OASIS (2019) define **MQTT** como un protocolo de mensajería publish/subscribe basado en clientes, broker y tópicos, mientras que Espressif Systems (2026d) presenta **ESP-NOW** como una comunicación local entre dispositivos ESP. Por ello, **MQTT** se asocia a la telemetría que sale hacia Internet y **AWS IoT Core**, mientras que **ESP-NOW** permite la comunicación entre ESPs en la red local, sin depender de un broker ni de conectividad IP. En este trabajo, **ESP-NOW** se presenta como un protocolo de borde o Edge que, aunque no pertenece a la capa cloud, brinda prestaciones útiles para sistemas IoT distribuidos en espacios físicos donde se requiere comunicación local ligera [OASIS, 2019, Espressif Systems, 2026d].

Por otra parte, la comunicación **UART** entre ESP2 y ESP3 cumple una función diferente. Espressif Systems (2026h) documenta **UART** como un mecanismo de comunicación serial que permite transmisión y recepción de datos mediante un controlador dedicado. En este proyecto, la ESP3 puede estar cerca de la ESP2, por lo que se aprovecha **UART** para transferir el **json** consolidado desde el nodo integrador hacia el gateway de AWS y no interferir con la comunicación Wi-Fi de la ESP3. Aunque **UART** es un protocolo simple y robusto para distancias cortas, el NIST (2019) permite recordar que los sistemas IoT deben cuidar la integridad y confiabilidad de los datos transmitidos. Por ello, se deben tener en cuenta sus limitaciones y configuraciones de delimitadores, tamaño máximo de trama, control de errores y criterios de descarte para evitar publicar mensajes incompletos o corruptos en AWS [Espressif Systems, 2026l, National Institute of Standards and Technology, 2019].

Desde el punto de vista de diseño, esta fue la solución propuesta a los problemas que surgieron en el camino con la adquisición y transmisión de datos en cada etapa. La combinación **ESP-NOW** + **UART** responde a la necesidad de comunicación en el espacio del generador eólico, aunque esta situación puede cambiar en otros sistemas y requerir nuevas arquitecturas. Espressif Systems (2026d) sustenta la comunicación local mediante **ESP-NOW**, mientras que AWS (2026g) permite sustentar la necesidad de conectar finalmente el sistema a una plataforma cloud mediante **MQTT** seguro. A su vez, esta combinación permite separar el dominio inalámbrico local del dominio de publicación a Internet, reduciendo la carga funcional sobre los diferentes nodos y especializando cada tarjeta en funciones específicas. Esta decisión también marca una diferencia frente a Gómez

Flórez (2024), donde la arquitectura previa concentraba mayor parte del procesamiento en una BeagleBone Black y un servidor físico remoto [Espressif Systems, 2026d, AWS, 2026g, Gómez Flórez, 2024].

2.5. MQTT, telemetría y contrato de datos

MQTT, Message Queuing Telemetry Transport, es un protocolo de mensajería de tipo publish/subscribe diseñado para escenarios donde usuarios o clientes publican mensajes a un broker mediante tópicos. OASIS (2019) define **MQTT** como un protocolo orientado a clientes ligeros, lo cual lo hace apropiado para sistemas IoT con recursos limitados. En este trabajo, la ESP3 opera como cliente **MQTT**, **AWS IoT Core** funciona como broker administrado y el mensaje **json** es la unidad de telemetría que viaja desde el prototipo hasta la nube. AWS (2026g) complementa este planteamiento al documentar que **AWS IoT Core** soporta comunicación segura de dispositivos mediante **MQTT** sobre **TLS** [OASIS, 2019, AWS, 2026g].

Para esta arquitectura, se usa la estructura de datos **json** como contrato de datos por varias razones. **MQTT** transporta mensajes de aplicación, pero no define por sí mismo la estructura interna del payload; por eso, se requiere un contrato estable para que el firmware y la nube interpreten los mismos campos. Por un lado, **json** es un formato legible y con facilidad de depuración durante pruebas. Por otra parte, es compatible con los servicios de AWS, por lo que esta estructura actúa como una interfaz estable entre el firmware del sistema físico, las reglas IoT, **DynamoDB**, **S3**, **Athena**, **Lambda**, Glue y la visualización. En este punto, AWS (2026k) permite reforzar el papel de **Lambda** como interfaz HTTP para entregar respuestas estructuradas hacia aplicaciones web, aunque la definición específica de nombres de campos, tipos de datos y unidades corresponde al diseño del contrato de datos de este proyecto. Por lo anterior, es importante garantizar la persistencia en los nombres de los campos, tipos de datos y unidades que deben permanecer consistentes [AWS, 2026h,k].

El contrato de datos planteado incorpora información de las variables físicas y metadatos del sistema. El NIST (2016) señala que los sistemas IoT deben transformar observaciones físicas en datos útiles para sistemas digitales, mientras que el NIST (2019) advierte la necesidad de conservar trazabilidad, integridad y gestión de riesgos sobre esa información. En este trabajo, las variables describen la dinámica física en función del tiempo: temperatura, velocidad del viento, velocidad de la turbina, humedad, voltaje y presión atmosférica. Los metadatos permiten

medir trazabilidad: `clientId`, `timestamp`, estado de conexión, estado de la memoria y otros campos que se contemplen para monitorear procesos clave de la ESP. Esta estructura permite analizar el comportamiento energético y operativo del sistema IoT [National Institute of Standards and Technology, 2016, 2019].

Una buena práctica de arquitectura es separar o distinguir entre el tópico **MQTT**, el identificador del dispositivo y la estructura del payload. OASIS (2019) permite sustentar que **MQTT** organiza la mensajería mediante tópicos, mientras que AWS (2026g) permite administrar la comunicación de dispositivos en **AWS IoT Core**. En este sentido, el tópico recibe y organiza el flujo de los mensajes, el `clientId` identifica la fuente de telemetría y el **json** estructura los valores medidos. Mezclar estas responsabilidades o no asignar una distinción clara tiende a dificultar el escalamiento a múltiples sistemas y complica las consultas históricas por dispositivo [OASIS, 2019, AWS, 2026g].

2.6. Seguridad: TLS, certificados X.509 y principio de mínimo privilegio

Al tratarse de una arquitectura donde no todas las comunicaciones son iguales, la seguridad debe analizarse por tramos. En el tramo ESP1–ESP2, red local soportada en **ESP-NOW**, se puede operar con cifrado mediante claves PMK/LMK configurando pares seguros y con una adecuada gestión de credenciales, direcciones y secretos para salvaguardar la seguridad de los nodos. Espressif Systems (2026d) documenta mecanismos de seguridad para proteger tramas en **ESP-NOW**. Sin embargo, el tramo ESP3–AWS funciona de forma diferente, ya que **AWS IoT Core** requiere autenticación de dispositivos a través de certificados **X.509** y comunicación **MQTT** segura sobre **TLS**. AWS (2026j) permite sustentar que los certificados **X.509** se usan para autenticar clientes en **AWS IoT Core**, mientras que AWS (2026i) establece que las comunicaciones hacia el Device Gateway deben cifrarse en tránsito mediante **TLS**. Esta separación permite diferenciar la seguridad de borde, la seguridad de transporte y la autorización en servicios cloud [Espressif Systems, 2026d, AWS, 2026j,i] [Malhotra et al., 2021, Sasi et al., 2024, Bakhshi et al., 2024, Liu et al., 2024].

La protección de los datos es fundamental en este tipo de sistemas. Rescorla (2018) explica que **TLS** protege la comunicación frente a escucha, manipulación y falsificación de mensajes

en el canal cliente-servidor. Por eso, exigir implementación de **SSL/TLS** no es un capricho, sino una necesidad técnica cuando la ESP3 se expone publicando datos hacia Internet. AWS (2026g) también sustenta esta necesidad al indicar que **AWS IoT Core** requiere cifrado en tránsito para los protocolos de aplicación soportados, como **MQTT** y **HTTP**. En este prototipo, la sincronización de hora juega un rol esencial, por eso se maneja un **DS3231** y se implementa la conexión con servidores de tiempo mediante **SNTP** desde la ESP3. Esto se requiere para establecer una conexión confiable con **AWS IoT Core** a través de **TLS/X.509** y para conservar trazabilidad temporal de los datos [Rescorla, 2018, AWS, 2026g].

Adicionalmente, el uso de certificados **X.509** permite que cada dispositivo tenga una identidad criptográfica propia otorgada por **AWS IoT Core** y embebida mediante el firmware en la ESP. AWS (2026j) permite sustentar que **AWS IoT Core** valida los certificados **X.509** durante la autenticación del cliente. Esto incrementa la seguridad del sistema, pero también agrega complejidad y responsabilidades adicionales relacionadas con el manejo adecuado de certificados, rotación, versionamiento, manejo seguro durante pruebas, monitoreo del funcionamiento adecuado del **DS3231** y actualizaciones propias de **esp-idf**. El NIST (2019) complementa este punto al advertir que los sistemas IoT deben gestionar riesgos de seguridad desde el diseño y durante todo su ciclo de vida [AWS, 2026j, National Institute of Standards and Technology, 2019].

En AWS, el principio de mínimo privilegio pretende que la política IoT permita únicamente las acciones necesarias para el dispositivo, tales como conexión o publicación en tópicos específicos. AWS (2026f) plantea que **AWS IoT Core** usa políticas para autorizar acciones sobre recursos determinados. Las buenas prácticas en la asignación de permisos garantizan que cada función o rol asignado haga lo que le corresponde, cumpliendo la función asignada sin comprometer la seguridad de los demás elementos y dejando la puerta abierta para escalar a más dispositivos sin dificultad para la asignación de políticas [AWS, 2026f,i].

2.7. Arquitecturas serverless y Big Data para telemetría IoT

Una arquitectura **serverless**, o sin servidor, es una forma de desarrollo en la nube en la cual el cliente se enfoca principalmente en su solución debido a que la administración de la infraestructura es responsabilidad del proveedor de servicios. AWS (2026n) plantea que este enfoque permite diseñar, desplegar y operar cargas **serverless** en la nube sin administrar servidores dedicados. Para este proyecto, la arquitectura cloud usa **AWS IoT Core**, **Lambda**, **DynamoDB**, **S3**, **Glue**,

Athena y **CloudFront** para recibir, procesar, almacenar, consultar y visualizar datos sin necesidad de desplegar una instancia EC2. Esta decisión se articula con el propósito planteado desde el anteproyecto, donde se buscaba migrar desde una arquitectura previa basada en servidor físico hacia una arquitectura soportada en servicios de nube [AWS, 2026n,g].

La visualización o monitoreo se separa en dos etapas que impactan directamente la arquitectura. Por una parte, la obtención del “último dato” es una decisión central, que permite obtener el último dato reportado a AWS por cualquier turbina mediante `clientId` para visualizarse en cards que se actualizan cada 30 segundos. AWS (2026g) permite sustentar la recepción de la telemetría mediante **AWS IoT Core**, mientras que AWS (2026c) permite sustentar el uso de **DynamoDB** para organizar datos en tablas, ítems, atributos y claves primarias. Por otra parte, se establece el manejo del “histórico” a través de **S3**, Glue y **Athena**, el cual responde a la necesidad de consulta del usuario en la capa de visualización, por ejemplo, última hora o últimas 24 horas. AWS (2026b) permite sustentar la necesidad de particionar datos en **Athena** para mejorar la eficiencia de las consultas, AWS (2026a) explica la relación entre **Athena** y Glue Data Catalog, y AWS (2026k) permite sustentar el uso de **Lambda** como capa de consulta. Esta separación permite optimizar costos, latencia y carga de la aplicación [AWS, 2026g,c,a].

El uso de **S3**, Glue y **Athena** convierte la telemetría proveniente de la ESP3 en un repositorio confiable y escalable que se puede consultar mediante SQL. Sin embargo, debe tenerse presente que para que esta ruta sea eficiente se deben estructurar los **json** recibidos en **AWS IoT Core** en particiones temporales, controlar el tamaño de los objetos, limitar los rangos consultados y controlar la cantidad de consultas al día, dado que en esta arquitectura se paga por uso y el mayor consumo puede concentrarse en las consultas históricas. AWS (2026m) explica que los prefijos permiten organizar objetos en **S3** de forma jerárquica, AWS (2026a) permite sustentar la catalogación de metadatos con Glue Data Catalog y AWS (2026b) explica que el particionamiento permite reducir datos escaneados cuando las consultas filtran por claves de partición. Esta lógica también permite proyectar el sistema hacia escenarios de **Big Data**, no porque el prototipo inicial ya produzca grandes volúmenes de información, sino porque la arquitectura se diseña para soportar crecimiento futuro en cantidad de dispositivos, frecuencia de publicación y volumen histórico [AWS, 2026m,a,b].

Para interconectar la sección de almacenamiento con el frontend de visualización se implementan funciones **Lambda** de AWS, las cuales permiten ejecutar código bajo demanda y actuar como

intermediarias detrás de cada solicitud del usuario. AWS (2026k) describe **Lambda** Function URL como una función integrada que permite exponer una función **Lambda** mediante HTTPS, actuando como una API ligera para el dashboard. En este trabajo, esto limita que el dashboard consulte directamente las fuentes de datos, como **DynamoDB** o **S3**. En lugar de ello, el usuario realiza peticiones HTTP de tipo GET desde el dashboard a la URL de la función **Lambda**, y esta centraliza la validación de parámetros, la consulta específica a **S3** mediante **Athena** y Glue, la normalización de la respuesta en estructura **json** y la aplicación de políticas de acceso desde el navegador mediante **CORS**. AWS (2026l) permite sustentar la importancia de **CORS** para controlar las solicitudes entre diferentes orígenes en aplicaciones web [AWS, 2026k,l].

2.8. Computación en el borde y gestión de concurrencia

Edge Computing, o computación en el borde, es una práctica que permite ejecutar parte del procesamiento en la fuente u origen de los datos, en este caso, en las tarjetas **ESP32-C6** que miden las variables del proceso. El NIST (2016) permite sustentar esta idea al considerar que los sistemas IoT no solo capturan datos, sino que también pueden realizar procesamiento local antes de enviar información a servicios externos. Esto reduce la dependencia de conectividad permanente y evita enviar datos desorganizados a la nube o a un servidor remoto. El firmware que se crea para el proceso local permite que estas tarjetas se comuniquen con los sensores, ejecuten los algoritmos de lectura de datos, cuenten pulsos combinando interrupciones y temporizadores de alta precisión, consoliden los datos en múltiples estructuras, serialicen la información y la publiquen en AWS, facilitando el procesamiento y la integridad del proceso. Espressif Systems (2026e) permite sustentar el uso de **FreeRTOS** en **esp-idf** para manejar tareas, sincronización y concurrencia en dispositivos **ESP32-C6** [National Institute of Standards and Technology, 2016, Espressif Systems, 2026f] [Sisinni et al., 2018, Atzori et al., 2010].

Las tarjetas ESP32 se definen como SoC, System on Chip, y pueden programarse de múltiples maneras, principalmente a partir de su entorno nativo **esp-idf** con lenguaje C y C++, aunque también existen alternativas como MicroPython. Espressif Systems (2026c) documenta que **esp-idf** usa **FreeRTOS** y que, para **ESP32-C6**, se trabaja en modo de un solo núcleo para la ejecución principal. Estas tarjetas se apoyan en **FreeRTOS** como kernel integrado de tiempo real, lo que permite trabajar con una arquitectura de firmware soportada en tareas concurrentes, temporizadores, callbacks e ISR (Interrupt Service Routine), con control explícito de prioridades.

Esta capacidad es importante en estos sistemas IoT porque deben atender múltiples eventos al mismo tiempo sin bloquear el sistema [Espressif Systems, 2026f,h] [Al-Fuqaha et al., 2015, Sisinni et al., 2018, Khan et al., 2012].

En la ESP1 se realiza el conteo de pulsos para medir la velocidad de la turbina, el muestreo de voltajes para asociar una salida eléctrica a las condiciones de operación, la conexión con el módulo **GPS** y la sincronización con **ESP-NOW**. En la ESP2 se realiza la recepción de la trama **ESP-NOW**, la lectura del **BME680**, **DS3231** y pantalla **OLED** a través de un único bus **I2C**, el almacenamiento en memoria **SD** de diez muestras a través de **SPI**, la consolidación del paquete **json** completo y la transmisión por **UART**. En la ESP3 se realiza el ensamblado del **json**, la conexión a **Wi-Fi**, el despliegue de **MQTT** sobre **SSL/TLS** y el envío hacia **AWS**. Espressif Systems (2026f) sustenta el uso del controlador **I2C** para comunicar dispositivos en un bus compartido, mientras que Espressif Systems (2026h) soporta la comunicación **UART** entre nodos cercanos. El NIST (2019) permite justificar que la gestión de estos procesos debe considerar la integridad de los datos y la reducción de fallos operativos [Espressif Systems, 2026g,l, National Institute of Standards and Technology, 2019].

Para lograrlo de forma robusta, se usan funciones de **FreeRTOS** que permiten desacoplar la captura de eventos de la lógica de aplicación mediante colas. En este esquema, los callbacks o ISR solo empujan datos y estos se procesan a través de una tarea dedicada. También se usan semáforos para señalar disponibilidad de nuevas muestras y mutex para proteger estructuras compartidas, evitando condiciones de carrera y lecturas inconsistentes. Así, a través de esta arquitectura se logra consistencia operativa, reducción de pérdida de eventos y sostenimiento de las comunicaciones aun bajo carga, mostrando que el SoC **ESP32-C6** puede orquestar pipelines de adquisición, preprocesamiento, transmisión segura y escalable dentro de un entorno RTOS. Esta solución representa una evolución frente al trabajo de Gómez Flórez (2024), donde la adquisición y procesamiento se apoyaban en una arquitectura con BeagleBone Black y servidor remoto, mientras que en este nuevo diseño se distribuye el procesamiento en varios nodos de bajo costo [Espressif Systems, 2026f, Gómez Flórez, 2024].

Por último, la gestión de concurrencia también influye en la calidad de los datos. Wang et al. (2023) permiten relacionar el monitoreo de variables eólicas con la necesidad de contar con datos operativos y ambientales que puedan alimentar análisis posteriores en aplicaciones IoT. Si las variables se miden en escalas temporales diferentes, debe indicarse claramente su manejo y

métodos de operacionalización para no generar relaciones equivocadas. En este caso, el sistema toma muestras cada segundo, almacena diez muestras en estructura relacional CSV en la memoria local **SD** y promedia las que operan bajo esta lógica numérica, generando cada diez segundos un **json** donde todas sus variables están estructuralmente relacionadas en el tiempo. El NIST (2016) permite sustentar esta decisión porque en sistemas IoT la información debe organizarse de forma que pueda ser explotada posteriormente por servicios digitales [Wang et al., 2023, National Institute of Standards and Technology, 2016] [Tchakoua et al., 2014, Akyuz and demircan, 2019, Pryor et al., 2020].

2.9. Síntesis del marco de referencia

Este marco de referencia construido permite justificar la arquitectura propuesta como una integración entre múltiples tecnologías de la Industria 4.0. El NIST (2016) permite entender esta integración como una relación entre dispositivos físicos, comunicación, procesamiento y servicios digitales. En este trabajo se manejan sistemas físicos contruidos con impresión 3D, se integran sistemas embebidos para IoT, sensórica, comunicación local, múltiples protocolos de comunicación, seguridad criptográfica y cloud computing a partir de servicios **serverless** de AWS para soportar toda la infraestructura de datos. AWS (2026n) permite sustentar el enfoque **serverless** de la arquitectura, AWS (2026k) permite sustentar el uso de funciones **Lambda** como intermediarias entre almacenamiento y visualización, y AWS (2026a) permite explicar el papel del catálogo de datos cuando **Athena** consulta información almacenada en **S3**. Lo anterior manifiesta una fortaleza de este enfoque: cada etapa tiene responsabilidades delimitadas, como medición, transmisión, integración, publicación, almacenamiento, consulta y visualización. Esto muestra cómo se estructura un pipeline completo de datos, desde la adquisición hasta su visualización o modelamiento a partir de la estructura histórica [National Institute of Standards and Technology, 2016, AWS, 2026n,k] [Botta et al., 2016, Marjani et al., 2017, R., Preetha et al., 2023, Alwahedi et al., 2024] [Irawan et al., 2023] [Yang et al., 2023] [Du et al., 2019] [Bassett et al., 2015] [Alami et al., 2023].

A partir de la base teórica presentada, el siguiente capítulo describirá con mayor detalle la arquitectura de hardware **IoT Edge**. Se abordará qué mide cada ESP, por qué se hace así, cuáles interfaces usa, cómo se estructuran las tareas, qué datos se transmiten entre nodos y cómo se valida cada tramo antes de publicar en AWS. Espressif Systems (2026d) permite sustentar la

parte relacionada con comunicación local mediante **ESP-NOW**, mientras que AWS (2026g) permite sustentar la comunicación segura entre el gateway y **AWS IoT Core**. Esta secuencia permite conectar con precisión el marco conceptual elaborado con la implementación real del prototipo, manteniendo continuidad con la tesis raíz de Gómez Flórez (2024), pero proponiendo una arquitectura más distribuida, económica y orientada a servicios cloud [Espressif Systems, 2026d, AWS, 2026g, Gómez Flórez, 2024].

Diseño e implementación de la arquitectura de hardware IoT Edge

Este capítulo desarrolla con mayor profundidad la arquitectura de hardware **IoT Edge** propuesta e implementada para el monitoreo permanente de las variables asociadas con el proceso de un generador eólico de eje vertical de pequeña escala prototipado previamente. La capa Edge, en general se entiende como el conjunto de sistemas embebidos, sensores, periféricos (**DS3231**, **SD**, Pantalla **OLED**), tareas de firmware y todos aquellos procesos que permiten convertir las señales medidas en telemetría estructurada para ser publicada en **AWS IoT Core**. En el prototipo de este trabajo, esta capa se implementa a partir de tres tarjetas **ESP32-C6** con funciones diferenciadas. ESP1 como nodo que mide variables del generador y transmite mediante **ESP-NOW**. ESP2 como nodo que mide variables ambientales, recibe **ESP-NOW** de ESP1, estructura en **json** y serializa. Y ESP3 como nodo gateway **MQTTs** hacia **AWS IoT Core** [National Institute of Standards and Technology, 2016, Gómez Flórez, 2024, Espressif Systems, 2023, 2026d, Bray, 2017, AWS, 2026g].

La decisión de distribuir la arquitectura en tres nodos responde a un criterio de ingeniería tras surgir esta necesidad en el proceso de implementación. Inicialmente, todo se realizaba con una única ESP. Sin embargo, al llegar al límite se encontró que algunos procesos se debían separar para garantizar la funcionalidad completa de la tarjeta. Al identificar estas limitaciones, se dividió la arquitectura en los tres nodos propuestos, permitiendo un mejor balance de carga y aprovechamiento de las funciones de cada tarjeta, aislar fallas de sensórica de fallas de conectividad y mejorar la trazabilidad de toda la arquitectura [National Institute of Standards and Technology, 2016, Espressif Systems, 2026f, National Institute of Standards and Technology, 2019, Sisinni

et al., 2018, Marjani et al., 2017].

Arquitectura IoT Edge basada en tres ESP32-C6

Adquisición de campo, integración local, serialización JSON y puente UART/MQTT hacia AWS

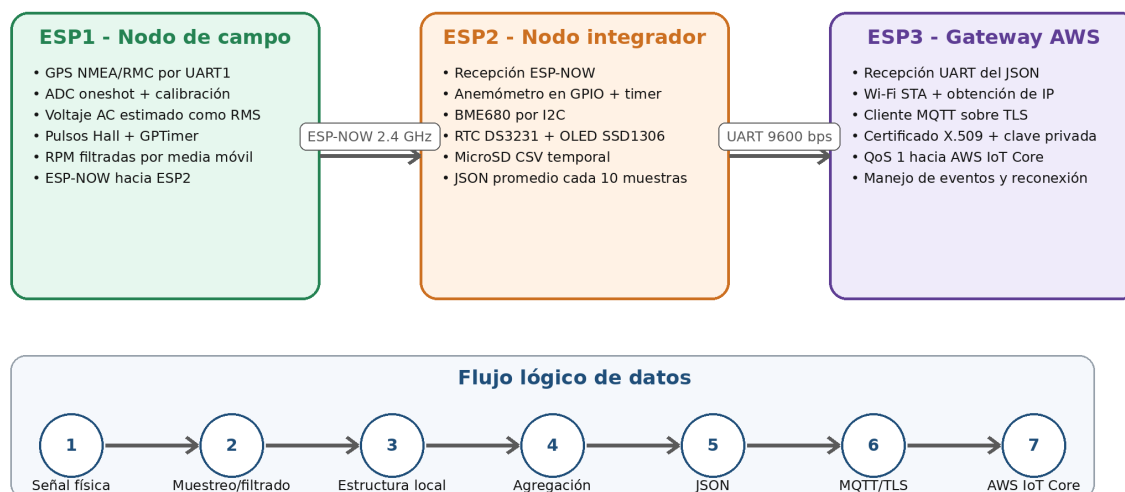


Figura nro. 3.1: Arquitectura IoT Edge basada en tres ESP32-C6: nodo de campo, nodo integrador y gateway AWS. Fuente: elaboración propia con apoyo de ChatGPT, modelo GPT-5.5 Thinking de OpenAI, 2026.

La Figura nro. 3.1 resume la arquitectura propuesta. En el nodo 1 (ESP1) se convierten señales de GPS, voltaje y RPM de la turbina en una estructura local. En el nodo 2 (ESP2) se agrega velocidad del viento, temperatura, presión, humedad, altitud, tiempo, visualización **OLED** y almacenamiento temporal en tarjeta **SD** y se transmite **json** consolidado mediante **UART**. En el nodo 3 (ESP3) se recibe el **json** por **UART**, se establece conectividad **Wi-Fi** y publica la telemetría por **MQTT** sobre **TLS** en **AWS IoT Core**. Por ende, el sistema no es únicamente una tarjeta con sensores conectados, sino un pipeline distribuido completo que garantiza desde la adquisición hasta la transmisión segura de la información recolectada [National Institute of Standards and Technology, 2016, Gómez Flórez, 2024, Espressif Systems, 2026l, AWS, 2026g, Rescorla, 2018].

3.1. Propósito de la capa **IoT Edge**

La capa **IoT Edge** tiene como función principal acercar el procesamiento de los datos al punto físico donde se miden las señales. En este proyecto, las medidas nacen de la necesidad de analizar el funcionamiento de un generador eólico de eje vertical de pequeña escala, así que se miden pulsos asociados a la rotación de la turbina, voltaje generado, coordenadas geográficas, velocidad del viento, temperatura, presión atmosférica, humedad y altitud. Si estas señales se envían hacia la nube sin procesamiento, muy seguramente podría recibir datos incompletos, no se sabría a qué timestamp pertenece ese dato y todo sería un caos. Por ello, el Edge cumple funciones de adquisición, limpieza, filtrado, agregación, almacenamiento temporal, serialización y control básico de la integridad de los datos, lo que permite tener una arquitectura más fiable, estable y económica [National Institute of Standards and Technology, 2016, Botta et al., 2016, Gómez Flórez, 2024, Marjani et al., 2017, National Institute of Standards and Technology, 2019].

Desde la perspectiva de hardware/sensado como servicio, la capa **IoT Edge** no se limita a conectar sensores a tarjetas ESP32-C6. Su función es convertir mediciones físicas en datos digitales organizados, trazables y listos para ser transmitidos. Por ello, el valor de esta capa no está solo en el dispositivo, sino en su capacidad para entregar telemetría confiable al resto de la arquitectura [National Institute of Standards and Technology, 2016, 2019, Espressif Systems, 2023, 2026f].

Con base en lo anterior, la capa **Edge** se encarga de definir una estructura o contrato de datos común para el sistema, de manera que las variables físicas medidas puedan convertirse en telemetría estructurada, trazable y procesable por los servicios posteriores. El voltaje se mide capturando muestras mediante **ADC**, la velocidad de la turbina se mide capturando pulsos mediante interrupciones y temporizadores precisos, la localización se obtiene desde tramas **GPS** de un módulo específico, la velocidad del viento se obtiene también mediante interrupciones y temporizadores, y las variables ambientales se obtienen mediante **I2C**. Finalmente, el mensaje se normaliza en **json**, una estructura común utilizada por los servicios de AWS, dado que las reglas IoT, las tablas, las consultas y el dashboard dependen directamente de nombres de campos estables, tipos de datos consistentes y unidades claramente documentadas [Marjani et al., 2017, Espressif Systems, 2026a,c, National Marine Electronics Association, 2018, Espressif Systems, 2026g, AWS, 2026h].

La capa **Edge** también cumple la función de actuar como primera frontera de confiabilidad. Antes

de que los datos lleguen a **AWS** se deben resolver problemas de integridad, rebote de sensores, pérdida de paquetes **ESP-NOW**, desconexión del **GPS**, errores de **UART**, fallos en el bus **I2C**, pérdida de conectividad Wi-Fi, entre otros factores. Por esta razón, el reto no consiste únicamente en leer sensores; debe incluirse temporización, gestión de tareas recurrentes, indicadores de estado del sistema, validación de estructura y registros de diagnóstico [National Institute of Standards and Technology, 2019, Espressif Systems, 2026d,l,g,f,n].

3.2. Requisitos funcionales de la arquitectura embebida

Los requisitos funcionales de la capa de hardware surgen del objetivo general del sistema, el cual consiste en medir una serie de variables físicas que permiten monitorear el funcionamiento de un generador eólico de eje vertical de pequeña escala y publicar los datos de forma trazable. Además, debe entregar estos datos al gateway en un formato compatible con **MQTT** y con la posterior ingesta en **AWS IoT Core** [National Institute of Standards and Technology, 2016, Wang et al., 2023, Gómez Flórez, 2024, OASIS, 2019, AWS, 2026g].

La tabla 3.1 presenta un resumen de los requisitos funcionales del sistema y los relaciona con el nodo responsable, mecanismos implementados y salidas esperadas. De esta manera, se establece una relación entre el requerimiento funcional y el código fuente ya que permite verificar que cada variable tenga un origen físico, un tratamiento local diferenciado y una salida normalizada. También, ayuda a identificar vacíos en la validación y trazabilidad de cada variable, lo cual es esencial porque permite que la arquitectura sea más robusta y confiable [ISO/IEC/IEEE, 2018, National Institute of Standards and Technology, 2016, 2019, Espressif Systems, 2026f].

Tabla nro. 3.1: Requisitos funcionales de la arquitectura embebida IoT Edge.

Requisito funcional	Nodo responsable	Mecanismo	Evidencia esperada
		implementado	
Capturar voltaje del generador	ESP1	ADC oneshot, calibración y cálculo RMS	Variable <code>voltaje_ac</code> en estructura ESP-NOW
Estimar RPM del eje	ESP1	GPIO con interrupción y GPTimer periódico	Variable <code>rpm</code> filtrada

Requisito funcional	Nodo responsable	Mecanismo implementado	Evidencia esperada
Obtener coordenadas	ESP1	GPS por UART, sentencia RMC y conversión a decimal	latitud, longitud, gps_fix
Medir velocidad del viento	ESP2	GPIO del anemómetro + GPTimer	wind_speed en json final
Medir variables ambientales	ESP2	BME680 por I2C	temperature, humidity, pressure, altitude
Trazabilidad temporal	ESP2	RTC DS3231 por I2C	timestamp en json
Visualización local	ESP2	OLED SSD1306 por I2C	Pantallas rotativas de estado
Respaldo local	ESP2	MicroSD por SPI/FATFS	Archivo data.csv
Publicación cloud	ESP3	UART + Wi-Fi + MQTT/TLS	Publicación en tópico IoT

Adicional a los requisitos funcionales, la arquitectura debe cumplir con requisitos de operación dado que requiere ejecutar tareas concurrentes sin bloquear mediciones críticas o procesos que se estén ejecutando, mantener un canal **ESP-NOW** de baja latencia entre ESP1 y ESP2, evitar publicar datos sin la estructura **json** mínima y registrar la trazabilidad de los eventos de conectividad. Muchos de estos requisitos se fueron estructurando conforme se conocía con más profundidad las capacidades de las ESP y son coherentes con la naturaleza de **esp-idf**, para lo cual se usa FreeRTOS para organizar tareas, priorizar eventos, manejar prioridades y a su vez, con **MQTTs** se separa cliente, broker, tópico y calidad del servicio QoS [ISO/IEC/IEEE, 2018, Espressif Systems, 2026f,d, OASIS, 2019, AWS, 2026g, National Institute of Standards and Technology, 2019].

3.3. Trazabilidad del código fuente usado para el capítulo

El análisis técnico de este capítulo se soporta en tres estructuras de firmware, una para cada ESP. Las carpetas se presentan rotuladas como Firmware-ESP1, Firmware-ESP2, Firmware-ESP3. Para mantener la trazabilidad académica, la Tabla. 3.2 documenta los archivos principales generados, su función y el hash SHA-256 calculado a partir de los archivos entregados [National Institute of Standards and Technology, 2019, ISO/IEC/IEEE, 2018, Espressif Systems, 2026f, National Institute of Standards and Technology, 2015].

Tabla nro. 3.2: Trazabilidad de archivos fuente analizados para la arquitectura IoT Edge.

Etapas	Carpeta	Archivo	Función técnica	SHA-256
		principal	dominante	
ESP1	Firmware-ESP1	main.c	GPS, ADC, RPM y ESP-NOW	f2eb15...
ESP2	Firmware-ESP2	main.c	Integración, BME680, OLED, SD, UART y ESP-NOW	ef4034...
ESP3	Firmware-ESP3	main.c	UART, Wi-Fi, MQTT/TLS y AWS IoT Core	c5cccd...

Con la tabla anterior no se pretende reemplazar los anexos de código, busca que el lector conozca cuál versión del firmware se soporta en la descripción técnica. Esto es importante porque permite controlar la trazabilidad de cada etapa, lo cual es importante cuando los proyectos tienden a escalar con el tiempo. Al incluir hashes, se pretende evitar que la tesis describa una versión diferente del firmware, lo cual permite el mejoramiento de la documentación a futuro [IEEE Computer Society, 2012, ISO/IEC/IEEE, 2018, National Institute of Standards and Technology, 2015, 2019].

3.4. Arquitectura física y asignación de responsabilidades

Como se ha planteado, la arquitectura física se distribuye en tres tarjetas **ESP32-C6**. La ESP1 se ubica espacialmente cerca al generador eléctrico y los sensores que requieren medición directa (voltaje del generador, velocidad de rotación de la turbina). La ESP2 actúa como nodo central, recibe datos de la ESP1, mide velocidad del viento y variables del sensor **BME680**, almacena en memoria **SD**, lee timestamp desde el módulo **DS3231**, visualiza en **OLED**, concentra y estructura los datos en **json** y transmite mediante **UART**. La ESP3 se dedica principalmente a la conectividad con la nube, lo que permite aislar **Wi-Fi**, certificados y **MQTTs** de las tareas de adquisición de datos [Gómez Flórez, 2024, Wang et al., 2023, Espressif Systems, 2026f, Bosch Sensortec, 2024, Espressif Systems, 2026l, AWS, 2026g].

La división por funciones tiene varias ventajas prácticas que favorecen la modularidad. Por un lado, si la conexión a AWS falla, el sistema puede continuar midiendo y almacenando datos localmente para después ser enviados a la nube. Si falla **ESP-NOW**, el **json** puede reflejar un estado de desconexión y usar valores predeterminados o inferir mediante procesos para datos faltantes. Si alguno de los sensores falla, el sistema continúa operando y solo reporta la falla. Esta flexibilidad es deseable en estos sistemas IoT que operan con recursos limitados y comunicaciones que pueden ser intermitentes [National Institute of Standards and Technology, 2016, 2019, Espressif Systems, 2026d, Marjani et al., 2017, Gómez Flórez, 2024].

La división de la arquitectura por funciones tiene varias ventajas prácticas que favorecen la modularidad. Por un lado, si la conexión a AWS falla, el sistema puede continuar midiendo y almacenando datos localmente para después ser enviados a la nube. Por otra parte, si falla **ESP-NOW**, el **json** puede reflejar un estado de desconexión y usar valores predeterminados o inferir valores mediante procesos de imputación de datos. Si alguno de los sensores falla, se reporta la anomalía y puede continuar operando normalmente. Estos sistemas flexibles son deseables en arquitecturas IoT, ya que suele operarse con recursos limitados y comunicaciones que pueden ser intermitentes [National Institute of Standards and Technology, 2016, 2019, Espressif Systems, 2026d, Marjani et al., 2017, Sisinni et al., 2018].

Tabla nro. 3.3: Distribución física y lógica de responsabilidades entre las tres ESP32-C6.

Nodo	Rol	Entradas principales	Salidas principales	Protocolos/periféricos
ESP1	Nodo de campo	GPS, ADC de voltaje, pulsos Hall	Estructura binaria a ESP2	UART, ADC, GPIO, GPTimer, ESP-NOW
ESP2	Nodo integrador	ESP-NOW, anemómetro, BME680, DS3231	CSV local, OLED, json por UART	ESP-NOW, GPIO, GPTimer, I2C, SPI, UART
ESP3	Gateway cloud	json por UART	Mensaje MQTT/TLS	UART, Wi-Fi, TCP/TLS, MQTT

En la tabla 3.3 se muestra que en cada nodo se combinan diferentes periféricos. La ESP1 se enfoca en adquisición y transmisión local. La ESP2 enfatiza en integración de múltiples protocolos y periféricos. Y la ESP3 se concentra en conectividad segura con la nube. Por consiguiente, cada nodo debe probarse con criterios distintos, asegurando estabilidad para cada una de las etapas y que cada una cumpla a cabalidad los requerimientos planteados [National Institute of Standards and Technology, 2016, ISO/IEC/IEEE, 2018, Espressif Systems, 2026f,l, AWS, 2026g].

3.5. Modelo de capas aplicado al hardware **IoT Edge**

El sistema embebido no implementa TCP/IP en todos sus tramos. Entre sensores y microcontroladores predominan señales físicas, GPIO, ADC, **I2C**, **SPI** y **UART**; entre ESP1 y ESP2 se usa **ESP-NOW**, que funciona sobre tramas **Wi-Fi** de capa de enlace sin requerir direccionamiento IP; entre ESP2 y ESP3 se usa **UART**, que es un canal serie punto a punto; y solo entre ESP3 y AWS aparece el stack **Wi-Fi/IP/TCP/TLS/MQTT** [Espressif Systems, 2026g,l,d, Eddy, 2022b, Rescorla, 2018, OASIS, 2019]

La estructura por capas presentada evita una confusión que es recurrente, y es que en IoT no todos los sensores o actuadores del sistema están directamente conectados a Internet y se pueden

plantear múltiples arquitecturas y formas de establecer las comunicaciones según las necesidades del sistema. Como se evidencia en este sistema la mayoría de mediciones se desarrollan en tramos locales no IP. Internet aparece al final, cuando la ESP3 transforma el json recibido por **UART** desde la ESP2 en un mensaje **MQTT** cifrado sobre **TLS**, requerido por AWS para establecer una conexión segura. Por ejemplo, esta separación permite separar las responsabilidades de los nodos y concentrar en un único nodo la seguridad de la nube [National Institute of Standards and Technology, 2016, Espressif Systems, 2026l, OASIS, 2019, Rescorla, 2018, AWS, 2026g,i].

Tabla nro. 3.4: Modelo por capas aplicado a los tramos del sistema IoT Edge.

Tramo	Capa física	Enlace	Red/transporte	Aplicación/datos
Sensores -> ESP1	Señales analógicas, pulsos, UART GPS	GPIO/ADC/UART	No aplica	Variables físicas locales
ESP1 -> ESP2	Radio 2.4 GHz	ESP-NOW/MAC	No usa IP	Estructura mensaje_espnw_t
Sensores -> ESP2	I2C, GPIO, SPI	Buses locales	No aplica	BME680, DS3231, OLED, SD
ESP2 -> ESP3	UART 9600 bps	Serial punto a punto	No usa IP	json terminado en \n
ESP3 -> AWS	Wi-Fi 802.11	IP/TCP/TLS	MQTT sobre TLS	Publicación en tópico IoT

La tabla 3.4 permite evidenciar la necesidad de implementar controles de seguridad por tramo, ya que en cada nodo se manejan protocolos diferentes y los riesgos varían notablemente. En el tramo 1 (ESP1-ESP2), es necesario proteger **ESP-NOW** a nivel de par y clave, además de proteger las MACs y tener estrategias contra la suplantación de estas. Por otra parte, en el nodo dos deben controlarse las distancias y el manejo del bus **I2C**, además, en **UART** debe controlarse el tamaño de la trama, establecer delimitadores y validación de la estructura. En el nodo tres, **MQTTs** debe protegerse mediante certificados **X.509** otorgados por AWS, políticas IoT y sincronización de tiempo (SNTP). Como se aprecia, cada protocolo suele tener sus propios controles diseñados para

riesgos identificados y esto hace que cada vez la seguridad sea algo más necesario y articulado [National Institute of Standards and Technology, 2019, Espressif Systems, 2026d,g,l, AWS, 2026j,f].

3.6. ESP1: nodo de adquisición de campo

La ESP1 cumple el rol de nodo de adquisición a nivel del generador. En el archivo Firmware-ESP1/main/main.c se observa una estructura basada en tres tareas principales `gps_task`, `voltage_task` y `espnw_task`, acompañadas por una interrupción GPIO para pulsos y un GPTimer para cálculo periódico de RPM. La estructura compartida `mensaje_t` contiene latitud, longitud, `voltage_ac`, `rpm` y `gps_fix`, que son las variables enviadas hacia ESP2 [Espressif Systems, 2026f,b,c,a, National Marine Electronics Association, 2018, Espressif Systems, 2026d].

En la tabla 3.5 se presenta la asignación de pines y periféricos utilizados. El **GPS** (módulo NEO-6M) se usa `UART_NUM_1` con RX en GPIO20 y TX en GPIO21, el voltaje usa `ADC_CHANNEL_0` y se toman decenas o cientos de muestras por segundo, el sensor de pulsos se conecta al GPIO22 y el LED de estado al GPIO2. La comunicación **ESP-NOW** se soporta en las direcciones MAC de cada ESP, en este caso, apunta hacia la dirección **MAC** de la ESP-2 **40:4C:CA:42:13:00**, mientras se configura el canal **Wi-Fi** en “1” para mantener la coherencia con el receptor [National Marine Electronics Association, 2018, Espressif Systems, 2026l,a,b,d, IEEE Standards Association, 2018].

Tabla nro. 3.5: Configuración de hardware identificada en ESP1.

Elemento	Configuración en ESP1	Función
UART GPS	UART_NUM_1, RX GPIO20, TX GPIO21, 9600 bps	Lectura de tramas NMEA/RMC
ADC voltaje	ADC_CHANNEL_0, 12 bits, atenuación 12 dB	Lectura de señal acondicionada
Sensor Hall	GPIO22, interrupción por flanco negativo	Conteo de pulsos del eje
LED estado	GPIO2	Indicación visual de envío
GPTimer	Resolución de 1 MHz, alarma cada 1 s	Cálculo periódico de RPM

Elemento	Configuración en ESP1	Función
ESP-NOW	Canal 1, peer MAC ESP2	Envío de mensaje_t

3.6.1. Adquisición GPS en ESP1

En este proyecto se añade un módulo **GPS**, el cual permite principalmente monitorear la ubicación del dispositivo y se pueden extraer datos adicionales. Este proceso se soporta en el módulo comercial NEO-6M y en la librería minmea.h. La adquisición se implementa mediante la tarea en FreeRTOS `gps_task`, con prioridad adecuada y que instala el driver **UART**, configura velocidad de 9600 bps y reconstruye líneas NMEA hasta encontrar saltos de línea. El firmware filtra sentencias RMC mediante `minmea_sentence_id` y extrae latitud, longitud y estado de validez con `minmea_parse_rmc`. La sentencia RMC es adecuada porque contiene información mínima recomendada de posición, tiempo, fecha, curso y velocidad, y la transmisión NMEA se basa en mensajes ASCII delimitados por caracteres especiales [National Marine Electronics Association, 2018, Espressif Systems, 2026l,f, National Institute of Standards and Technology, 2019].

Dado que los sistemas de almacenamiento, visualización y análisis geográfico suelen consumir coordenadas en grados decimales y no en formato ddmm.mmmm o dddmm.mmmm, se hace necesario un proceso de transformación de coordenadas. En este firmware, se hace mediante la función `convertir_a_decimal`, que transforma el formato NMEA de grados y minutos en grados decimales. Si la dirección es sur o oeste, el código cambia el signo de la coordenada [National Marine Electronics Association, 2018, Butler et al., 2016].

Matemáticamente, la conversión usada por el firmware puede expresarse como:

$$\theta_{\text{dec}} = \left\lfloor \frac{\theta_{\text{nmea}}}{100} \right\rfloor + \frac{\theta_{\text{nmea}} - 100 \lfloor \theta_{\text{nmea}}/100 \rfloor}{60}.$$

Donde,

θ_{nmea} representa la latitud o longitud codificada en grados y minutos,

θ_{dec} representa la coordenada en grados decimales. El signo final depende del hemisferio reportado por el receptor **GPS**: norte y este se toman como positivos; sur y oeste se toman como negativos

Esta transformación debe documentarse porque cualquier error de signo produciría una ubicación geográfica incorrecta en la base histórica [National Marine Electronics Association, 2018, Butler et al., 2016, National Institute of Standards and Technology, 2019].

3.6.2. Medición de voltaje en ESP1

La medición de voltaje se implementa mediante la tarea de FreeRTOS `voltage_task` mediante el driver ADC oneshot y el esquema de calibración `adc_cali_create_scheme_curve_fitting`. En este proceso se configura el canal a 12 bits, atenuación de 12 dB, se toman 200 muestras por segundo, se elimina un offset de 1.65 V asociado al acondicionamiento del voltaje a medir y se hace una estimación del voltaje RMS a partir de las muestras válidas y la aplicación de una media móvil de los últimos cinco valores para tratar con outliers instantáneos. El uso de calibración ADC es importante porque la conversión directa de conteos crudos a voltaje puede introducir error si no se consideran características internas del chip [Espressif Systems, 2026f,a, Gómez Flórez, 2024, National Institute of Standards and Technology, 2019].

El cálculo RMS aplicado por la ESP1 se puede representar como:

$$V_{\text{rms}} = K \sqrt{\frac{1}{N_v} \sum_{i=1}^{N_v} (v_i - V_{\text{off}})^2},$$

donde,

V_{off} es el offset del circuito de acondicionamiento,

N_v es el número de muestras válidas,

K representa el factor de escala del divisor o acondicionador

En el código, `offset = 1.65f`, `num_muestras = 200` y `voltage_divider_ratio = 2.0f`; por tanto, debe tenerse presente el circuito real de acondicionamiento y el procedimiento experimental usado para ajustar K .

La etapa de medición de voltaje tiene un punto crítico, y es que se está midiendo voltaje sinusoidal de muy pequeña amplitud (~300 mV). Para medir voltaje tan pequeños, se optó por una metodología en la cual se desplaza la señal negativa de la onda hacia cero (0) mediante un offset continuo, se mide la señal desplazada pero positiva y después se le resta el valor del offset

mediante código. Esto puede variar, y existen muchos otros métodos para medir tanto voltaje CC como AC. En este caso, no se alcanza a generar una señal lo suficientemente grande para hacer rectificación a CC y así medir una señal más estable, por lo que se advierte que el proceso de instrumentación y medición de variables eléctricas, en este caso voltaje, debe adaptarse a las necesidades de cada dispositivo [Espressif Systems, 2026a, Gómez Flórez, 2024, Kester, 2005, National Institute of Standards and Technology, 2019].

3.6.2.1. Acondicionamiento de señal para medición de voltaje AC

La medición del voltaje del generador no puede conectarse directamente al ADC de la **ESP32-C6**. El ADC admite un rango limitado de voltaje y requiere que la señal esté referida a tierra del circuito de medición. Por lo tanto, el diseño debe cumplir tres condiciones: 1) reducir la amplitud mediante un divisor resistivo, 2) desplazar la señal alterna hacia un punto medio seguro (0 V) cuando se requiera capturar semiciclos positivos y negativos como en este caso, y 3) proteger la entrada frente a sobretensiones o transitorios (etapa que no se implementa a profundidad en este proyecto). Esto se explica en el circuito de referencia mostrado en la tesis de maestría sobre el prototipado y análisis comparativo de dos turbinas eólicas de eje vertical [Gómez Flórez, 2024], en el que un primer divisor reduce la amplitud de la señal del generador y un segundo divisor crea una referencia intermedia para que la onda pueda ser leída por el conversor analógico–digital [Espressif Systems, 2026a, Kester, 2005, National Institute of Standards and Technology, 2019].

El divisor resistivo se puede expresar de forma general como:

$$V_{\text{adc}} = V_{\text{in}} \frac{R_2}{R_1 + R_2}.$$

Si $R_1=R_2$, la señal aplicada al ADC corresponde aproximadamente a la mitad de la señal de entrada (señal de salida del generador). Esta relación es útil, pero se debe tener presente que esa premisa solo se cumple en escenarios ideales, en la realidad, depende de la tolerancia de las resistencias, dinámica del sistema, respuesta del generador en el tiempo, de la temperatura, entre otros factores. Por lo tanto, es importante acercarse mediante la validación experimental y debido a eso en el firmware se incorpora un factor `voltage_divider_ratio` y una calibración ADC para convertir la lectura digital en una estimación física de voltaje [Espressif Systems, 2026a, Kester, 2005, Gómez Flórez, 2024, National Institute of Standards and Technology, 2019].

3.6.3. Estimación de RPM en ESP1

La estimación de la velocidad de giro de la turbina se hace calculando las revoluciones por minuto (RPM). Para esto, se acopla un sensor de efecto Hall o Reed Switch en la base del generador cerca al eje, de manera que detecte la energía de un imán cada vez que da una vuelta. Este sensor se conecta al GPIO22 y los pulsos se miden mediante la interrupción `sensor_isr_handler`, la cual incrementa `contador_pulsos` cuando detecta un flanco negativo y aplica un debounce (retardo) de 10 ms usando `esp_timer_get_time`. El uso de interrupciones es importante porque permite contar eventos rápidos sin esperar a que una tarea periódica consulte el pin, pero exige cuidado en la duración de la rutina ISR y en el uso de variables volátiles [Espressif Systems, 2026b,c,f, Gómez Flórez, 2024, Tchakoua et al., 2014].

El cálculo periódico se ejecuta cada segundo a través de GPTimer en `timer_callback`. En el código se define `PULSOS_POR_REVOLUCION = 2` y `ESCALA_RPM = 60/(INTERVALO_SEGUNDOS * PULSOS_POR_REVOLUCION)`. Con una ventana de un segundo, cada pulso representa treinta RPM cuando se tienen dos pulsos por vuelta. El código filtra valores mayores a 500 RPM ya que el límite de rotación de la turbina está por debajo de ese umbral, y aplica una media móvil de tres muestras para reducir variaciones bruscas [Espressif Systems, 2026c,b,f, Gómez Flórez, 2024, Tchakoua et al., 2014].

La ecuación general de RPM implementada es: [Espressif Systems, 2026c,b]

$$RPM = \frac{N_p}{N_{ppv}} \cdot \frac{60}{\Delta t},$$

donde,

N_p es el número de pulsos contados en la ventana,

N_{ppv} es el número de pulsos por vuelta y,

Δt es la duración de la ventana en segundos

En esta implementación, $N_{ppv} = 2$ y $\Delta t = 1$, por lo que $RPM = 30N_p$.

3.6.4. Envío ESP-NOW desde ESP1

La tarea de FreeRTOS `espnw_task` envía cada un segundo la estructura `mensaje_t` hacia la ESP2. El envío usa `esp_now_send` y registra en consola voltaje, RPM, estado **GPS** y coordenadas y activa un LED durante 200 ms como confirmación visual. Sin embargo, debe tenerse en cuenta de que esta es únicamente una confirmación visual de que el paquete de datos se envió, no de que se recibió en ESP2 [Espressif Systems, 2026f,d, National Marine Electronics Association, 2018, Espressif Systems, 2026b, National Institute of Standards and Technology, 2019].

En el código de ESP1, el peer **ESP-NOW** se configura con `.encrypt = true`. Esto significa que en este proyecto la comunicación local ESP1-ESP2 cifra los datos que se transmiten a través del protocolo. Para una arquitectura sólida, ESP1 y ESP2 deben usar PMK/LMK coherentes y activar cifrado por peer [Espressif Systems, 2026d, National Institute of Standards and Technology, 2019].

3.7. ESP2: nodo integrador, almacenamiento local y serialización json

El nodo 2 (ESP2) es el más complejo de la arquitectura Edge presentada. Su firmware se encuentra en Firmware-ESP2/main/main.c. En este nodo se integran: 1) recepción **ESP-NOW** de la información enviada por ESP-1, 2) conteo de pulsos del anemómetro con la misma lógica con la que se determina la velocidad de la turbina en ESP-1, 3) coordinación del bus **I2C** para la lectura del **BME680**, del **DS3231** y la escritura en la pantalla **OLED SSD1306**, 4) Manejo de memoria SD a través de **SPI**, construcción de promedios y envío de los datos en estructura **json** por **UART** a la ESP3 [Espressif Systems, 2026d,g, Bosch Sensortec, 2024, Espressif Systems, 2026f,i, National Institute of Standards and Technology, 2019].

Se podría pensar que la función de la ESP2 es convertir múltiples fuentes de datos (sensores-hardware) en un mensaje coherente que pueda ser interpretado por el usuario. En el código, para la recepción por **ESP-NOW** `mensaje_espnw_t` replica la estructura esperada desde ESP1: `latitud`, `longitud`, `voltaje_ac`, `rpm` y `gps_fix`. Además, ESP2 genera `velocidad_viento` a partir de pulsos locales obtenidos mediante interrupción ISR y GPTimer, lee ambiente mediante **BME680**, calcula altitud a partir de la presión atmosférica, toma timestamp del **DS3231** y calcula uso de memoria con `esp_get_free_heap_size`. La salida final se produce como un **json** de 512 bytes

enviado por **UART** a la ESP3 [Espressif Systems, 2026d,b,c, Bosch Sensortec, 2024, Espressif Systems, 2026l, National Institute of Standards and Technology, 2019].

Tabla nro. 3.6: Subsistemas integrados en ESP2 según el firmware activo.

Subsistema ESP2	Componente	Configuración identificada	Función
Anemómetro	GPIO7	Interrupción + GPTimer 1 s	Velocidad de viento
I2C	SDA GPIO5, SCL GPIO6	I2C_NUM_0, 400 kHz en dispositivos	BME680, OLED, DS3231
OLED	SSD1306, dirección 0x3C	128x64	Visualización local
Ambiental	BME680, dirección 0x77	Temperatura, humedad, presión	Variables de entorno
Tiempo	DS3231, dirección 0x68	Registros BCD	Timestamp local
MicroSD	SPI, MOSI 23, MISO 22, SCLK 21, CS 3	FATFS, <code>data.csv</code>	Respaldo y promedios
UART salida	UART1, TX GPIO15, RX GPIO9	9600 bps	json hacia ESP3
ESP-NOW	Peer ESP1	Canal 1	Datos remotos

3.7.1. Recepción ESP-NOW en ESP2

La recepción **ESP-NOW** se implementa mediante las funciones `inicializar_espnw_receptor` y `on_receive_espnw`. El receptor (ESP2) inicializa **ESP-NOW**, configura una PMK, registra la MAC de ESP1 como peer y asigna el callback de recepción. Cuando llega un paquete con tamaño igual a `sizeof(mensaje_espnw_t)`, se copia la estructura, se actualizan voltaje, RPM, coordenadas y timestamp de última recepción. Dado que se conoce la estructura de datos enviada por el emisor, se programa la recepción de forma tal que los datos de entrada se puedan copiar fácilmente, esto reduce procesamiento al momento del “parsing” porque ambas ESP comparten una estructura de datos común [Espressif Systems, 2026d, National Institute of Standards and Technology, 2019, Marjani et al., 2017].

Para determinar si ESP1 está conectado, la ESP2 está constantemente comparando el tiempo actual con `ultima_recepcion_espnw`. Si la diferencia supera tres segundos (tiempo modificable), el **json** usa valores sentinela (banderas) preestablecidos como -1.0 para voltaje y RPM. Este mecanismo es importante porque disminuye la pérdida de información ya que evita confundir “cero medido” con “dato no disponible”. En sistemas de monitoreo esto es muy importante porque permite diferenciar valor físico cero (por ejemplo: el anemómetro no está rotando porque no hay viento), pérdida de sensor (el anemómetro fue destrozado por un fuerte viento) y pérdida de comunicación (se perdió la conexión con Internet y no se volvió a conectar). Esto es fundamental para la calidad del dato y para que quien monitorea tenga claridad sobre el fenómeno [Espressif Systems, 2026d, National Institute of Standards and Technology, 2019, Marjani et al., 2017, Sisinni et al., 2018, Tchakoua et al., 2014].

3.7.2. Medición de velocidad del viento en ESP2

La velocidad de viento se calcula en ESP2 mediante un anemómetro conectado al GPIO7. La interrupción incrementa `contador_pulsos` y el GPTimer calcula `velocidad_viento = contador_pulsos * ESCALA_VELOCIDAD` cada segundo, tal cual como se hizo con la velocidad de la turbina en ESP1. En el código, `ESCALA_VELOCIDAD = 0.47f`; por tanto, cada pulso por segundo equivale a 0.47 m/s según el ajuste experimental definido. Este valor corresponde al perímetro que recorre un alabe del anemómetro en una vuelta, por ende, el número de vueltas por segundo por ese perímetro corresponde a la distancia aproximada recorrida por el fluido [Espressif Systems, 2026b,c, Gómez Flórez, 2024, Wang et al., 2023, Tchakoua et al., 2014].

La ecuación implementada puede escribirse como:

$$v = K_a \cdot N_p,$$

donde,

v es la velocidad estimada del viento en m/s,

K_a es el factor de calibración del anemómetro (perímetro) y,

N_p es el número de pulsos por ventana de un segundo.

3.7.3. Bus I2C compartido: BME680, OLED y DS3231

La tarjeta **ESP32-C6** es una actualización muy poderosa de la ESP32 WRoom, sin embargo, aunque soporta más protocolos también recorta en otros. Por ejemplo, en **UART** cuenta con UART0 (predestinado para programar la tarjeta), UART1 (se aprovecha en ESP1 con **GPS** y en ESP2 para transmitir a ESP3). En **I2C**, esta tarjeta cuenta con un bus en el sistema principal y otro en el sistema de baja potencia LP **I2C**. En la ESP2, se usa este bus maestro en GPIO5 y GPIO6 y se encarga de atender y coordinar la comunicación con el sensor **BME680**, el cual provee datos de temperatura, presión atmosférica, humedad; del reloj RTC **DS3231** y escribe en la pantalla **OLED SSD1306**. Como varios dispositivos utilizan el mismo medio al tiempo para enviar y recibir datos (bus físico - 2 cables), se usó la funcionalidad de FreeRTOS i2c_mutex para proteger operaciones críticas y evitar las colisiones lógicas entre tareas ya que existían [Espressif Systems, 2023, 2026l,g, Bosch Sensortec, 2024, Espressif Systems, 2026f, National Institute of Standards and Technology, 2019].

El **BME680** se inicializa en la dirección 0x77 (hexadecimal), configura oversampling, filtro IIR, heater profile y duración de medición. Bosch define el **BME680** como un sensor digital 4 en 1 capaz de medir gas, humedad, presión y temperatura, aunque en este proyecto se usan principalmente temperatura, humedad y presión para caracterizar la potencia eólica que recibe la turbina, pues estas variables se relacionan a través de la densidad del aire como se muestra en la tesis de maestría sobre el prototipado y análisis comparativo de dos turbinas eólicas de eje vertical [Gómez Flórez, 2024]. La altitud se deriva a partir de presión mediante una ecuación barométrica aproximada [Bosch Sensortec, 2024, Espressif Systems, 2026g, Wang et al., 2023].

La ecuación de altitud usada en el firmware es:

$$h = 44330 \left(1 - \left(\frac{P}{P_0} \right)^{0.1903} \right),$$

donde,

P es la presión medida o corregida y,

P_0 corresponde a la presión de referencia al nivel del mar o una presión base ajustada.

En el código se observa `SEA_LEVEL_PRESSURE_HPA = 1202.5f` y ajustes empíricos sobre presión, temperatura y humedad. Estos ajustes deben corresponder a validaciones experimentales,

lo cual es importante a la hora de adaptar el sistema en otros entornos.

El RTC **DS3231** es un módulo electrónico que se comunica mediante I2c y tiene la capacidad de brindar fecha y hora en tiempo real para construir la variable de timestamp y poder interrelacionar todas las variables del sistema en función del tiempo. Por otra parte, la pantalla **OLED** permite mostrar datos operativos sin depender de un computador o una pantalla más costosa, también se comunica a través de **I2C**. El uso de visualización local es bastante útil porque permite verificar rápidamente si existe señal de viento, voltaje, RPM, memoria, temperatura o presión y determinar posibles fallas o conocer los valores actuales del sistema sin ver el dashboard. Sin embargo, como **OLED**, **BME680** y **DS3231** comparten **I2C**, si existe una falla de bus, se pueden afectar varias funciones a la vez [Analog Devices, 2015, Solomon Systech, 2008, Espressif Systems, 2026g, Bosch Sensortec, 2024, National Institute of Standards and Technology, 2019].

3.7.4. Almacenamiento local en microSD

El firmware programado para la ESP2 también monta sobre **SPI** y **FATFS** almacenamiento local a través de una tarjeta micro **SD** de 2 GB y usando `esp_vfs_fat_sdspi_mount`. Se crea un archivo llamado `data.csv` con encabezado y registra timestamp, velocidad de viento, voltaje, temperatura, humedad, presión, altitud, latitud, longitud, estado **ESP-NOW**, uso de memoria y RPM de la turbina. Esta capa se usa en este caso para almacenar 10 muestras y así enviar una muestra completa cada 10 segundos a AWS a partir del promedio de las variables continuas. También, es importante implementar un sistema de almacenamiento que actúe cuando no hay conexión hacia (AWS - ESP3) con el propósito de no perder datos en medio de esas contingencias [Espressif Systems, 2026k,d, National Institute of Standards and Technology, 2019, Marjani et al., 2017, Gómez Flórez, 2024].

A partir de la función `sd_write_sample` se escriben muestras individuales en el archivo `data.csv` y se acumulan las variables en una estructura relacional para promediarlas cada diez muestras o cada 10 segundos, o sea, ya no son variables separadas. Al alcanzar diez muestras, `data_task` calcula los promedios para cada variable y construye el **json** que se envía por **UART** a la ESP3. Esta estructura se define principalmente para reducir tráfico hacia AWS, minimizar costos y suavizar ruido de los sensores [Espressif Systems, 2026k,l, Marjani et al., 2017, National Institute of Standards and Technology, 2019, AWS, 2026g].

3.7.5. Generación del json en ESP2

El **json** consolidado se genera en `data_task` con `sprintf` dentro de un buffer de 512 bytes. Los campos observados son `timestamp`, `wind_speed`, `voltage`, `temperature`, `humidity`, `pressure`, `altitude`, `lat`, `lon`, `rpm`, `esp_now_connected` y `memory_usage`. Este contrato debe mantenerse estable porque **AWS IoT Core**, las reglas IoT, **DynamoDB**, **S3**, **Athena** y el dashboard dependen de los nombres y tipos de esos campos [OASIS, 2019, AWS, 2026g,h,c,m,a].

```
{
  "timestamp": "18/05/26 12:30:00",
  "wind_speed": 2.35,
  "voltage": 1.42,
  "temperature": 24.80,
  "humidity": 63.20,
  "pressure": 1012.45,
  "altitude": 1280.50,
  "lat": 4.823465,
  "lon": -75.724832,
  "rpm": 120.0,
  "esp_now_connected": true,
  "memory_usage": 18.40
}
```

3.7.6. Envío UART de ESP2 hacia ESP3

Una vez generado y alistado el **json**, la ESP2 lo envía a la ESP3 por **UART** mediante `uart_write_bytes`. En el programa se configura UART1 a 9600 bps (bits per second o baudios) con TX en GPIO15 y RX en GPIO9 para esta tarea. UART0 por defecto se deja para programar la tarjeta. Por facilidad, el **json** se termina con salto de línea, lo que permite que el receptor identifique mensajes completos. **UART** es apropiado para este tramo porque ESP2 y ESP3 son nodos físicos cercanos, conectados punto a punto y sin necesidad de direccionamiento IP [Espressif Systems, 2026l, Bray, 2017, National Institute of Standards and Technology, 2016, 2019].

Aunque es relativamente sencillo con respecto a otros protocolos, una implementación completa

de **UART** requiere controles explícitos de robustez. En ese contexto, el receptor debe tener la capacidad de manejar tramas incompletas, buffers con capacidad insuficiente, mensajes concatenados, caracteres dañados o corruptos, entre otros retos. En este proyecto, la ESP3 reconstruye el **json** enviado desde la ESP2 a partir de la búsqueda de llaves preestablecidas de apertura y cierre de la trama. La estrategia planteada es efectiva, sin embargo, puede mejorarse usando delimitadores explícitos como longitud de trama, checksum o CRC para garantizar la integridad y completitud del mensaje [Espressif Systems, 2026l, Bray, 2017, National Institute of Standards and Technology, 2019, Koopman and Chakravarty, 2004].

3.7.7. Tareas concurrentes en ESP2

Como se ha evidenciado, en la ESP2 se utilizan múltiples (5) tareas **FreeRTOS**: monitoreo de memoria, adquisición/integración de datos, visualización **OLED**, recepción **UART** de diagnóstico y watchdog del **BME680**, cada una con su prioridad correspondiente. Además, usa interrupciones para conteo de pulsos, GPTimer, callback **ESP-NOW** para recepción, tres periféricos con **I2C** y uno (1) con **SPI**. Esta combinación de tareas, callbacks e interrupciones permite responder a eventos de diferente naturaleza, pero exige coordinación mediante mutex y cuidado en el acceso a variables compartidas [Espressif Systems, 2026f,l, Bosch Sensortec, 2024, Espressif Systems, 2026d,g,k].

Tabla nro. 3.7: Tareas concurrentes identificadas en ESP2.

Tarea o callback	Prioridad asignada	Función	Recurso crítico
uart_receive_task	5	Diagnóstico UART	UART1
monitor_memoria_task	4	Monitoreo de heap	Memoria global
data_task	3	Lectura, agregación, SD y json	I2C, SD, UART
bme_watchdog_task	3	Supervisión del BME680	I2C/BME680
display_task	2	Pantallas OLED	I2C/OLED
on_receive_espnw	callback	Actualización remota	Variables ESP-NOW

timer_callback	ISR/callback	Cálculo velocidad viento	Contador de pulsos
----------------	--------------	--------------------------	--------------------

La tabla 3.7 muestra que la tarea data_task concentra la mayor responsabilidad funcional. Esta tarea lee el RTC, solicita mediciones al **BME680**, escribe en la SD, calcula promedios y memoria, construye el **json** y lo transmite por **UART**, lo que termina sobrecargando mucho la función. Aunque es plenamente funcional, desde el punto de vista de robustez, una mejora futura sería desacoplar almacenamiento SD, construcción **json** y transmisión **UART** mediante colas **FreeRTOS**, de forma que una escritura lenta en **SD** no retrase el envío de datos [Espressif Systems, 2026f, Bosch Sensortec, 2024, Espressif Systems, 2026k, Bray, 2017, Espressif Systems, 2026l, National Institute of Standards and Technology, 2019].

3.8. ESP3: gateway UART, Wi-Fi y MQTT/TLS hacia AWS IoT Core

La ESP3 actúa como la frontera entre la arquitectura embebida local (ESP1–ESP2–ESP3) y AWS. Para esta etapa, el firmware corresponde a Firmware-ESP3/main/main.c. La función de ESP3 es recibir el **json** enviado por ESP2 mediante **UART**, verificar la estructura mínima, conectarse a **Wi-Fi** como estación, iniciar el cliente **MQTT** sobre **TLS**, autenticarse con certificado **X.509** y publicar la telemetría en el tópico configurado en **AWS IoT Core**. Esta separación hace que ESP1 y ESP2 no tengan que gestionar certificados ni sesiones **TLS** ya que es relativamente complejo, lo cual podría analizarse como una ventaja de la arquitectura planteada [Bray, 2017, Espressif Systems, 2026l, OASIS, 2019, Rescorla, 2018, AWS, 2026g,j].

En el firmware se configura UART1 a 9600 baudios con TX en GPIO15 y RX en GPIO22 para interconectarse con la ESP2. La tarea uart_receive_task lee bytes periódicamente y reconstruye mensajes entre { y }, valida que exista la estructura mínima de llaves y publica el contenido en el tópico si mqtt_connected es verdadero, es decir, si se logró conectar a AWS. Este enfoque ayuda a mejorar la tolerancia frente a la fragmentación básica del flujo serial y evita la publicación de strings que no sean **json** en la estructura predeterminada, esto facilita la limpieza e interpretación de la data [Espressif Systems, 2026l, Bray, 2017, OASIS, 2019, AWS, 2026g, National Institute of Standards and Technology, 2019].

Tabla nro. 3.8: Configuración funcional del gateway ESP3.

Elemento ESP3	Configuración identificada	Función
UART	UART_NUM_1, 9600 bps, TX GPIO15, RX GPIO22	Recepción del json desde ESP2
Wi-Fi	Modo estación, WPA2-PSK	Conexión IP local
MQTT	Host AWS IoT, puerto 8883, QoS 1	Publicación cloud
TLS	Transporte SSL/TLS	Cifrado e integridad
Identidad	client_id = ESP32C6-AWS	Identificación del dispositivo
Certificados	Certificado de dispositivo y llave privada embebidos	Autenticación X.509
Mutex	mqtt_mutex	Protección del cliente MQTT

3.8.1. Conectividad Wi-Fi en ESP3

La conexión a **Wi-Fi** se implementa mediante la función `wifi_init_sta`, la cual crea la interfaz STA, registra manejadores de eventos, configura SSID y contraseña de la red de Internet, inicia el driver y espera la obtención de IP. El manejador de eventos actualiza la bandera `wifi_connected` cuando se recibe `IP_EVENT_STA_GOT_IP` y reconecta en caso de desconexión. Este patrón es habitual en **esp-idf** porque **Wi-Fi** se maneja mediante eventos asíncronos [Espressif Systems, 2026n,e,f, National Institute of Standards and Technology, 2019].

En un proyecto real, ningún tipo de credencial debe compartirse, por el contrario, deben gestionarse con buenas prácticas de seguridad y embeberse en firmware para evitar exposiciones de seguridad. Prácticas más seguras sugieren usar configuraciones externas, particiones NVS cifradas, aprovisionamiento controlado o variables no versionadas [National Institute of Standards and Technology, 2019, AWS, 2026i,j, Espressif Systems, 2026j,m].

3.8.2. MQTT sobre TLS y certificados X.509

La conexión **MQTT** (Message Queuing Telemetry Transport) se configura con la función `esp_mqtt_client_config_t`. El broker usa el endpoint de **AWS IoT Core**, el puerto estándar

y oficial para conexiones **MQTTs** 8883 y transporte **MQTT_TRANSPORT_OVER_SSL**. La identidad del dispositivo se precisa mediante `client_id` y la autenticación usa certificado de dispositivo y llave privada creados en **AWS IoT Core** embebidos en el firmware. A su vez, este servicio usa certificados **X.509** para autenticar dispositivos antes de permitir comunicación, y **TLS** protege la confidencialidad e integridad contra escucha, alteración y falsificación de los datos [Espressif Systems, 2026i, OASIS, 2019, AWS, 2026g,j].

El evento **MQTT_EVENT_CONNECTED** indica la conexión marcando la bandera `mqtt_connected = true` y registra el endpoint otorgado por AWS, `client ID` y el tópico. El evento **MQTT_EVENT_PUBLISHED** confirma la publicación de la telemetría en el tópico y **MQTT_EVENT_ERROR** permite diagnosticar fallas de transporte o handshake **TLS**. Esta estructuración por eventos es bastante importante porque en sistemas IoT no solo se trata de llamar una función de publicación; se debe confirmar el estado real del cliente, registrar causas de error y garantizar la trazabilidad completa ya que se trata de sistemas que tienen una alta interacción con la red de Internet [Espressif Systems, 2026i, OASIS, 2019, AWS, 2026g, National Institute of Standards and Technology, 2019].

3.8.3. Recepción y validación json en ESP3

La función `verify_and_format_json` revisa que el mensaje recibido mediante **UART** contenga llaves de apertura y cierre, extrae el contenido entre la primera { y la última }, verifica que el tamaño no exceda el tamaño del buffer definido y produce una cadena lista para publicar a partir de la estructura preestablecida. Esta validación es mínima, pero evita enviar al broker fragmentos **UART** sin estructura y conservar la homogeneidad e integridad en la base de datos [Espressif Systems, 2026l, Bray, 2017, OASIS, 2019, AWS, 2026g, National Institute of Standards and Technology, 2019].

El uso de QoS 1 (Quality of Service nivel 1) en `esp_mqtt_client_publish` indica que desde el cliente se solicita confirmación de entrega al broker al menos una vez. Esto mejora confiabilidad frente a QoS 0, aunque puede permitir duplicados si ocurre retransmisión, situación que debe contemplarse en el procesamiento. En una arquitectura de datos histórica, los duplicados deben manejarse con timestamp, `clientId` e identificadores de mensaje para evitar registros repetidos o inconsistencias en series temporales [Espressif Systems, 2026i, OASIS, 2019, AWS, 2026g, Marjani et al., 2017, National Institute of Standards and Technology, 2019].

3.9. Contrato de datos Edge–Cloud

El contrato de datos es el elemento que une el hardware, firmware y la nube. Si los nombres de campos cambian entre ESP2, AWS IoT Rule, **DynamoDB**, **Athena** y frontend, la arquitectura completa se rompe aunque la comunicación **MQTT** funcione bien. Por esta razón, el contrato debe definirse como una especificación estándar del sistema y no como una salida cualquiera del `snprintf` usado en el firmware, todo está interconectado en este caso [ISO/IEC/IEEE, 2018, Bray, 2017, AWS, 2026h,c,a, National Institute of Standards and Technology, 2019].

En la tabla 3.9 se presenta una propuesta de estructura para el contrato de datos. Se recomienda mantener los nombres en inglés para compatibilidad con código y servicios de AWS, pero documentar en español su significado, unidad, tratamiento de errores y presentación. También, se recomienda incluir `clientId` para nombrar los sistemas de forma explícita en el **json** generado por ESP2 o agregado por ESP3 antes de publicar, porque **DynamoDB** y **Athena** usan este campo llave e consulta en el caso de tener múltiples sistemas reportando datos [ISO/IEC/IEEE, 2018, Bray, 2017, AWS, 2026h,c,a, National Institute of Standards and Technology, 2019].

Tabla nro. 3.9: Especificación propuesta del contrato json Edge–Cloud.

Campo	Tipo	Unidad	Origen	Tratamiento recomendado
<code>clientId</code>	<code>string</code>	No aplica	ESP3 o configuración	Obligatorio
<code>timestamp</code>	<code>string</code>	Fecha/hora local	DS3231/ESP2	Obligatorio; formato estable
<code>wind_speed</code>	<code>number</code>	m/s	ESP2/anemómetro	Rango físico y calibración
<code>voltage</code>	<code>number</code>	V	ESP1/ADC	−1 . 0 si no hay ESP-NOW
<code>temperature</code>	<code>number</code>	°C	ESP2/BME680	Calibración documentada
<code>humidity</code>	<code>number</code>	%	ESP2/BME680	0–100 %
<code>pressure</code>	<code>number</code>	hPa	ESP2/BME680	Validar rango local

Campo	Tipo	Unidad	Origen	Tratamiento recomendado
altitude	number	m	Derivada de presión	Depende de P_0
lat	number	grados	ESP1/GPS	0 si no hay fix, o null futuro
lon	number	grados	ESP1/GPS	0 si no hay fix, o null futuro
rpm	number	rpm	ESP1/Hall	-1.0 si no hay ESP-NOW
esp_now_connected	boolean	No aplica	ESP2	Calculado por timeout
memory_usage	number	%	ESP2/heap	Diagnóstico

3.10. Gestión temporal y sincronización

En este proyecto la gestión temporal se implementa principalmente en la ESP2 a través del RTC **DS3231**. El timestamp se construye con `strDate` y `strTime`, y se usa para el CSV local y en el **json** hacia ESP3. Esto permite mayor respaldo frente a la conexión **Wi-Fi** ya que, aunque la nube no esté disponible, la ESP2 puede registrar temporalmente los datos en microSD con hora local, enviar a la nube una vez se tenga disponibilidad de red y allí generar el merge para garantizar el procesamiento de la información [Analog Devices, 2015, Bray, 2017, Espressif Systems, 2026k,n, AWS, 2026g, National Institute of Standards and Technology, 2019].

3.11. Seguridad de la capa hardware Edge

Como se evidenció anteriormente, la seguridad en la capa Edge debe analizarse por tramo. En ESP1–ESP2, **ESP-NOW** permite cifrado mediante PMK y LMK. En ESP2–ESP3, **UART** es físico y punto a punto, pero no tiene cifrado ni autenticación; por tanto, debe protegerse en un futuro con validación de formato y control de acceso físico. En ESP3–AWS, **MQTT/TLS** si introduce cifrado, autenticación de dispositivo y autorización mediante política IoT [Espressif Systems, 2026d,l, OASIS, 2019, AWS, 2026j,f].

En la tabla 3.10 se resumen los riesgos y controles. No se busca afirmar que el prototipo ya está completamente listo, sino diferenciar el nivel experimental del nivel recomendado para despliegue. Con esta diferenciación se desea mostrar que se reconocen las vulnerabilidades del diseño, se proponen controles y se establece una ruta de madurez técnica [National Institute of Standards and Technology, 2019, AWS, 2026i,f, ISO/IEC/IEEE, 2018].

Tabla nro. 3.10: Riesgos y controles de seguridad para la capa IoT Edge.

Tramo	Riesgo principal	Estado observado	Control recomendado
Sensor/ADC	Ruido, mala calibración	RMS y media móvil	Calibración documentada y pruebas patrón
GPIO pulsos	Rebote, falsos pulsos	Debounce ESP1; conteo ESP2	Debounce también en anemómetro
ESP-NOW	Intercepción o spoofing local	Peer con cifrado efectivo	Activar PMK/LMK y <code>encrypt=true</code>
UART ESP2–ESP3	Tramas corruptas	Búsqueda de { }	Longitud, CRC y delimitador formal
Wi-Fi	Exposición de credenciales	Macros en código	NVS protegida o aprovisionamiento
MQTT/TLS	Certificados expuestos	Certificados embebidos	Manejo seguro y rotación de certificados
AWS IoT	Permisos excesivos	Depende de política	Least privilege por <code>clientId/tópico</code>

3.12. Integridad, calidad y pérdida de datos

En el sistema ya se han incorporado algunos mecanismos de calidad de los datos: por una parte, se generó la implementación de timeout para detectar pérdida de la comunicación **ESP-NOW**, se fijaron valores sentinela (banderas fijas) para variables remotas, se generan promedios cada diez muestras para reducir ruido, se gestiona escritura en CSV local y logs seriales. Con estos mecanismos se busca mejorar esta etapa porque permiten identificar fallas sin detener todo el

sistema. Sin embargo, deben complementarse con validaciones de rango, control de timestamp y verificación de integridad de tramas **UART** [Espressif Systems, 2026d,l,k, Marjani et al., 2017, National Institute of Standards and Technology, 2019].

La calidad del dato debe evaluarse en tres momentos: antes de enviar por **ESP-NOW**, antes de construir **json** y antes de publicar por **MQTT**. Antes de **ESP-NOW** se validan magnitudes físicas como voltaje y RPM; antes de **json** se validan promedios, **GPS**, ambiente y memoria; antes de **MQTT** se valida estructura **json**. Esta segmentación permite ubicar el origen de errores cuando un valor extraño aparece en **DynamoDB**, **S3** o el dashboard [National Institute of Standards and Technology, 2019, Espressif Systems, 2026d, Bray, 2017, OASIS, 2019, AWS, 2026c,m].

3.13. Estrategia de validación de la capa IoT Edge

La validación de la capa de hardware debe ser por etapas y de manera paulatina. Primero se prueba cada sensor y periférico por separado; luego se prueba cada nodo; después se prueban los enlaces ESP1–ESP2, ESP2–ESP3 y finalmente se prueba la publicación ESP3–AWS. Esta secuencia evita depurar simultáneamente ADC, **GPS**, **ESP-NOW**, **UART**, **Wi-Fi** y **MQTT**, lo cual aumentaría la incertidumbre técnica y por el contrario se recomienda depurar cada elemento e ir integrando paso a paso cada segmento [ISO/IEC/IEEE, 2018, National Institute of Standards and Technology, 2019, National Marine Electronics Association, 2018, Espressif Systems, 2026d,l, AWS, 2026g].

En la tabla 3.11 se proponen pruebas específicas para probar cada etapa. Debe definirse que para cada prueba exista una evidencia concreta: log serial, valor medido, archivo CSV, captura **OLED**, mensaje **UART**, publicación **MQTT** o registro en **AWS IoT Core**. Algunas de estas pruebas deben hacerse con instrumentos físicos y otras se deben implementar a través de la consola de esp-idf [ISO/IEC/IEEE, 2018, National Institute of Standards and Technology, 2019, Solomon Systech, 2008, Espressif Systems, 2026l,i, AWS, 2026g].

Tabla nro. 3.11: Casos de prueba recomendados para validar la arquitectura IoT Edge.

Código (HW)	Prueba	Variable/flujo	Evidencia mínima
HW-01	Lectura GPS ESP1	latitud, longitud, gps_fix	Log con fix válido

Código (HW)	Prueba	Variable/flujo	Evidencia mínima
HW-02	ADC ESP1	voltaje_ac	Comparación con multímetro
HW-03	RPM ESP1	rpm	Comparación con tacómetro o conteo manual
HW-04	ESP-NOW	mensaje_espnw_t	Log de recepción ESP2
HW-05	Anemómetro ESP2	wind_speed	Comparación con patrón
HW-06	BME680 ESP2	temperature, humidity, pressure	Lectura estable en consola/OLED
HW-07	RTC ESP2	timestamp	Fecha/hora coherente
HW-08	MicroSD ESP2	data.csv	Archivo con encabezado y registros
HW-09	UART ESP2–ESP3	json	Log en ESP3 con payload completo
HW-10	MQTT ESP3	Publicación cloud	Evento MQTT_EVENT_PUBLISHED

3.14. Síntesis del capítulo

En este capítulo se propone una arquitectura IoT - Edge que convierte el generador eólico de eje vertical de pequeña escala en un sistema medible, distribuido y conectado a la red de Internet. El sistema se divide en tres nodos. La ESP1 captura las variables del generador AC y las transmite por **ESP-NOW**. En la ESP2 se consolidan las variables locales y remotas (ESP1), registra datos en la **microSD**, visualiza información en la pantalla **OLED** y genera un **json** con todos los datos del sistema. En la ESP3 se recibe el **json** por **UART** y se publica en **AWS IoT Core** mediante **MQTTs**. Esta estructura permite delimitar las responsabilidades, mejorar la depuración de problemas y explicar el sistema desde la medición de variables físicas hasta la telemetría cloud

[National Institute of Standards and Technology, 2016, Gómez Flórez, 2024, Espressif Systems, 2026d, Bray, 2017, Espressif Systems, 2026l, AWS, 2026g].

El análisis del sistema evidencia que el diseño ya incorpora decisiones técnicas importantes que llevan este prototipo a un nivel más avanzado de desarrollo con el SoC **ESP32-C6**. Se hace uso de interrupciones, GPTimer, ADC calibrado, **I2C** compartido con mutex, almacenamiento CSV en memoria SD, promedios de variables continuas, valores centinela prefijados, **MQTTs** QoS 1 y autenticación con certificados. A su vez, se identifican mejoras necesarias en cifrado **ESP-NOW**, calibración de sensores, gestión de secretos y credenciales, validación del **json** y sincronización temporal más precisa [Espressif Systems, 2023, 2026c,a,g,k, AWS, 2026j].

Diseño e implementación de la arquitectura de software en AWS

En este capítulo se describe la arquitectura de software implementada en AWS para la recepción, el almacenamiento, la extracción y la visualización de datos del sistema. La capa de hardware embebido transforma señales del mundo físico en mensajes de tipo **json** que son enviados a AWS para convertirlos en información disponible e histórica para el dashboard para consultas analíticas o validación experimental. La arquitectura se implementó a partir de servicios de tipo **serverless**, los cuales no requieren del despliegue de servidores tipo instancias EC2 para funcionar o ejecutar las funciones. En este proceso, se usa **AWS IoT Core** para la recepción **MQTTs**, reglas IoT para enrutamiento, Amazon **DynamoDB** para procesar el último paquete de datos disponible, **Amazon S3** para almacenamiento de datos históricos en estructura **json**, **AWS Glue** Data Catalog para metadatos y ETL, Amazon **Athena** para automatizar consultas SQL, **AWS Lambda** Function URLs para exponer funciones y consultas HTTP y Amazon **CloudFront** + **S3** para distribuir la interfaz web [Botta et al., 2016, Marjani et al., 2017, AWS, 2026n,h,c,a].

Para este trabajo, la decisión de usar servicios administrados responde a 3 razones: 1) Se desea analizar la operación del sistema sin la administración y mantenimiento de servidores, 2) Se busca escalamiento por uso sin mayor complejidad arquitectural ya que el sistema puede crecer en número de turbinas, frecuencia de muestreo y tiempo de operación, 3) El sistema tiene dos patrones de visualización, por una parte requiere actualización constante del último paquete de datos con baja latencia (**DynamoDB**) y por otra parte proporciona series históricas filtradas por rango de fechas y variables (**S3** + **Glue** + **Athena**) [AWS, 2026n, Hashem et al., 2015, Marjani et al., 2017, AWS, 2026c,m,b].

Arquitectura de software en AWS para el sistema IoT de monitoreo eólico

Ingesta segura, almacenamiento híbrido, consulta histórica y visualización web

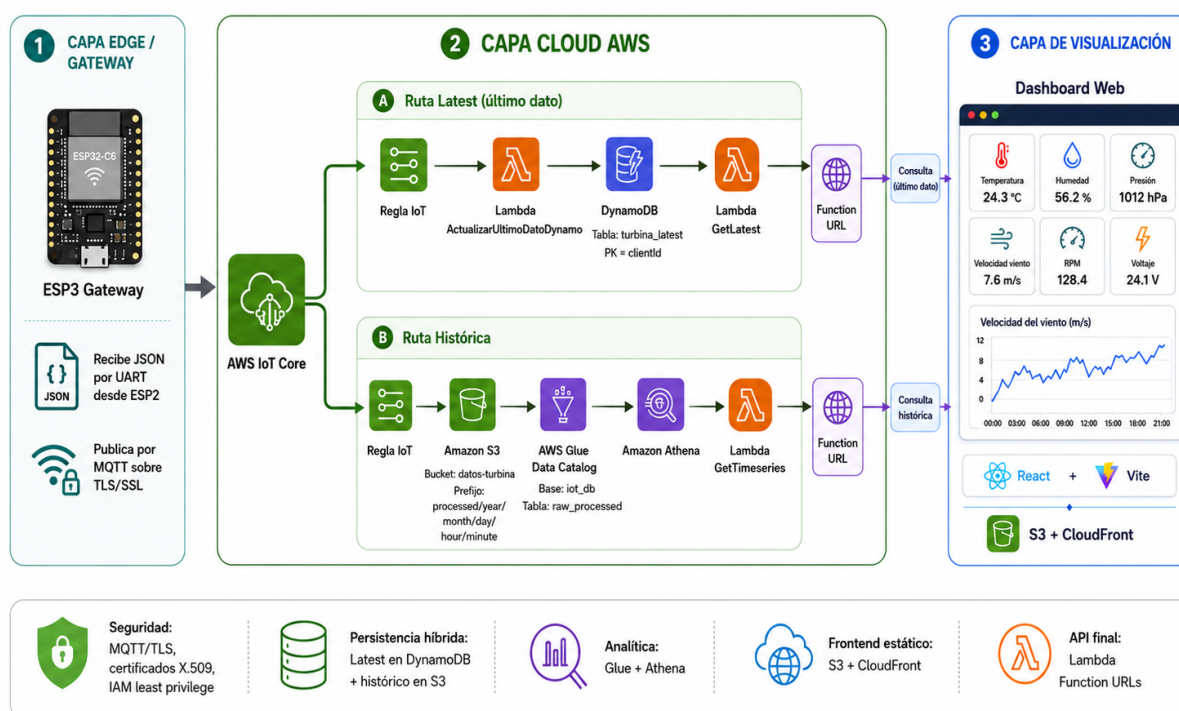


Figura nro. 4.1: Arquitectura de software en AWS para el sistema IoT de monitoreo eólico. Fuente: elaboración propia con apoyo de ChatGPT, modelo GPT-5.5 Thinking de OpenAI, 2026.

La Figura nro. 4.1 sintetiza la arquitectura adoptada en este trabajo. El proceso inicia en la ESP3, la cual actúa como gateway y publica la estructura json mediante MQTTs en AWS IoT Core. Desde IoT Core, el mensaje se distribuye a dos rutas diferentes: 1) La ruta Latest para mantener el último paquete de datos actualizado a través de DynamoDB, 2) La ruta histórica para mantener la base de datos histórica y consultarla cuando el usuario lo requiera. El dashboard desplegado mediante S3 + CloudFront consume datos de la BD mediante Lambda Functions URLs.

4.1. Propósito de la capa cloud

La capa cloud tiene como propósito reemplazar lo que en la tesis de maestría sobre el prototipado y análisis comparativo de dos turbinas eólicas de eje vertical se presentó como la arquitectura de software en servidor local mediante Python + SQL + ShinyApps de R [Gómez Flórez, 2024]. En este caso, el subsistema Edge mide las señales in situ y las transmite, mientras que la capa Cloud garantiza una ingesta segura de los datos, persistencia, consulta, exposición HTTP y visualización adaptada. Por lo tanto, la nube no es solo un repositorio para una base de datos, en este caso permite desarrollar una arquitectura de procesamiento liviano, segura y escalable que convierte la información del generador eólico de eje vertical en información útil para el usuario [Botta et al., 2016, Marjani et al., 2017, AWS, 2026h,k,n].

La arquitectura planteada separa responsabilidades entre servicios con el propósito de evitar errores o procesos innecesarios. **AWS IoT Core** recibe el paquete de datos y lo enruta mediante políticas IoT, **DynamoDB** sostiene el último paquete de datos, **Amazon S3** almacena histórico, **Athena** ejecuta SQL sobre el DataLake, **AWS Lambda** encapsula la lógica de consulta mediante funciones y **CloudFront** distribuye el FrontEnd. Esta separación mediante servicios es muy interesante porque permite separar responsabilidades, niveles de permisos, determinar los costos con precisión y mayor escalabilidad por componente sin mayores ajustes arquitecturales [AWS, 2026h,c,m,a,k, Amazon Web Services, 2026d].

La capa cloud puede entenderse como una aproximación de software como servicio para el monitoreo del sistema eólico. El usuario no interactúa directamente con scripts, servidores o bases de datos, sino con un dashboard web que consume servicios como Lambda, DynamoDB, S3, Athena y CloudFront. De esta manera, la arquitectura transforma la telemetría enviada por el hardware en un servicio digital de consulta y visualización [Mell and Grance, 2011, AWS, 2026n,k,c,a, Amazon Web Services, 2026d].

4.2. Requisitos funcionales y no funcionales

Los requisitos funcionales de la arquitectura presentada se obtienen a partir del análisis y depuración del sistema. La ESP2 genera un contrato **json** que obedece a una lógica y responde a necesidades específicas y es enviado a AWS por la ESP3. En la nube se debe recibir mensajes **MQTTs**, validar la identidad del dispositivo que envía los datos, enrutar la telemetría recibida, mantener operativo el último valor por `clientId`, almacenar el histórico, permitir consultas **SQL** por rango temporal, ejecutar las funciones **Lambda** y exponer respuestas **HTTP** al dashboard. Como se ha mostrado, esto responde a una secuencia de sucesos y eventos en el tiempo, por lo que se debe conservar estructuras y variables estables porque cualquier cambio por mínimo que sea puede alterar el contrato **json** y por ende la lógica en AWS [ISO/IEC/IEEE, 2018, Bray, 2017, AWS, 2026g,j,h,c].

Tabla nro. 4.1: Requisitos funcionales de la arquitectura AWS.

Requisito	Servicio principal	Decisión implementada	Justificación
Ingesta segura	AWS IoT Core	MQTT/TLS en puerto 8883	Autenticación por certificado y broker administrado
Autorización del dispositivo	AWS IoT Policy	Política asociada a certificado X.509	Controla conexión, publicación y tópico
Último dato	DynamoDB	Tabla <code>turbina_latest</code> , PK <code>clientId</code>	Lectura rápida para tarjetas del dashboard
Histórico	Amazon S3	Bucket <code>datos-turbina</code> , prefijo <code>processed/</code> particionado	Almacenamiento económico y durable
Catálogo	AWS Glue Data Catalog	Base <code>iot_db</code> , tabla <code>raw_processed</code>	Metadatos para consulta SQL
Consulta histórica	Amazon Athena	SQL sobre S3/Glue	Series temporales por rango y variable

Requisito	Servicio principal	Decisión	Justificación
		implementada	
API serverless	AWS Lambda	GetLatest y	Endpoints HTTP
	Function URLs	GetTimeseries	simples para React
Visualización	S3 + CloudFront	Dashboard React/Vite	Frontend estático distribuido

Por otra parte, los requisitos no funcionales se relacionan con aspectos de seguridad, costo, mantenibilidad y escalabilidad de la arquitectura. En términos de seguridad, en este proyecto se implementa un nivel aceptable pero que podría ser mejor. Se implementa **MQTT** sobre **TLS** ya que así lo requiere **AWS IoT Core**, certificados **X.509** que genera AWS y se embeben en el firmware de ESP3, permisos mínimos de **IAM**, políticas IoT restrictivas y **CORS** controlado para la interacción con el FrontEnd. Adicionalmente, el costo es otro requisito importante a satisfacer, optimizarlo exige separar las consultas frecuentes de último dato de las consultas de datos históricos, dado que son más pesadas y requieren un tratamiento diferente. La mantenibilidad del flujo requiere una estructura de datos clara, nombres claros y consistentes de los recursos, variables del sistema documentadas, roles documentados y trazabilidad de errores. Por último, la escalabilidad requiere particiones temporales, límites de consultas y una ruta para transformar **json** pequeños en **parquet** y así optimizar la estructura de datos de manera que se adapte a la arquitectura presentada [National Institute of Standards and Technology, 2019, AWS, 2026i,j,n,b, Apache Software Foundation, 2026].

4.3. Flujo general de datos

El flujo de datos se puede representar mediante la implementación de una cadena de servicios especializados para cada etapa. La ESP3 comunica el sistema físico con la arquitectura cloud a través del envío del contrato **json** a un tópico **MQTTs**, **AWS IoT Core** recibe los datos enviados por ESP3, una regla IoT toma acciones sobre los datos recibidos, **DynamoDB** se encarga de conservar el último paquete de datos, **Amazon S3** almacena el histórico, **AWS Glue** cataloga las particiones para el ETL, **Athena** ejecuta las consultas SQL, las funciones **Lambda** entregan resultados HTTP y el dashboard desplegado sobre **React/Vite** muestra los datos más recientes en “cards” por variable y gráficos históricos. Se puede resumir de la siguiente manera: [AWS,

2026g,h,c,m,a,k].

```
ESP3 Gateway
-> MQTT/TLS
-> AWS IoT Core
-> IoT Rule
-> Ruta Latest: ActualizarUltimoDatoDynamo -> DynamoDB ->
-> Lambda GetLatest -> Function URL
-> Ruta Histórica: S3 -> Glue Data Catalog -> Athena ->
-> Lambda GetTimeseries -> Function URL
-> React + Vite
-> S3 + CloudFront
```

La separación entre ruta *Latest* e histórica es una decisión central. El último dato se consulta de forma frecuente, por ejemplo, cada cierto número de segundos desde el dashboard, 30 segundos en este caso; por eso se almacena en **DynamoDB**, que permite lectura directa por clave. En cambio, el histórico se consulta por rangos de tiempo, variables y periodos específicos; por eso se almacena en **S3**, se cataloga con Glue y se consulta con **Athena**. Esta separación evita usar **Athena** como base operativa de baja latencia y evita usar **DynamoDB** como repositorio histórico de análisis pesado [AWS, 2026c,m,a,b,n].

4.4. Ingesta segura con AWS IoT Core

AWS IoT Core funciona como punto de entrada de la arquitectura cloud planteada. La ESP3 se conecta como cliente a **AWS IoT Core** mediante **MQTT** usando **TLS** y certificados **X.509** para asegurar la transmisión de los datos. Esta estructura busca que no se expongan las credenciales de usuario de AWS directamente en el firmware de la ESP3. Desde la visión de la seguridad, el certificado **X.509** se usa para autenticar el dispositivo ante AWS, la política IoT autoriza o deniega las acciones que puede ejecutar y el tópico **MQTT** define el canal lógico de publicación de los datos [AWS, 2026g, OASIS, 2019, Rescorla, 2018, AWS, 2026j,f,i].

En la implementación se utilizó la política ESP32C6-Policy y el tópico sensor/data. La política permite conectar, publicar, suscribirse y recibir mensajes. Es importante tener presente que para una versión escalable a otros dispositivos es necesario restringir `iot:Connect` al `clientId` esperado y

limitar la publicación al tópico requerido. Esta restricción evita que un certificado comprometido publique en rutas ajenas al prototipo o suplante otros dispositivos [AWS, 2026f,j,i, National Institute of Standards and Technology, 2019].

La regla IoT también tiene como propósito desacoplar el firmware de los servicios posteriores. La ESP3 no requiere comunicarse con **DynamoDB**, o con **S3**, o con otro servicio de tipo IoT, solo publica en un tópico y se relaciona con **AWS IoT Core**. Una vez se recibe el paquete **json**, la regla IoT procesa el mensaje y ejecuta las acciones configuradas. Esta separación es útil porque permite que la arquitectura de hardware no genere múltiples conexiones a AWS y que se pueda modificar la arquitectura de software sin tocar el hardware siempre que se conserve la estructura **JSON** que se recibe en AWS IoT Core [AWS, 2026g,h, Bray, 2017, AWS, 2026n, Marjani et al., 2017].

4.5. Ruta de último dato: DynamoDB

La ruta *Latest* se plantea dada la necesidad de conocer el estado actual del sistema priorizando costos bajos de consulta y visualización. En el dashboard presentado, las cards de temperatura, humedad, presión, velocidad del viento y otras variables no requieren consultar los datos históricos permanentemente, solo se necesita el último registro válido por dispositivo. Por esta razón, se implementó la tabla `turbina_latest` en **DynamoDB**, con `clientId` como clave de partición [AWS, 2026c,d,n].

Tabla nro. 4.2: Diseño lógico de la ruta de último dato en DynamoDB.

Elemento	Valor implementado o recomendado	Función
Tabla	<code>turbina_latest</code>	Estado actual del dispositivo
Clave de partición	<code>clientId</code>	Identificación del gateway/dispositivo
Operación de escritura	<code>PutItem</code> o actualización por mensaje	Sobrescribe el último estado
Operación de lectura	<code>GetItem</code>	Consulta rápida desde <code>GetLatest</code>
Consumidor	Dashboard React	Tarjetas de último dato

El uso de **DynamoDB** evita ejecutar consultas a **Athena** para refrescar el estado operativo del dashboard. Esto permite reducir latencia, costo y complejidad al depender solo de un servicio y resolución de la consulta por clave y no por filtrado sobre el histórico. Además, con la estructura planteada, se puede escalar muy fácilmente a múltiples turbinas u otros dispositivos a partir de un `clientId` diferente por dispositivo. Por la naturaleza de la arquitectura planteada, las estructuras de datos son flexibles y pueden añadirse nuevos campos para calidad de datos, trazabilidad de actualizaciones, o lo que el usuario requiera [AWS, 2026c,d,b,n].

4.6. Ruta histórica: Amazon S3

La ruta histórica almacena todos los mensajes recibidos desde la ESP3, la cual envía una estructura **json** con las variables y metadata definida cada diez segundos y que se almacenan en **Amazon S3**. Estos datos se almacenan en una estructura particionada por niveles a partir del tiempo **DD/MM/YY HH:mm:ss** que permite hacer uso de datos específicos y con alto nivel de granularidad. **S3** se usa como lago de datos por su durabilidad, bajo costo, capacidad de organizar objetos mediante prefijos y otras bondades como el control de versiones, niveles de seguridad, entre otros. En la implementación se usó un bucket llamado `datos-turbina` y una estructura temporal denominada `processed/`, con particiones por año, mes, día, hora y minuto (aproximadamente 5 paquetes/minuto). Esta organización facilita que **Athena** reduzca los datos escaneados cuando las consultas filtran por campos de partición, mejorando latencia y costos por consulta [Bray, 2017, AWS, 2026m,b,a, Hashem et al., 2015, Marjani et al., 2017].

```
s3://datos-turbina/processed/year=2026/month=05/day=18/hour=15/minute=22/
```

Se definió usar la partición temporal planteada porque la telemetría IoT tiende a crecer de forma acumulativa, por ejemplo, en función del tiempo o evento que desencadena el muestreo. Si todos los **json** se almacenan en una única carpeta, se tendrán que escanear muchos objetos en cada consulta para encontrar lo requerido. Por el contrario, al particionar por timestamp, una consulta para un día u hora específica puede limitarse a los prefijos correspondientes. Esto mejora la latencia, reduce bastante los costos por consulta y facilita la navegación y mantenimiento del data lake del histórico [AWS, 2026m,b,a, Hashem et al., 2015, Marjani et al., 2017].

Una limitación de la versión actual es que los mensajes **json** pueden generar muchos objetos en **S3** si la frecuencia de publicación aumenta. Esta es una estrategia funcional para prototipado y

validación, pero en una arquitectura más madura y con varios dispositivos si conviene compactar los datos por ventanas de tiempo y convertirlos a **Parquet**, reduciendo la cantidad de objetos y volumen escaneado por **Athena** [Bray, 2017, AWS, 2026m,b,n, Apache Software Foundation, 2026, Marjani et al., 2017].

4.7. Catálogo y transformación con AWS Glue

AWS Glue cumple dos funciones en la arquitectura: 1) catalogar los datos históricos y 2) habilitar transformaciones hacia formatos más eficientes como **Parquet**. El Glue Data Catalog mantiene la base `iot_db` y la tabla `raw_processed`, que describe el esquema disponible para consultas con **Athena**. Sin catálogo, **Athena** no tendría una definición estructurada de columnas, tipos y particiones sobre los objetos almacenados en **S3** [AWS, 2026a,e,m,b].

Adicionalmente, la arquitectura también contempla trabajos Glue como Job ETL proceso y `json_to_parquet_converter`. El objetivo es convertir datos **json** hacia **Parquet** y preparar una capa optimizada para consulta de histórico en un entorno de múltiples dispositivos y demasiados datos. Se plantea para preparar una estructura más coherente con el diseño del data lake: primero se captura el dato crudo o procesado en **json** para trazabilidad; luego se transforma a un formato columnar para análisis eficiente [Bray, 2017, Amazon Web Services, 2026f, Apache Software Foundation, 2026, AWS, 2026b, Marjani et al., 2017].

Tabla nro. 4.3: Capas de datos previstas para el histórico IoT.

Capa	Formato	Uso	Ventaja
Entrada histórica	json	Persistencia directa desde IoT	Simple, trazable y flexible
Catálogo	Glue Data Catalog	Esquema y particiones	Permite consulta SQL
Optimizada	Parquet	Consulta analítica	Reduce lectura y costo

4.8. Consulta histórica con Amazon Athena

Athena es un servicio de AWS que permite hacer consultas de datos en **S3** usando SQL **sin cargar previamente la información en una base de datos relacional**, lo que permite reducir costos. En este proyecto **Athena** ejecuta consultas sobre la tabla `iot_db.raw_processed`, filtrando por particiones temporales y ordenando por la variable `timestamp`. Esta capacidad es adecuada para el dashboard histórico, en el cual el usuario puede solicitar series de las diferentes variables en un intervalo de tiempo determinado [Amazon Web Services, 2026b, AWS, 2026a,b,m].

Un ejemplo lógico de consulta histórica es:

```
SELECT *
FROM iot_db.raw_processed
WHERE year = '2026'
      AND month = '05'
      AND day = '15'
ORDER BY "timestamp" DESC
LIMIT 20;
```

Debido a las necesidades identificadas en este proyecto, se considera obligatorio el uso de particiones de `timestamp` para las consultas desde **Lambda**. Si la función **Lambda** `GetTimeseries` permite rangos muy amplios o no filtra por `year`, `month` y `day`, **Athena** podría escanear más datos de los requeridos y aumentar la latencia o costos innecesarios que encarecen la arquitectura. Por esta razón, en la función **Lambda** debe implementarse validación de fechas, limitar resultados y mapear explícitamente las variables permitidas, evitando usar SQL con parámetros no controlados que puedan generar errores en las consultas [AWS, 2026b,k,n, Amazon Web Services, 2026a, OWASP Foundation, 2026].

4.9. Capa API con Lambda Function URLs

Para reducir costos, la capa API para interconectar el dashboard con la fase de almacenamiento se implementa mediante dos funciones de **AWS Lambda** expuestas mediante Function URLs. Este mecanismo permite la creación de endpoints HTTPs directos para funciones predeterminadas con configuración **CORS** y sin desplegar una API REST por completo. Para el propósito de este

proyecto, esta forma soluciona la necesidad, mantiene el diseño **serverless** y puede escalarse de manera segura [AWS, 2026k,l,n, Amazon Web Services, 2026h].

Las dos funciones son GetLatest y GetTimeseries. GetLatest gestiona todo el ecosistema del “último dato” consultando **DynamoDB** por clientId. GetTimeseries gestiona la ruta para la data histórica construyendo una consulta SQL mediante **Athena** y devuelve puntos para la visualización en el dashboard. Esta división de tareas evita mezclar dos patrones de acceso diferentes dentro de una única función y facilita asignar permisos mínimos a cada rol **IAM** conservando la práctica de mínimo privilegio [AWS, 2026k,d, Amazon Web Services, 2026b, AWS, 2026b, Amazon Web Services, 2026h].

Tabla nro. 4.4: Funciones Lambda principales de la capa API y procesamiento.

Función	Fuente	Consulta	Uso
GetLatest	DynamoDB turbina_latest	Lectura por clientId	Cards de último dato
GetTimeseries	Athena raw_processed	Rango temporal	Gráfico histórico
Actualizar UltimoDato Dynamo	Regla IoT	Actualiza latest	Mantener último dato

Las Function URLs deben responder con encabezados **CORS** adecuados para el dominio del dashboard. Durante la depuración se identificó que duplicar Access-Control-Allow-Origin puede producir errores en navegador; por ello, la arquitectura debe definir una sola fuente de verdad para **CORS**: o se configura en la Function URL o se controla en el código **Lambda**, pero no de manera conflictiva en ambos lugares. Para despliegue público, el origen permitido debe restringirse al dominio **CloudFront** del dashboard [AWS, 2026l, Mozilla Developer Network, 2026, Amazon Web Services, 2026d,h].

4.10. Roles IAM y mínimo privilegio

IAM es un servicio de AWS que se especializa en seguridad y permite que cada servicio de la arquitectura actúe sobre otros recursos sin exponer credenciales estáticas. La arquitectura debe asignar permisos según las necesidades que llevan a la creación de cada función: GetLatest

requiere permisos para leer **DynamoDB**, mientras que **GetTimeseries** requiere ejecutar **Athena**, leer metadatos de Glue y acceder a resultados en **S3**. Por ejemplo, una regla IoT que escribe en **S3** o invoca **Lambda** no debería tener permisos sobre recursos ajenos, Glue debe limitarse a rutas y bases del proyecto y **AWS IoT Core** debe escribir en **S3** únicamente mediante una regla IoT autorizada por un rol **IAM**, no mediante credenciales expuestas en alguna ESP. Allí radica la importancia de **IAM** [Amazon Web Services, 2026h,j, AWS, 2026h, Amazon Web Services, 2026i,g,l].

Tabla nro. 4.5: Permisos mínimos esperados por servicio.

Rol o función	Permiso mínimo	Recurso esperado
GetLatest-role	dynamodb:GetItem	Tabla turbina_latest
GetTimeseries-role	athena:StartQueryExecution athena:GetQueryExecution athena:GetQueryResults	Workgroup Athena del proyecto
GetTimeseries-role	glue:GetDatabase glue:GetTable glue:GetPartitions	Base iot_db Tabla raw_processed
GetTimeseries-role	s3:GetObject s3:PutObject s3:ListBucket	Buckets de datos y resultados
AWSIoTRule ActionRole	s3:PutObject dynamodb:PutItem lambda:InvokeFunction	Recursos específicos del proyecto
AWSGlue ServiceRole-IoT	Lectura/escritura controlada	Prefijos processed/ y optimized-parquet/

Por otra parte, en las buenas prácticas de seguridad de la arquitectura de AWS se recomienda implementar el principio de mínimo privilegio, el cual consiste en otorgar los permisos mínimos indispensables para realizar las funciones necesarias a personas, programas o procesos. Esto también facilita la depuración, un rol granular permite identificar rápidamente qué permiso falta cuando algo falla, en comparación con roles con permisos muy amplios [Amazon Web Services, 2026h, AWS, 2026n,i, National Institute of Standards and Technology, 2019].

4.11. Dashboard web con React, S3 y CloudFront

En este proyecto la visualización se implementa como una aplicación **React/Vite**, en comparación con la visualización de referencia implementada con una ShinyApp de R. El frontend representado mediante un dashboard consulta GetLatest para actualizar cards con el último dato para diferentes variables y consulta GetTimeseries para construir gráficos históricos con Recharts. Esta separación permite que el usuario visualice el estado operativo y, al mismo tiempo, explore el comportamiento temporal de variables como velocidad del viento, temperatura o voltaje [Gómez Flórez, 2024, Meta Open Source, 2026, Vite, 2026, Recharts, 2026, AWS, 2026k].

El despliegue del dashboard se realiza como sitio estático en **S3** y distribución mediante **CloudFront**. En este patrón, **S3** funciona como origen del contenido y **CloudFront** como capa de distribución global y control de acceso. Para una Single Page Application, el objeto raíz debe ser index.html, y en una configuración completa se deben mapear errores 403/404 hacia index.html para soportar rutas del lado del cliente [Amazon Web Services, 2026m,d,c, Vite, 2026].

Por otra parte, desde la visión de la seguridad, el bucket donde se aloja el dashboard no debería exponerse públicamente si el servicio de AWS **CloudFront** es la puerta de entrada. La práctica recomendada es usar Origin Access Control u otro mecanismo equivalente para que los usuarios accedan al frontend mediante **CloudFront** y no directamente al bucket. También se recomienda restringir **CORS** en las Function URLs al dominio final del dashboard [Amazon Web Services, 2026e,l, AWS, 2026l, Amazon Web Services, 2026h].

4.12. Evidencias de implementación

Aunque el capítulo principal utiliza una figura limpia de arquitectura, se usan capturas de consola como evidencias de implementación [AWS, 2026g,n].

Tabla nro. 4.6: Evidencias recomendadas para documentación y validación.

Evidencia	Qué demuestra	Ubicación recomendada
S3 datos-turbina	Existencia del lago de datos y prefijos	Anexo o capítulo 5

Evidencia	Qué demuestra	Ubicación recomendada
DynamoDB turbina_latest	Último dato por <code>clientId</code>	Capítulo 5
Athena iot_db.raw_processed	Consulta histórica exitosa	Capítulo 5
Glue iot_db	Catálogo de datos	Anexo técnico
Lambda	Funciones de consulta/procesamiento	Anexo técnico
CloudFront	Dashboard publicado	Capítulo 5
Política IoT	Autorización MQTT	Anexo de seguridad

4.13. Síntesis del capítulo

La arquitectura cloud desarrollada en AWS convierte los mensajes publicados por la ESP3 en información operativa e histórica útil para el usuario. **AWS IoT Core** recibe los datos por **MQTTs**, **DynamoDB** conserva el último dato filtrando por `clientId`, **S3** almacena el histórico particionado, Glue cataloga los datos, **Athena** permite hacer consultas SQL a **S3**, **Lambda** expone endpoints HTTP y **CloudFront** despliega el dashboard web. Esta organización evita concentrar toda la solución en un único servicio y permite optimizar cada ruta según sus necesidades de uso [AWS, 2026g,h,c,m,a,k].

La propuesta final queda plenamente relacionada con el flujo y necesidades reales del sistema: Gateway ESP3, **AWS IoT Core**, regla IoT, **DynamoDB/S3**, Glue/**Athena**, **Lambda** Function URLs y React/**CloudFront**. El resultado obtenido es una arquitectura **serverless**, modular, segura, trazable y escalable para monitoreo de generación eólica de pequeña escala, con separación entre ingesta segura, almacenamiento híbrido, consulta histórica y visualización web, que puede adaptarse en otro tipo de procesos de generación eléctrica de cualquier escala o en otros ámbitos donde se requiera medición y almacenamiento de datos [Gómez Flórez, 2024, Botta et al., 2016, Marjani et al., 2017, AWS, 2026n,k, Amazon Web Services, 2026d].

Pruebas de despliegue y validación

En este capítulo se presenta la estrategia de validación del sistema desarrollado. Este proceso consiste en verificar progresivamente que cada etapa cumpla con su función dentro del flujo extremo a extremo que se propone. Este flujo inicia con la medición física de datos en las tarjetas ESP1 y ESP2, comunicación local, serialización hacia ESP3, consolidación de la estructura JSON, publicación a través de MQTTs, recepción e ingesta en AWS IoT Core, persistencia del último paquete en DynamoDB y S3, consistencia en las consultas de Athena, exposición por Lambda y visualización web. Esta organización evita evaluar el sistema únicamente desde el resultado final y permite identificar en qué tramo se origina una falla [ISO/IEC/IEEE, 2018, National Institute of Standards and Technology, 2019, AWS, 2026g,h,c,a].

La validación se diseñó con un enfoque incremental. Primero se verifican sensores y periféricos de manera aislada; luego se prueban las tres ESP32-C6 por separado; posteriormente se validan los enlaces ESP-NOW y UART; después se verifica la publicación desde ESP3 hacia AWS IoT Core; y finalmente se comprueba la persistencia, consulta y visualización. Esta secuencia es coherente con una arquitectura distribuida, ya que evita depurar simultáneamente errores de hardware, firmware, red, nube y frontend [ISO/IEC/IEEE, 2018, National Institute of Standards and Technology, 2019, Espressif Systems, 2026f,d,l, AWS, 2026g].

5.1. Estrategia general de validación

Para la validación del sistema se proponen cinco niveles con el propósito de medir todo el proceso: 1) El nivel uno corresponde a la validación física de mediciones (sensores tanto en

ESP1 como en ESP2), 2) El nivel dos busca validar el correcto funcionamiento del firmware a partir de la evaluación de las tareas concurrentes, 3) El nivel tres se encarga de la validación del funcionamiento de la comunicación local (tanto ESP-NOW entre ESP1-ESP2 y comunicación UART entre ESP2-ESP3), en el nivel cuatro se hace la validación de la arquitectura Cloud propuesta y en el nivel cinco a la validación del funcionamiento esperado del dashboard. La Tabla 1 resume esta estructura.

Tabla nro. 5.1: Niveles de validación propuestos para el sistema extremo a extremo.

Nivel	Alcance	Pregunta de validación	Evidencia esperada
V1	Sensores y señales	¿La variable física se captura con rango y unidad coherentes?	Lecturas por consola, comparación con patrón o instrumento externo
V2	Firmware por nodo	¿Cada ESP ejecuta sus tareas sin bloqueo crítico?	Logs, LED de estado, memoria disponible, reinicios controlados
V3	Comunicación local	¿ESP1, ESP2 y ESP3 intercambian datos completos?	Recepción ESP-NOW, json por UART, estado de conexión
V4	Nube AWS	¿El mensaje llega y se almacena correctamente?	IoT Core, DynamoDB, S3, Glue, Athena
V5	Dashboard	¿El usuario consulta último dato e histórico?	Tarjetas, gráfico, manejo de errores y CORS

La validación de la herramienta es una etapa muy importante, pues no debe limitarse únicamente a comprobar que se visualizan datos en pantalla. El sistema de monitoreo pretende tener disponible un medio para verificar que esos datos conservan significado físico, que están entre rangos esperables, que el mensaje mantiene una estructura estable, que las rutas cloud almacenan la información como se requiere y que el usuario puede consultar datos sin depender de la consola

de AWS o de un servidor local. Por esta razón, cada prueba registra la entrada, procedimiento, criterio de aceptación y evidencia.

5.2. Validación de la ESP1: adquisición de campo

La ESP1 se valida como nodo de adquisición en el generador. Sus pruebas se orientan a comprobar tres funciones: lectura GPS, medición de voltaje y estimación de RPM para la velocidad de la turbina. Para la lectura del GPS, la evidencia mínima es observar coordenadas válidas y estado `gps_fix` coherente. Para voltaje, la evidencia mínima es comparar la lectura calculada por firmware contra un multímetro o instrumento de medición. Para RPM, la evidencia mínima es contrastar el conteo de pulsos con una velocidad conocida o con un tacómetro externo.

Tabla nro. 5.2: Casos de validación del nodo ESP1.

Código	Prueba	Procedimiento	Criterio de aceptación
ESP1-01	GPS	Encender el módulo, esperar fijación y registrar latitud/longitud	<code>gps_fix=true</code> y coordenadas dentro del rango esperado
ESP1-02	Voltaje	Aplicar diferentes niveles de tensión acondicionada y comparar con patrón	Error dentro del rango definido para prototipo
ESP1-03	RPM	Generar pulsos controlados o rotación conocida	RPM proporcional a pulsos y ventana temporal
ESP1-04	ESP-NOW TX	Enviar estructura <code>mensaje_t</code> hacia ESP2	Confirmación por log de envío y recepción en ESP2

Con la medición de voltaje es necesario tener un cuidado especial, pues se trata de un circuito analógico, calibración **ADC** y cálculo digital. Adicionalmente, deben limitarse los rangos de voltaje en la instrumentación de manera que no se afecten los periféricos de la ESP1. El criterio

de aceptación se define de acuerdo con la necesidad actual, la cual consiste en validar el funcionamiento de la arquitectura, considerando que los datos de voltaje ya fueron contrastados frente a multímetro y osciloscopio en la tesis de maestría sobre el prototipado y análisis comparativo de dos turbinas eólicas de eje vertical [Gómez Flórez, 2024]. Por esta razón, en esta etapa se verifica principalmente que la lectura sea estable, que se mantenga dentro de rangos seguros para el ADC y que pueda integrarse correctamente al flujo de datos del sistema [Espressif Systems, 2026a, Kester, 2005, National Institute of Standards and Technology, 2019].

5.2.1. Evidencias (Prueba ESP1)

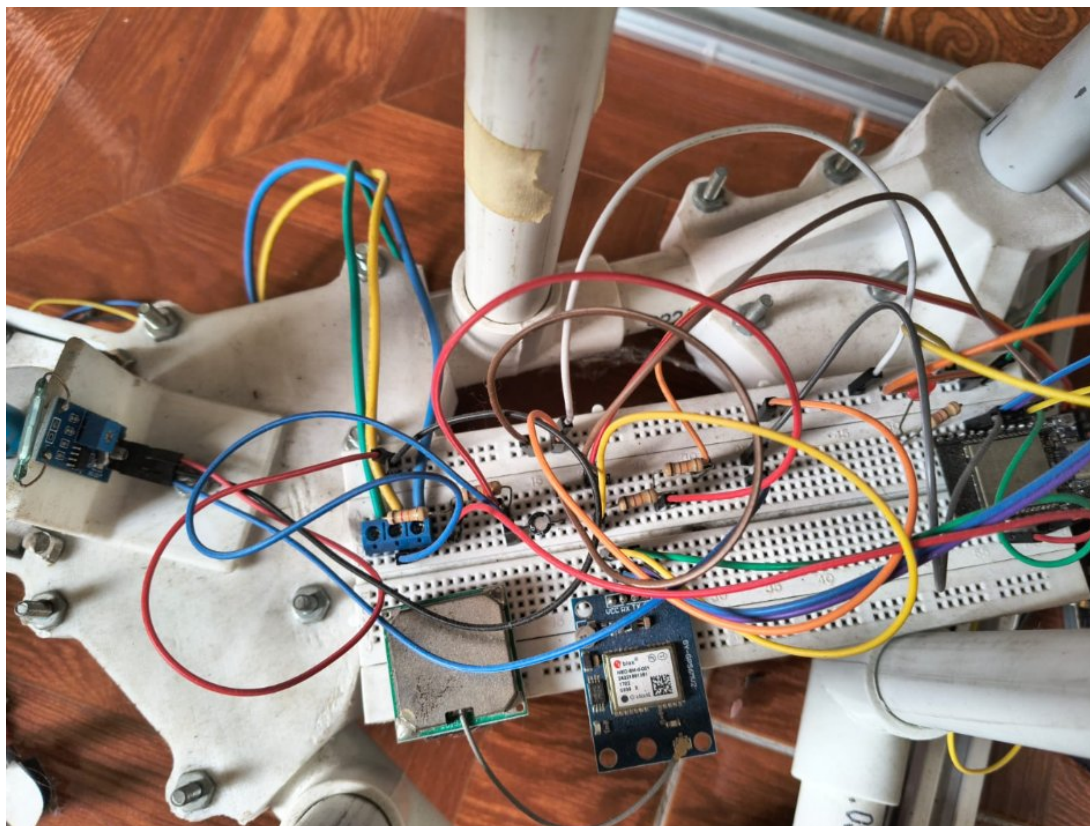


Figura nro. 5.1: ESP1 desenergizada: validación de condición de pérdida de enlace y valores controlados para voltaje y RPM.

Se desenergiza la tarjeta transmisora (ESP1) y se verifica que el sistema marque pérdida de enlace (`esp_now_connected=false`) y que `voltage` y `rpm` adopten valores controlados para evitar interpretaciones físicas erróneas.

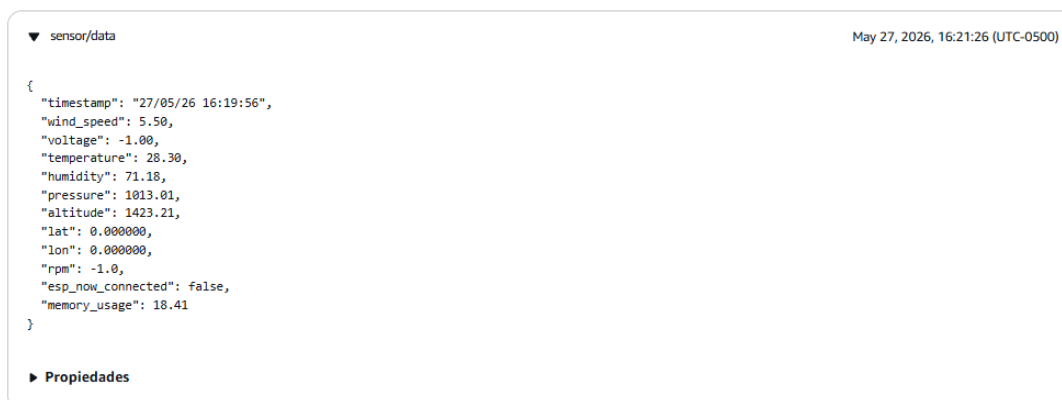


Figura nro. 5.2: Cliente MQTT - esp_now_connected false

Se inspecciona en el cliente de prueba MQTT de AWS IoT Core el último mensaje publicado y se confirma la marca `esp_now_connected=false`, junto con el tratamiento de dato perdido en las variables provenientes de ESP-NOW.

Se energiza nuevamente ESP1 mediante el PC y se valida en la consola del entorno esp-idf (VS Code) el restablecimiento del envío de datos y la recepción por parte de ESP2.

5.2.2. Evidencias (Prueba ESP2)

Se desenergiza la tarjeta ESP2 para inducir la interrupción del flujo hacia el gateway, validando que la arquitectura refleje la pérdida del componente intermedio.

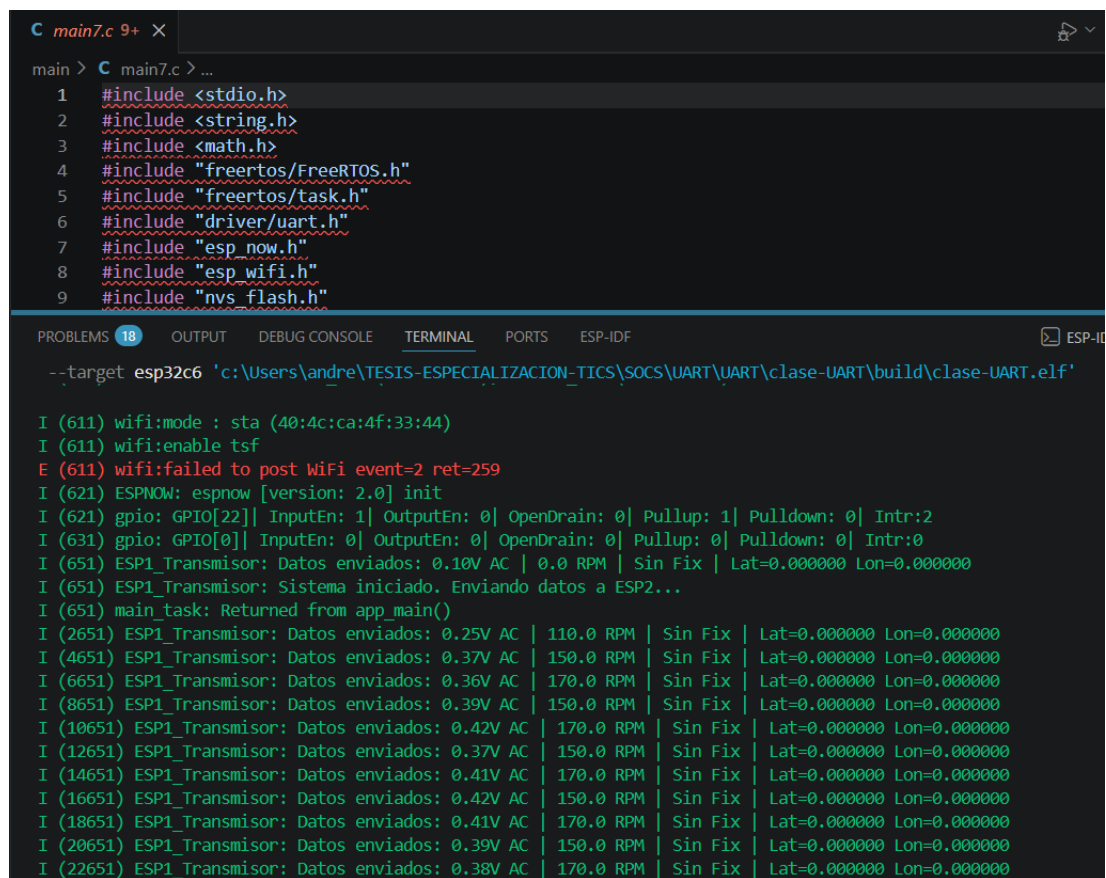
Con ESP2 desenergizada, se verifica en el cliente MQTT que no ingresan nuevos mensajes al tópico, confirmando el comportamiento esperado cuando el puente UART no está operativo.

Se energiza nuevamente ESP2 desde el PC y se verifica por consola el cumplimiento de la tarea de lectura/ensamble y la transmisión del json mediante UART hacia el gateway.

5.2.3. Evidencias (Prueba ESP3)

Se desenergiza la ESP3 (gateway) para validar el escenario de indisponibilidad del componente que publica hacia AWS IoT Core.

Con el gateway sin energía, se confirma en el cliente MQTT que no se reciben mensajes en el tópico de ingesta, lo cual valida la dependencia del flujo respecto a la ESP3.



```
C main7.c 9+ X
main > C main7.c > ...
1 #include <stdio.h>
2 #include <string.h>
3 #include <math.h>
4 #include "freertos/FreeRTOS.h"
5 #include "freertos/task.h"
6 #include "driver/uart.h"
7 #include "esp_now.h"
8 #include "esp_wifi.h"
9 #include "nvs_flash.h"

PROBLEMS 18 OUTPUT DEBUG CONSOLE TERMINAL PORTS ESP-IDF
--target esp32c6 'c:\Users\andre\TESIS-ESPECIALIZACION-TICS\SOCS\UART\UART\clase-UART\build\clase-UART.elf'

I (611) wifi:mode : sta (40:4c:ca:4f:33:44)
I (611) wifi:enable tsf
E (611) wifi:failed to post WiFi event=2 ret=259
I (621) ESPNOW: espnow [version: 2.0] init
I (621) gpio: GPIO[22]| InputEn: 1| OutputEn: 0| OpenDrain: 0| Pullup: 1| Pulldown: 0| Intr:2
I (631) gpio: GPIO[0]| InputEn: 0| OutputEn: 0| OpenDrain: 0| Pullup: 0| Pulldown: 0| Intr:0
I (651) ESP1_Transmisior: Datos enviados: 0.10V AC | 0.0 RPM | Sin Fix | Lat=0.000000 Lon=0.000000
I (651) ESP1_Transmisior: Sistema iniciado. Enviando datos a ESP2...
I (651) main_task: Returned from app_main()
I (2651) ESP1_Transmisior: Datos enviados: 0.25V AC | 110.0 RPM | Sin Fix | Lat=0.000000 Lon=0.000000
I (4651) ESP1_Transmisior: Datos enviados: 0.37V AC | 150.0 RPM | Sin Fix | Lat=0.000000 Lon=0.000000
I (6651) ESP1_Transmisior: Datos enviados: 0.36V AC | 170.0 RPM | Sin Fix | Lat=0.000000 Lon=0.000000
I (8651) ESP1_Transmisior: Datos enviados: 0.39V AC | 150.0 RPM | Sin Fix | Lat=0.000000 Lon=0.000000
I (10651) ESP1_Transmisior: Datos enviados: 0.42V AC | 170.0 RPM | Sin Fix | Lat=0.000000 Lon=0.000000
I (12651) ESP1_Transmisior: Datos enviados: 0.37V AC | 150.0 RPM | Sin Fix | Lat=0.000000 Lon=0.000000
I (14651) ESP1_Transmisior: Datos enviados: 0.41V AC | 170.0 RPM | Sin Fix | Lat=0.000000 Lon=0.000000
I (16651) ESP1_Transmisior: Datos enviados: 0.42V AC | 150.0 RPM | Sin Fix | Lat=0.000000 Lon=0.000000
I (18651) ESP1_Transmisior: Datos enviados: 0.41V AC | 170.0 RPM | Sin Fix | Lat=0.000000 Lon=0.000000
I (20651) ESP1_Transmisior: Datos enviados: 0.39V AC | 150.0 RPM | Sin Fix | Lat=0.000000 Lon=0.000000
I (22651) ESP1_Transmisior: Datos enviados: 0.38V AC | 170.0 RPM | Sin Fix | Lat=0.000000 Lon=0.000000
```

Figura nro. 5.3: Consola esp-idf - flujo ESP1 a ESP2

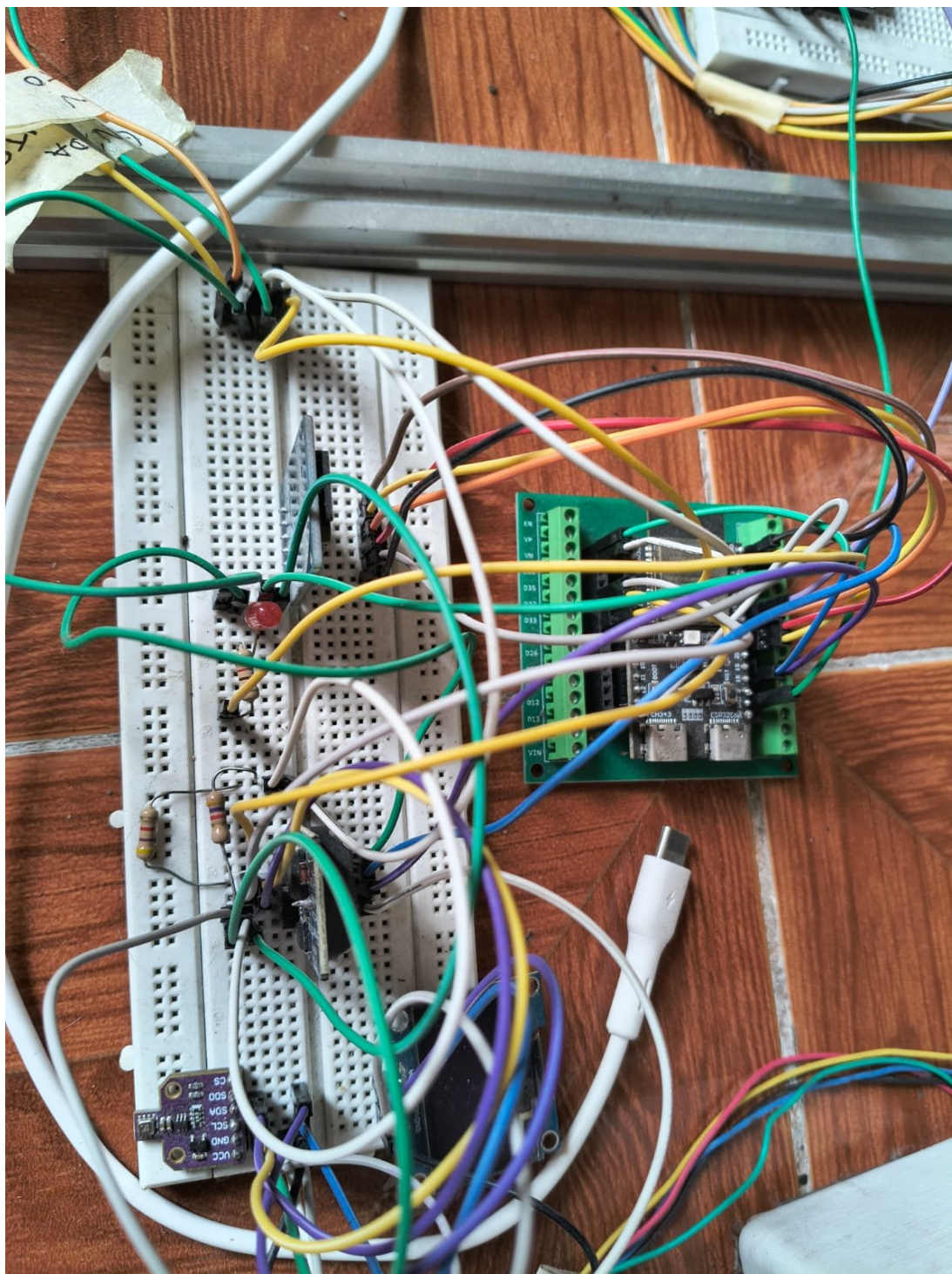


Figura nro. 5.4: ESP2 desenergizada

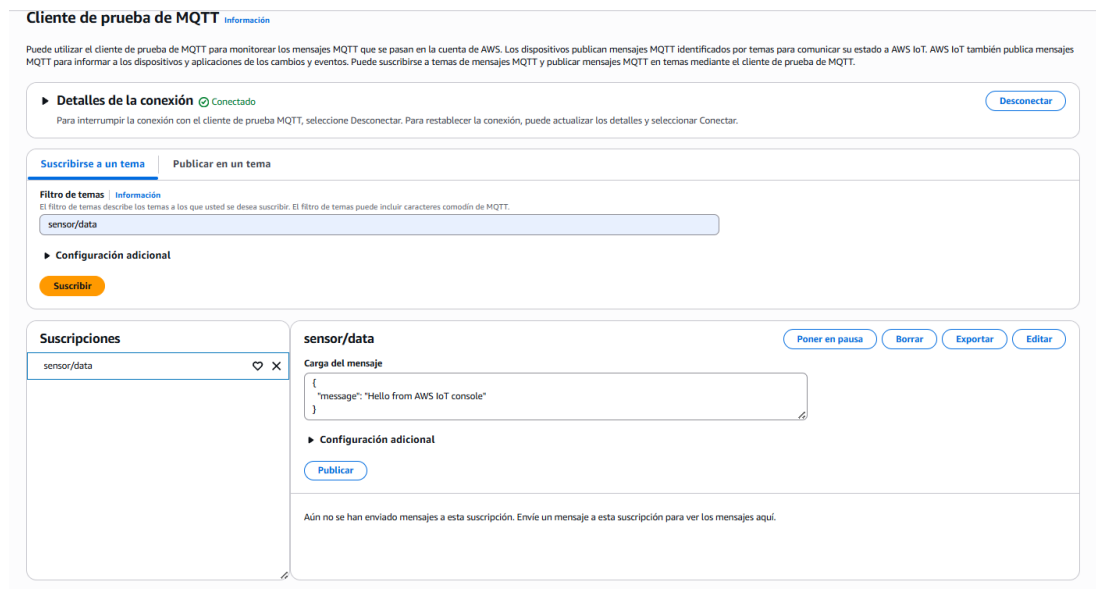


Figura nro. 5.5: Cliente MQTT - sin mensajes ESP2

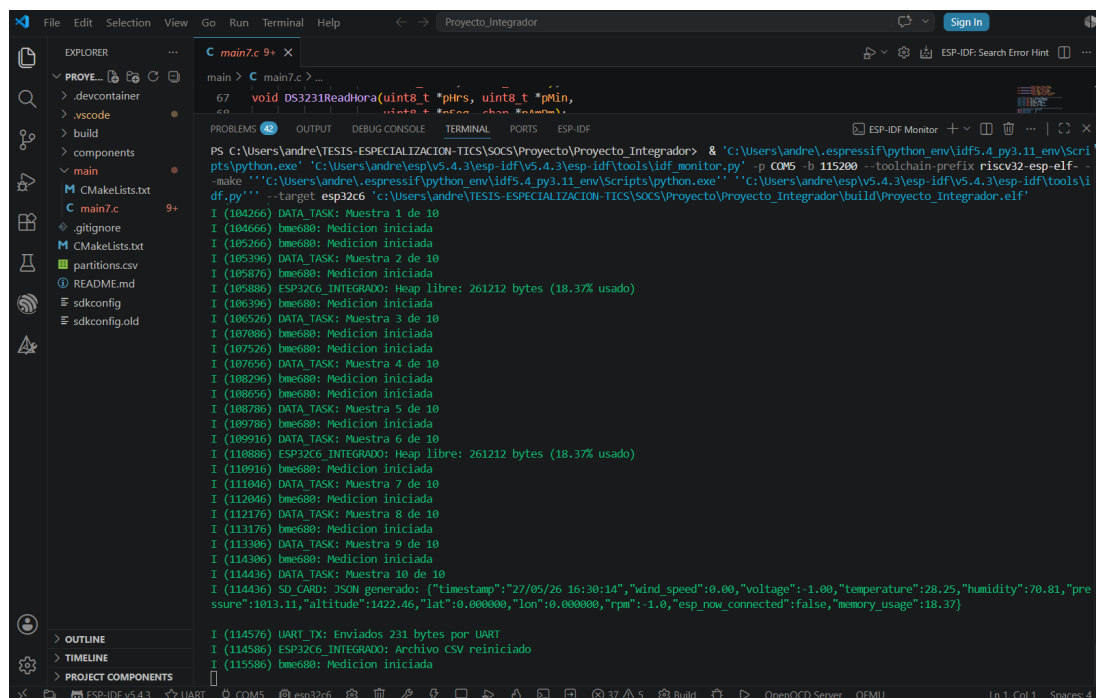


Figura nro. 5.6: Consola esp-idf - UART ESP2

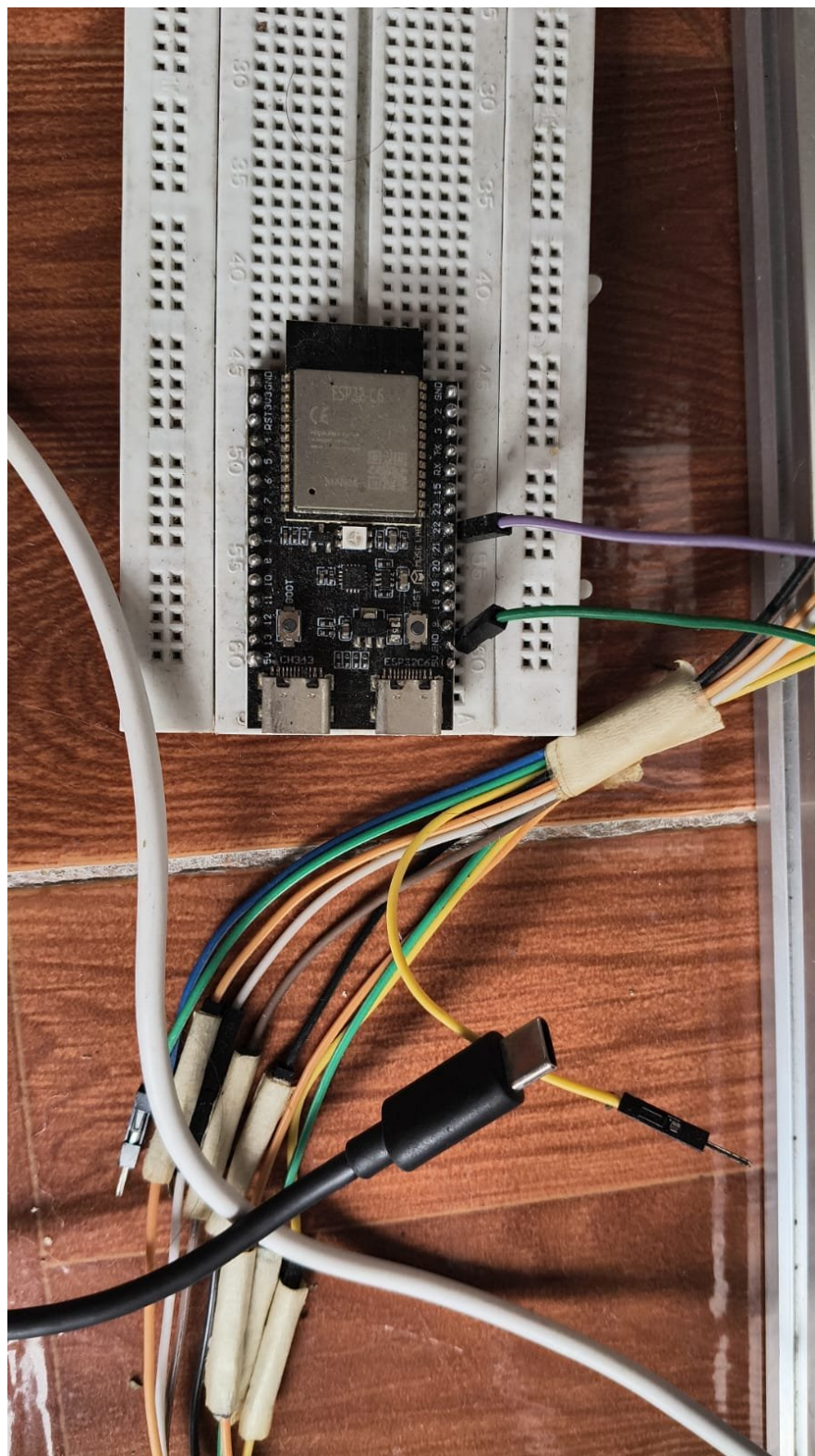


Figura nro. 5.7: ESP3 desenergizada

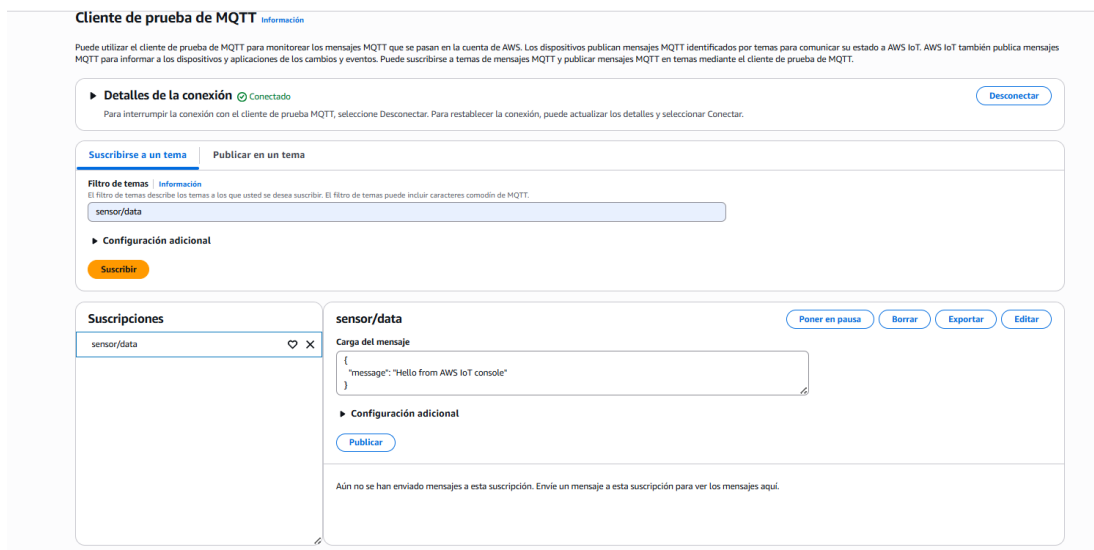


Figura nro. 5.8: Cliente MQTT - sin mensajes ESP3

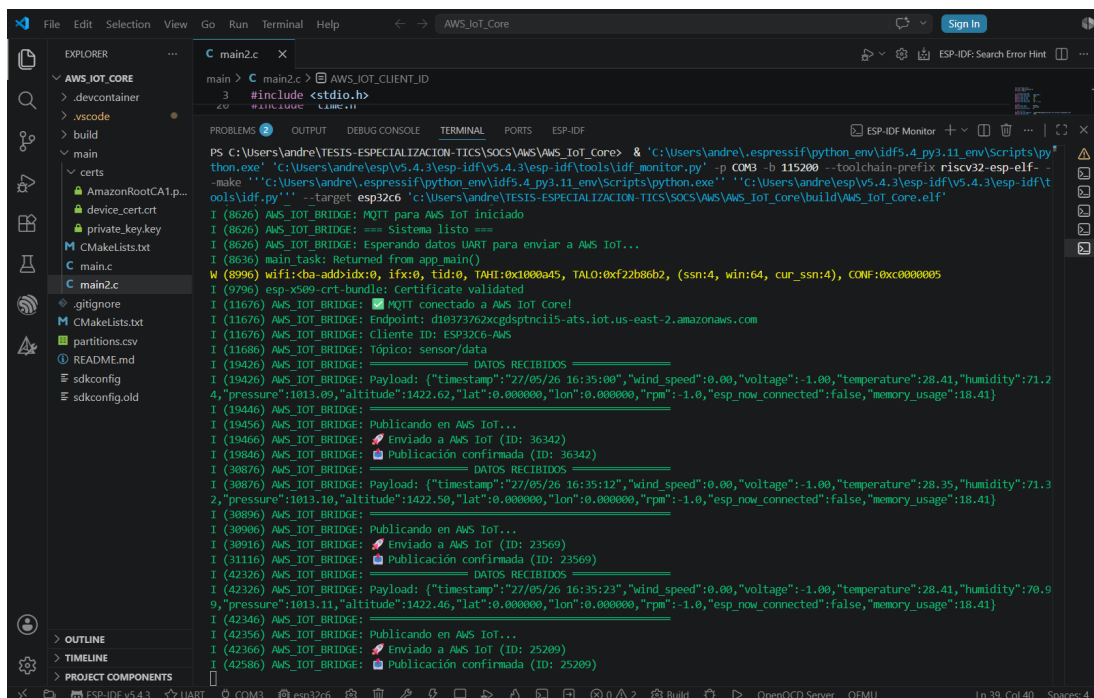


Figura nro. 5.9: Consola esp-idf - MQTT ESP3

Se energiza nuevamente ESP3 y se comprueba por consola la ejecución del firmware y la publicación MQTT hacia AWS IoT Core.

5.2.4. Evidencias (Prueba AWS IoT Core)

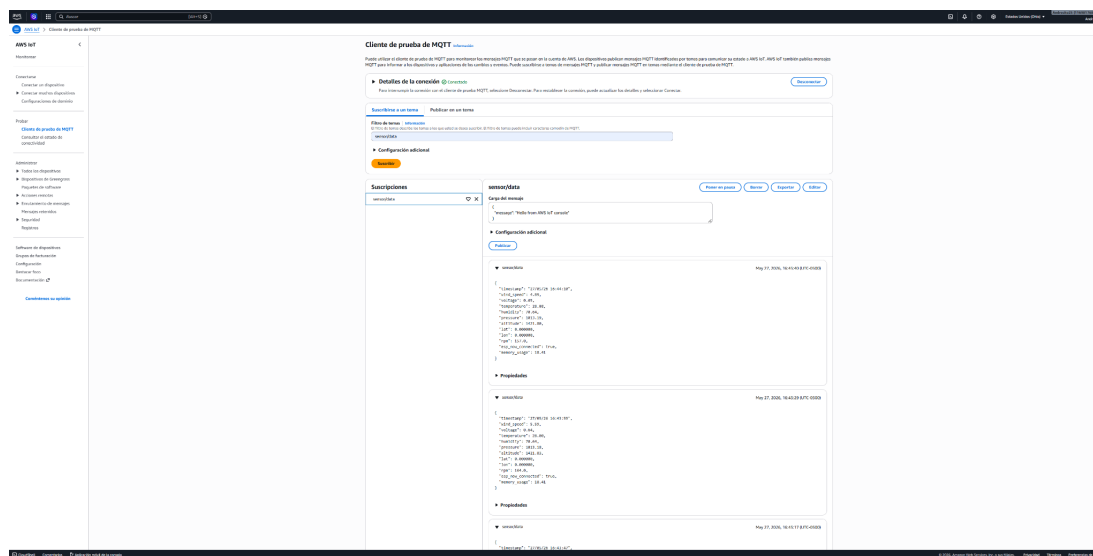


Figura nro. 5.10: IoT Core - recibiendo mensajes

Se suscribe el cliente de prueba MQTT al tópicos configurado y se verifica la recepción de mensajes con estructura json válida.

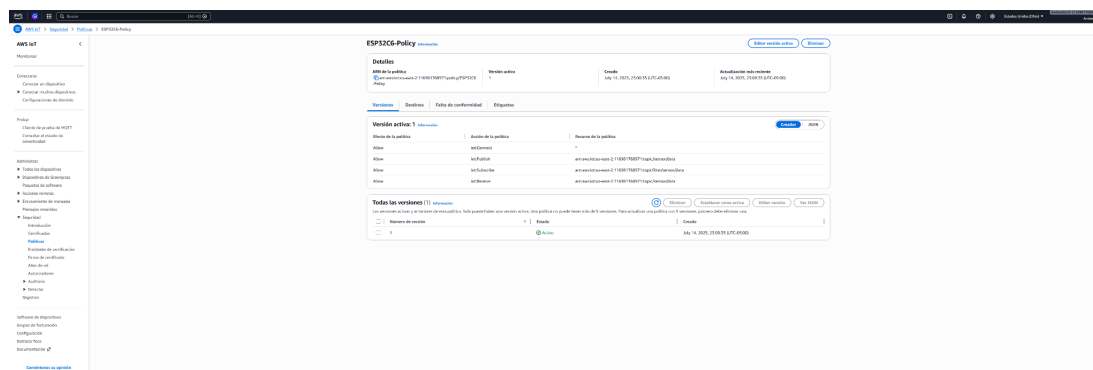


Figura nro. 5.11: Política IoT

Se verifica la política IoT asociada al certificado/dispositivo, confirmando que las acciones permitidas habilitan la conexión y la publicación en el tópicos autorizado.

5.2.5. Evidencias (Prueba DynamoDB)

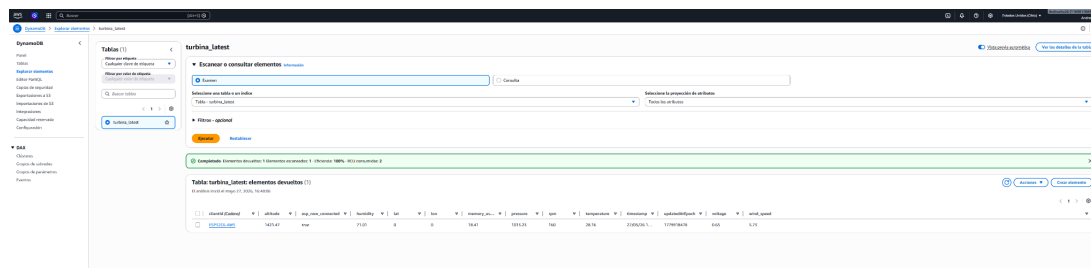


Figura nro. 5.12: DynamoDB turbina_latest

Se valida en la tabla `turbina_latest` la existencia de un único elemento actualizado para el `clientId`, confirmando la persistencia del “último dato” consumido por el dashboard.

5.2.6. Evidencias (Prueba S3 + Glue + Athena)

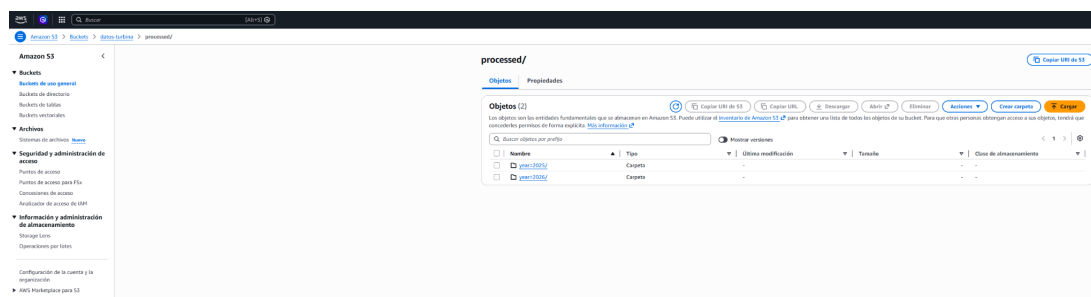


Figura nro. 5.13: S3 processed

Se verifica el bucket `datos-turbina` y la estructura de carpetas/prefijos temporales dentro de `processed/`, evidenciando la persistencia del histórico.

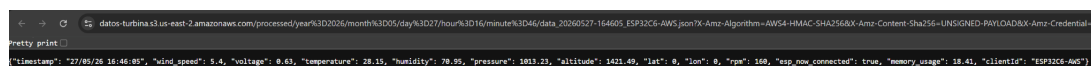


Figura nro. 5.14: S3 objeto json

Se inspecciona un archivo json específico almacenado en S3 y se confirma su contenido y consistencia con el contrato de datos.

Se verifica que AWS Glue reconoce el esquema de la tabla que referencia el histórico en S3, habilitando consultas posteriores en Athena.

[illegible]

Figura nro. 5.15: Glue table

The screenshot shows the Amazon SageMaker Unified Studio interface. The top header includes the Amazon logo and the text 'Realización de consultas en Amazon SageMaker Unified Studio'. Below this, there's a subtitle: 'Utiliza el motor de consultas de análisis de datos de Amazon SageMaker para ejecutar consultas de análisis de datos en la consola.' The left sidebar contains navigation links: 'Inicio', 'Consultas recientes', 'Consultas guardadas', and 'Configuración de consultas'. The main content area is divided into two sections. The top section, 'Datos', shows a table named 'tbl_1' with columns 'id', 'name', 'age', 'gender', 'height', 'weight', 'hair_color', 'eye_color', 'skin_color', 'hair_type', 'hair_color', 'eye_color', 'skin_color', 'hair_type', 'hair_color', 'eye_color', 'skin_color'. The bottom section, 'Consultas de la consola', shows a list of queries with columns 'Nombre de la consulta', 'Estado de la consulta', 'Tiempo de ejecución', 'Tiempo de ejecución', and 'Estado de la consulta'. The first query is 'Consultas' and its status is 'Completado'.

Figura nro. 5.16: Athena consulta

Se ejecuta una consulta en Athena para confirmar que los datos almacenados en S3 son consultables y que la ruta histórica funciona conforme a lo esperado.

5.2.7. Evidencias (Prueba Lambda)

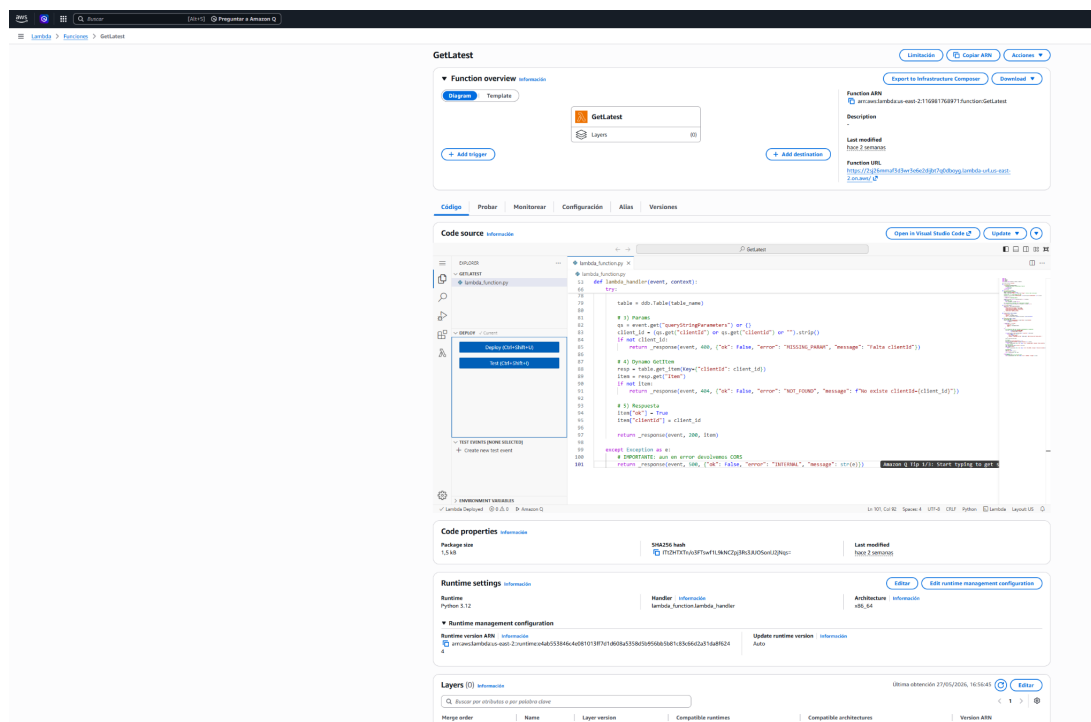


Figura nro. 5.17: Lambda GetLatest

Se verifica la función `GetLatest` y su `Function URL`, validando que la API devuelve el último paquete disponible para un `clientId`.

Se verifica la función `GetTimeseries` y su Function URL, confirmando la respuesta JSON para series temporales parametrizadas por métrica y ventana de tiempo.

5.2.8. Evidencias (Prueba Dashboard)

Se valida el despliegue del dashboard estático en S3, confirmando la disponibilidad de los artefactos del frontend.

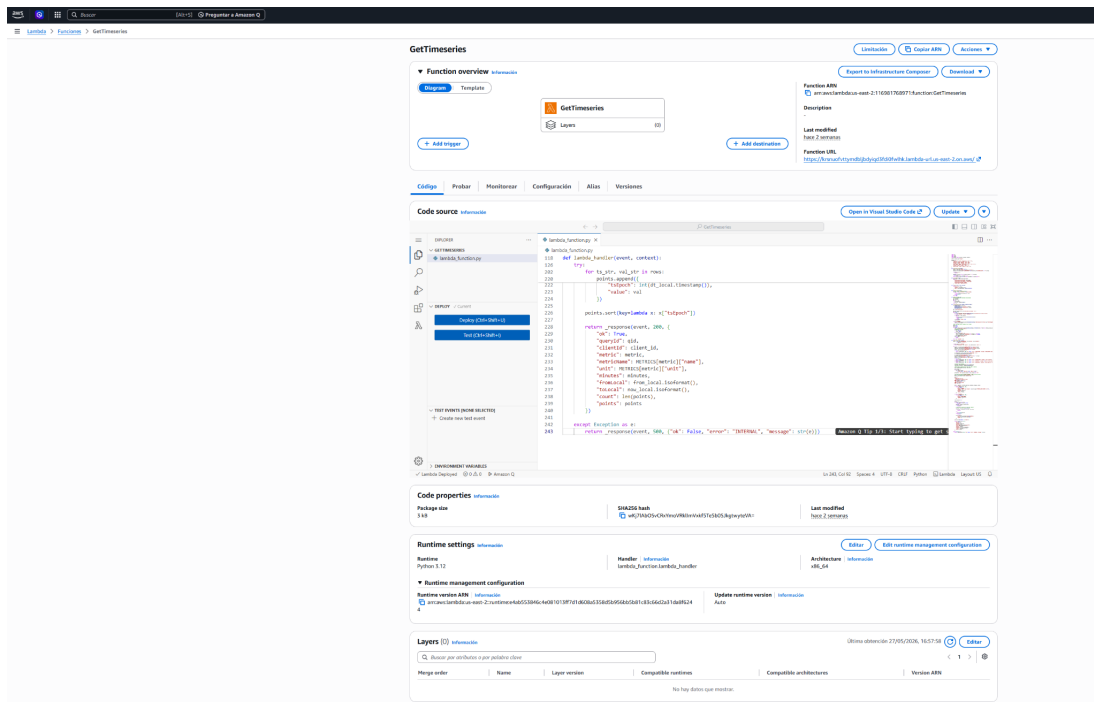


Figura nro. 5.18: Lambda GetTimeseries

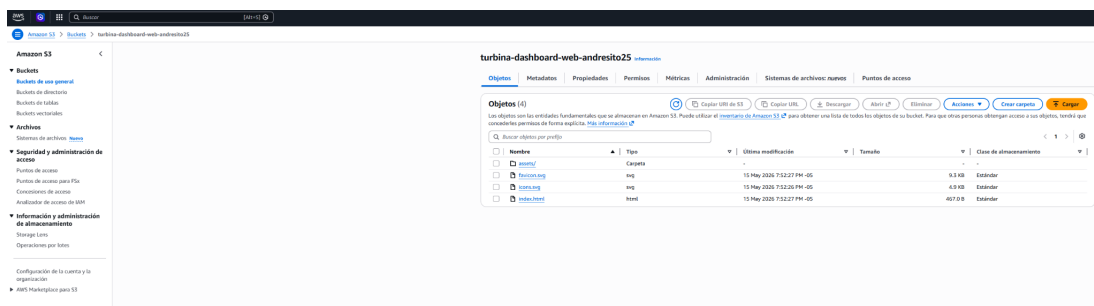
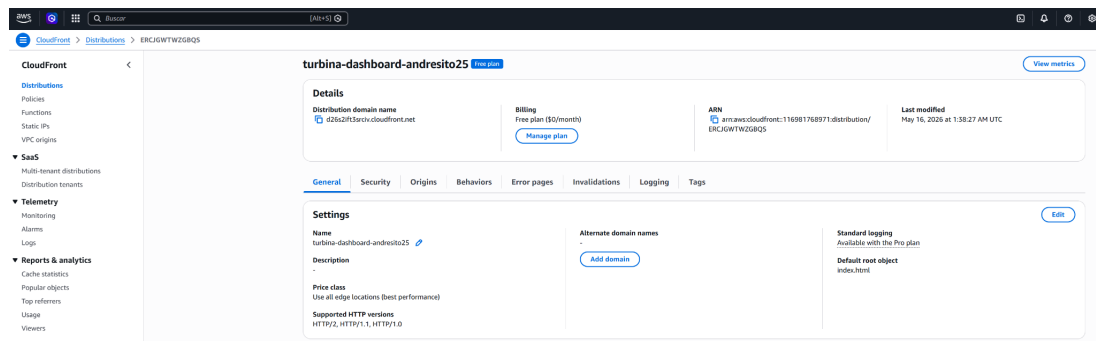


Figura nro. 5.19: Dashboard en S3



[Al-Fuqaha et al., 2015, Quincozes et al., 2019, Botta et al., 2016]

Se verifica la distribución de CloudFront que expone el dashboard, evidenciando la capa de entrega (CDN) sobre el contenido estático.

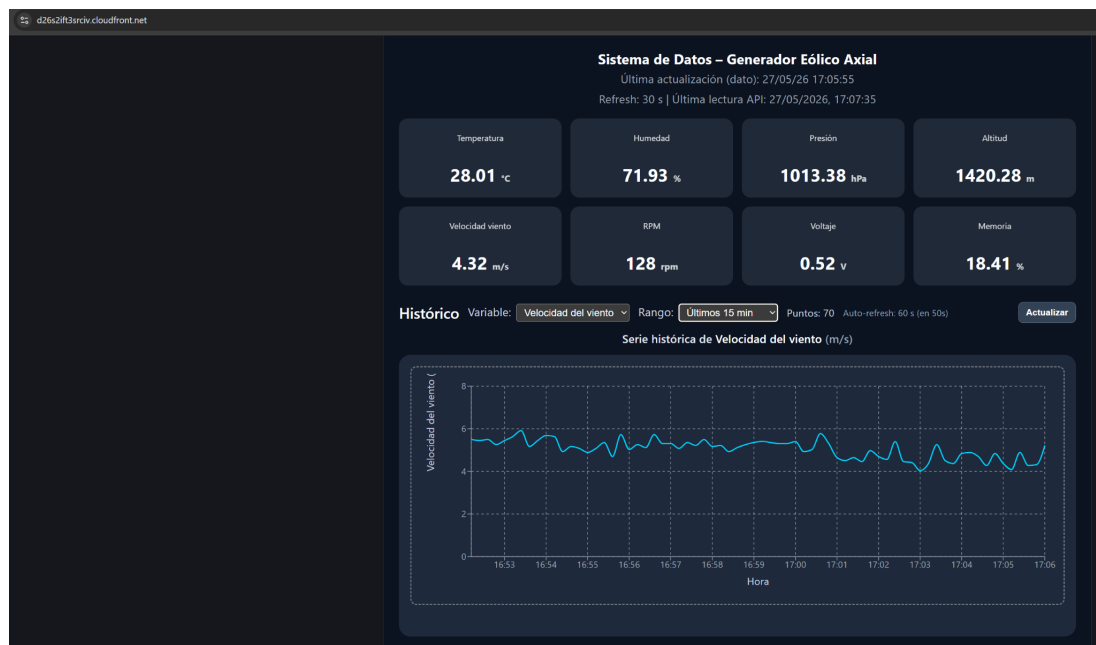


Figura nro. 5.20: Dashboard 15 min

Se valida en el dashboard la consulta histórica (últimos 15 minutos) para la variable de velocidad del viento, confirmando la integración con `GetTimeseries` y el renderizado de la serie.

Análisis de costos, resultados y escalabilidad

En este capítulo se analiza la arquitectura desde los resultados técnicos esperados, modelo de costos y la escalabilidad futura. El propósito no es fijar unos costos definitivos, pues estos pueden variar por múltiples aspectos, pero si proponer una metodología que permita hacer estimaciones de acuerdo a los parámetros con los que se desee configurar la arquitectura, como cambios en la frecuencia de publicación, número de turbinas, tamaño de los mensajes, volumen histórico, cantidad de consultas y servicios activados. Esta aproximación es coherente con una arquitectura **serverless** y basada en servicios administrados, donde el costo depende principalmente del uso, de la separación entre rutas de consulta, del almacenamiento histórico y de la forma como crece la telemetría IoT en el tiempo [AWS, 2026n, Amazon Web Services, 2026n,k, Hashem et al., 2015, Marjani et al., 2017, Botta et al., 2016].

6.1. Resultados técnicos consolidados

Como resultado principal de esta tesis se presenta una arquitectura IoT - Cloud funcional que conecta un sistema físico con una plataforma cloud consultable desde la web y escalable a las necesidades del usuario. La capa de Hardware tiene la capacidad de convertir señales del mundo físico en una estructura de datos de tipo json, coherente y entendible, publica esta telemetría mediante MQTTs en AWS IoT Core y la capa de software en la nube asegura la ingesta, almacenamiento en bases de datos, consulta, procesamiento y visualización mediante un dashboard. Se destaca que todos los componentes del hardware son software libre y open source, las funciones Lambda en AWS están programadas con Python y el dashboard se hizo con React/Vite.

Tabla nro. 6.1: Resultados técnicos consolidados de la arquitectura.

Resultado	Evidencia técnica esperada	Valor para la tesis
Adquisición distribuida	Tres ESP32-C6 con roles diferenciados	Modularidad y trazabilidad
Comunicación local	ESP-NOW y UART	Separación entre borde y gateway
Publicación segura	MQTT/TLS con certificados X.509	Seguridad de transporte e identidad
Último dato	Tabla <code>turbina_latest</code>	Visualización operativa de baja latencia
Histórico	Bucket <code>datos-turbina</code>	Persistencia y análisis posterior
Consulta temporal	Glue + Athena	SQL sobre datos particionados
Dashboard	React/Vite + CloudFront	Exposición web al usuario

El resultado quizá más importante es la integración completa entre capas. Esta arquitectura demuestra que el monitoreo de sistemas físicos, en este caso un generador eólico de eje vertical, puede ser factible con hardware de bajo costo y aplicando técnicas de arquitecturas Cloud que disminuyan los costos de esta y de acuerdo a las necesidades del usuario, planteen seriamente el debate entre si es mejor usar Cloud o computación tradicional.

Esta forma de separar la arquitectura entre una capa física de sensado y una capa de software en la nube también ayuda a entender mejor los costos. Por un lado, están los elementos que se compran, instalan y prueban en campo, como sensores, tarjetas, periféricos, cableado y montaje. Por otro lado, están los costos que crecen con el uso de la plataforma, como los mensajes enviados, el almacenamiento, las consultas, la ejecución de funciones y la distribución del dashboard. De esta manera, si el sistema crece, el análisis puede hacerse con más claridad: no cuesta lo mismo agregar más turbinas, aumentar la frecuencia de publicación, guardar más histórico o tener más usuarios consultando la información [Amazon Web Services, 2026n,k, AWS, 2026n, Marjani et al., 2017].

6.2. Variables del modelo de costos

El costo de la arquitectura depende de parámetros operativos, de una correcta lógica, uso y confiración de los servicios. Los datos más importantes son el número de turbinas o sistemas físicos, el periodo de publicación (en este caso un paquete cada 10 ~ 12 segundos), el tamaño del mensaje, el número de consultas del último paquete de datos (en este caso dos veces por minuto), el número de consultas históricas (las más costosas, deben limitarse), el tamaño de los datos almacenados (cada vez más)y la frecuencia de transformación hacia formatos optimizados (semanal, diaria, horaria). La Tabla nro. 6.2 define las variables principales identificados.

Tabla nro. 6.2: Variables del modelo paramétrico de costos.

Símbolo	Descripción	Unidad
N	Número de turbinas o dispositivos	dispositivos
T_s	Periodo de publicación	segundos/mensaje
B_m	Tamaño promedio del mensaje json	bytes
D	Días de operación al mes	días
Q_L	Consultas mensuales de último dato	consultas/mes
Q_H	Consultas mensuales históricas	consultas/mes
S_H	Datos escaneados por consulta Athena	MB/consulta
R	Registros generados por mes	registros/mes

El número de registros mensuales se estima como:

$$R = N \cdot D \cdot 24 \cdot 3600 \cdot \frac{1}{T_s}.$$

El volumen mensual de datos crudos se aproxima como:

$$V_{\text{mes}} = R \cdot B_m.$$

Estas expresiones permiten calcular el crecimiento aproximado de la base de datos del histórico y medir el impacto en el costo de la arquitectura de aumentar la frecuencia de publicación o

el número de turbinas. Si la publicación desde la ESP3 hacia AWS pasa de ejecutarse cada 60 segundos a cada 10 segundos, la cantidad de datos se incrementa por seis. Además, si se aumenta el número de turbinas por 10, el volumen de datos se incrementa por 60, lo que convierte el muestreo y procesamiento Edge en el hardware en un factor importante. De acuerdo a lo anterior, se plantea un modelo básico que sirve para aproximarse al costo de la arquitectura bajo ciertos parámetros y para orientar de mejor manera la decisión sobre cuál tipo de arquitectura implementar.

6.3. Escenario base: una turbina

Para el análisis del escenario base se considera el monitoreo de una única turbina, publicación de datos JSON en AWS cada 10~12 segundos y mensajes JSON pequeños, de aproximadamente 262 bytes por mensaje enviado. Este escenario representa el prototipo actual, por lo que solo se tiene un `clientId`, una ruta para el último paquete de datos y una ruta para el histórico. En este caso, los costos esperados son relativamente bajos porque el volumen de datos es pequeño, las consultas históricas se limitan a un ejercicio estricto de monitoreo y la infraestructura no requiere servidores permanentes. Sin embargo, la arquitectura es escalable, y así como se tiene una turbina de prueba, podría escalarse a n cantidad [AWS, 2026n,c].

Tabla nro. 6.3: Comportamiento cualitativo de costos para una turbina.

Componente	Comportamiento de costo	Observación
AWS IoT Core	Depende del número de mensajes	Crece con frecuencia de publicación
Lambda	Depende de invocaciones y duración	Bajo si las funciones son simples
DynamoDB	Depende de lecturas/escrituras y almacenamiento	Adecuado para último dato
S3	Depende de almacenamiento y solicitudes	Económico para histórico
Glue	Depende de crawler/job y catálogo	Puede ejecutarse bajo demanda
Athena	Depende de datos escaneados	Requiere particiones y filtros

Componente	Comportamiento de costo	Observación
CloudFront/S3 web	Depende de tráfico web	Bajo para prototipo académico

La elección de DynamoDB para la ruta del último paquete y de S3+Glue+Athena para el manejo del histórico, se debe a los impactos que se identificaron mediante pruebas. DynamoDB mostró ser más rápido y más económico que Athena para el escaneo del último dato por variable dado que no se hace sobre la base histórica. Mientras que Athena se reserva para análisis por rangos en momentos específicos [AWS, 2026b,d].

6.3.1. Aplicación numérica del modelo para el escenario base

Para la aplicación del modelo propuesto se parte del escenario base planteado, con una turbina que cuenta con un sistema que envía (ESP3) un json de 262 bytes cada 10~12 segundos, aproximadamente 5 paquetes por minuto, y con un dashboard operado bajo demanda. El sistema físico se supone bajo operación continua pero la visualización opera bajo dos modalidades diferentes: 1) las cards se actualizan cada 30 segundos, representando el valor más actualizado para cada variable medida del sistema, 2) el histórico se visualiza bajo demanda para rangos específicos definidos por el usuario.

Las reglas de juego del ejemplo se presentan en la Tabla nro. 6.4.

Tabla nro. 6.4: Reglas de juego para el escenario base de costos.

Parámetro	Valor usado en el ejemplo	Comentario
Número de turbinas, N	1	Prototipo actual
Periodo de publicación, T_s	12 s	Frecuencia aproximada de envío
Días de operación, D	30	Mes de referencia
Tamaño promedio del json, B_m	262 B (bytes)	Mensaje pequeño de telemetría
Mensajes mensuales	216.000	Calculado a partir del periodo de publicación
Consultas históricas	12 por día	Uso manual y controlado

Parámetro	Valor usado en el ejemplo	Comentario
Consultas históricas mensuales	360	Dashboard sin auto-refresh histórico
Rango histórico típico	15 min	Consulta corta para visualización
Particiones por consulta histórica	16 aprox.	Filtro por year/month/day/hour/minute
Formato histórico	json	Estado actual del prototipo
Capa optimizada	No incluida	Se deja como mejora futura

El número mensual de registros se calcula como:

$$R = N \cdot D \cdot 24 \cdot 3600 \cdot \frac{1}{T_s}.$$

Sustituyendo los valores del escenario base:

$$R = 1 \cdot 30 \cdot 24 \cdot 3600 \cdot \frac{1}{12}.$$

$$R = 216.000 \text{ registros/mes.}$$

Si cada mensaje json tiene un tamaño promedio cercano a 1 KB, el volumen mensual de datos crudos es:

$$V_{\text{mes}} = R \cdot B_m.$$

$$V_{\text{mes}} = 216.000 \cdot 262 \text{ B} \approx 56,6 \text{ MB/mes.}$$

Este valor sugiere que el volumen de datos almacenados por turbina mensualmente es relativamente pequeño y no es un factor crítico. El costo de la arquitectura es más sensible al número de

solicitudes generadas sobre el histórico. Las publicaciones MQTT, reglas ejecutadas, almacenamiento en S3, actualizaciones en DynamoDB, manejo de dashboard, suelen ser procesos más económicos.

La Tabla nro. 6.5 muestra una estimación aproximada de los costos del escenario base con los supuestos mencionados. Estos valores no son el costo total dado que este puede variar, por ejemplo, de acuerdo al número de consultas. Pero si es un valor aproximado de acuerdo a los supuestos y debe contrastarse contra AWS Cost Explorer.

Tabla nro. 6.5: Estimación mensual aproximada para una turbina con dashboard histórico bajo demanda.

Servicio	Driver de costo	Estimación mensual	Costo aproximado
AWS IoT Core - mensajes	Mensajes MQTT publicados	216.000 mensajes	0,22 USD
AWS IoT Core - reglas y acciones	Regla de ingesta y acciones asociadas	216.000 reglas y acciones asociadas	0,10 USD
S3 histórico	Escritura de objetos json	216.000 PutObject	1,08 USD
S3 almacenamiento	Datos crudos almacenados	56,6 MB/mes	< 0,01 USD
DynamoDB	Actualización de último dato	216.000 escrituras	0,14 USD aprox.
Lambda	Procesamiento serverless	Invocaciones de ingesta y consulta	0,05 a 0,25 USD
Athena	Consultas históricas bajo demanda	360 consultas/mes	0,02 USD aprox.
S3 solicitudes históricas	Listado de particiones filtradas	5.760 LIST aprox.	0,03 USD aprox.
CloudFront/S3 web	Entrega del dashboard	Uso académico bajo	Bajo

Bajo los supuestos planteados, el costo mensual esperado de la arquitectura Cloud para una turbina estaría entre 1,6 a 3,0 USD, aunque quizá se cuente con algunos beneficios de capa gratuita como en CloudFront. Esto se presume como un costo económico, sin embargo, debe tenerse en cuenta que al tratarse de una arquitectura escalable los costos y requerimientos tienden a aumentar por lo que se recomienda hacer un análisis para cada caso concreto.

6.3.2. Reglas operativas del dashboard para controlar costos

A partir del planteamiento anterior, el dashboard actualiza automáticamente el último dato para todas las variables con mayor frecuencia desde DynamoDB, pero las consultas históricas deben ejecutarse bajo demanda. El histórico no debe quedar refrescándose automáticamente cada minuto durante operación normal porque eso genera nuevas consultas sobre Amazon S3 y es allí donde más se cobra, salvo que ese sea precisamente el escenario que se desea medir como prueba de carga o porque se quiere un dashboard reactivo 24/7.

La regla operativa queda definida así:

Tabla nro. 6.6: Reglas de operación del dashboard para el escenario base.

Funcionalidad del dashboard	Servicio consultado	Frecuencia recomendada	Observación de costo
Último dato	DynamoDB mediante <code>GetLatest</code>	Frecuente	Bajo costo
Histórico	Athena/S3 mediante <code>GetTimeseries</code>	Bajo demanda	Depende de rango y consultas
Auto-refresh histórico	Athena/S3 mediante <code>GetTimeseries</code>	Evitar en operación normal	Puede acumular costos
Consulta larga	Athena/S3 mediante <code>GetTimeseries</code>	Máximo controlado	Limitar por variable <code>MAX_QUERY_MINUTES</code>

En el escenario base de costos se asume que el dashboard no deja el histórico en auto-refresh permanente. El usuario ingresa al dashboard cuando requiere revisar una variable, selecciona un rango preestablecido, ejecuta la consulta y cierra el dashboard. Esta forma de operación es de bajo costo y permite separar el monitoreo continuo del último dato del análisis histórico bajo

demanda del usuario.

6.3.3. Prueba prolongada de 24 horas para contraste con Cost Explorer

Para comparar la estimación de costos presentada anteriormente se realizó una prueba de funcionamiento del prototipo prolongada por 24 horas, en la cual el sistema de hardware IoT se mantuvo enviando datos permanentemente y la arquitectura cloud cumpliendo con las tareas de almacenamiento, consulta y visualización. En esta prueba, el dashboard se mantiene cerrado la mayor parte del tiempo y se abre solo en momentos aleatorios para hacer consultas históricas puntuales. De esta forma se evita la medición de consultas automáticas permanentes del histórico.

La Tabla nro. 6.7 presenta la bitácora propuesta.

Tabla nro. 6.7: Plan de consultas puntuales para la prueba controlada de 24 horas.

Momento	Acción	Variable	Rango
Inicio	Cerrar dashboard y dejar dispositivo enviando datos	No aplica	No aplica
T + 1 h	Consulta puntual	Voltaje	15 min
T + 4 h	Consulta puntual	Temperatura	15 min
T + 4 h	Consulta puntual	Velocidad del viento	60 min
T + 8 h	Consulta puntual	RPM	60 min
T + 8 h	Consulta puntual	Humedad	15 min
T + 12 h	Consulta puntual	Presión	60 min
T + 12 h	Consulta puntual	Memoria	15 min
T + 18 h	Consulta puntual	Voltaje	180 min
T + 24 h	Consulta final	Voltaje	15 min
T + 24 h	Consulta final	Temperatura	60 min

Cada consulta histórica debe quedar documentada con hora, variable, rango solicitado, pantallazo del dashboard, `queryId` de Athena, número de particiones de minuto y datos analizados. Al finalizar la prueba se revisará Cost Explorer agrupando por servicio, tipo de uso y operación

API. Los campos más importantes para validar la corrección son `ListBucket`, `PutObject`, `GetObject`, `USE2-Requests-Tier1`, `USE2-Requests-Tier2` y `PRESTO_QUERY`.

Si el costo observado difiere de la estimación, el modelo no se descarta, se calibra. Esa es precisamente la utilidad del enfoque paramétrico: permite iniciar con una estimación razonable, medir el comportamiento real de la arquitectura y ajustar los supuestos operativos a partir de evidencia.

6.4. Escalamiento a n turbinas

Para escalar el sistema a más turbinas, por ejemplo diez, la arquitectura se puede estructurar bajo la misma lógica pero debe incorporar `clientId` como variable categórica central, donde cada ESP gateway es una categoría que contiene una data asociada. Cada nodo debe publicar con identidad única de manera que sea diferenciable, DynamoDB debe obtener un paquete por turbina, S3 debe organizar prefijos por tiempo y por dispositivo y Athena debe permitir filtros por `clientId` y rango temporal. La Tabla nro. 6.8 resume los cambios requeridos.

Tabla nro. 6.8: Ajustes de arquitectura para escalar a diez turbinas.

Capa	Una turbina	Diez turbinas	Recomendación
MQTT	Un tópico general	Tópico por dispositivo o prefijo	<code>sensor/{clientId}/data</code>
IoT Policy	Política amplia de prueba	Política por <code>clientId</code> o grupo	Mínimo privilegio
DynamoDB	Un ítem	Diez ítems actualizados	PK <code>clientId</code>
S3	Prefijo temporal	Prefijo temporal + dispositivo	Facilitar consultas
Athena	Consulta por fecha	Consulta por fecha y dispositivo	Reducir escaneo
Dashboard	Un panel	Selector de turbina	Consulta filtrada

Además, escalar el sistema a múltiples turbinas también requiere planificar con deta-

lle la estructura de la base de datos. Una estructura como `processed/clientId=ESP32C6-AWS/year=2026/month=05/day=18/` está pensada para facilitar las consultas por dispositivo y fecha para una o pocas turbinas. Sin embargo, generar demasiadas particiones pequeñas para múltiples `clientId` puede aumentar la complejidad, dificultar la administración, el rendimiento y los costos.

Por último, el prototipo almacena mensajes en estructura json porque son fáciles de generar, depurar y está pensada para etapas tempranas de prototipado, pero no es óptima para análisis de gran volumen o manipulación de múltiples turbinas. Para consultas históricas frecuentes, se recomienda revisar otras estructuras y transformar los datos por ejemplo hacia Parquet, un formato columnar que reduce datos escaneados cuando Athena consulta solo algunas columnas (AWS 2026c, 2026f).

Tabla nro. 6.9: Comparación entre almacenamiento JSON y Parquet para histórico IoT.

Criterio	json	Parquet
Facilidad de depuración	Alta	Media
Escritura desde IoT Rule	Directa	Requiere transformación
Eficiencia de consulta	Menor	Mayor
Lectura por columnas	No optimizada	Optimizada
Uso recomendado	Prototipo y datos crudos	Histórico analítico

6.5. Discusión de resultados

Los resultados de este trabajo muestran que una arquitectura soportada en servicios Cloud de tipo serverless, puede ser adecuada para el monitoreo de pequeños sistemas energéticos porque garantiza seguridad, trazabilidad, escalabilidad y mayor independencia al no depender de servidores físicos que deben administrarse y mantenerse.

Ahora bien, también se evidenciaron varios retos que de no abordarse de una manera sistémica pueden dificultar la administración de la arquitectura: 1) El manejo de muchos objetos pequeños en estructura json si se envían datos muy frecuentemente. 2) La latencia de Athena para consultas históricas muy amplias. 3) Programar bien las políticas IoT, permisos de IAM y funciones Lambda.

6.6. Síntesis del capítulo

El modelo de costos planteado resulta oportuno para determinar la cantidad de datos a procesar y así tener criterios de decisión sobre la frecuencia de publicación, particionamiento, estructura para la BD histórica y otros aspectos importantes para la parametrización de la arquitectura. De esto dependen los costos y la experiencia de usuario. La decisión de separar la visualización histórica de la visualización del último fue una decisión acertada, pues se logró evidencias que el manejo de Athena + Glue + S3 para esta operación “simple” dificultaba la operación e incrementaba los costos.

Conclusiones y trabajos futuros

Este capítulo recoge las conclusiones principales del trabajo desarrollado y plantea las líneas de continuidad que quedan abiertas. Las conclusiones no se presentan como una lista aislada de logros, sino como una lectura del sistema completo: desde la medición en campo con tarjetas ESP32-C6, pasando por la comunicación local y la publicación segura en AWS IoT Core, hasta el almacenamiento, la consulta histórica, la visualización web y el análisis de costos.

7.1. Conclusiones

1. El sistema desarrollado demuestra que es posible desarrollar una arquitectura de hardware distribuida soportada en tarjetas ESP32 para el monitoreo de un sistema de generación eólica de pequeña escala. La separación en tres nodos (3 tarjetas independientes), facilitó la división de tareas y responsabilidades por nodo, permitiendo que el nodo 1 cumpla tareas de captación de datos, el nodo 2 capta datos, integra y serializa y el nodo 3 garantiza la conexión con AWS y el envío del paquete JSON mediante MQTTs.
2. Se concluye que la combinación de protocolos ESP-NOW entre ESP1-ESP2 para comunicación local y UART entre ESP2-ESP3 para enviar la estructura de datos al gateway fue eficaz y aprovechó de buena manera los recursos de hardware disponibles para resolver las necesidades de comunicación, permitiendo construir una capa IoT Edge funcional, útil y económica para este tipo de sistemas.
3. Se demostró que es posible implementar una comunicación MQTT/TLS con certificados X.509 desde una tarjeta ESP32 hacia AWS IoT Core para generar una transmisión segura y

robusta de la telemetría enviada desde la ESP3. Esta conexión no solamente se limita a la publicación de un mensaje MQTT, requiere validación de identidad del gateway, políticas IoT en AWS, manejo de certificados, manejo de claves privadas, conexión a endpoint correcto, tópico MQTT y validación de la calidad de la cadena. El desarrollo de esta etapa constituye uno de los mayores logros en el proyecto.

4. La arquitectura cloud planteada se adapta a los requisitos de funcionamiento del sistema. DynamoDB se encarga de procesar el último paquete de datos para atender la visualización permanente, mientras que S3 + AWS Glue + Athena atienden la base de datos histórica y se gestiona la interacción con el almacenamiento a través de funciones Lambda + dashboard en CloudFront. Esta división permite reducir los costos de la arquitectura al usar una base de datos rápida y barata como DynamoDB para la visualización permanente, mientras que con Athena se manejan los rangos temporales de las consultas controladas al histórico.
5. El contrato JSON entre la ESP3 y AWS IoT Core se convierte en un elemento central ya que toda la arquitectura Cloud queda supeditada a esta estructura. Los nombres de las variables, las unidades, tipos de datos y convenciones preestablecidas para valores no disponibles, conectan el firmware con AWS IoT Core, reglas IoT, DynamoDB, S3, Glue, Athena, Lambda y el dashboard, el más mínimo cambio debe registrarse en toda la cadena. En la medida en que este contrato sea estable, el sistema es trazable y mantenible, lo cual es esencial para garantizar la escalabilidad futura.
6. Se comprueba que S3 es un servicio económico y apto para garantizar el almacenamiento masivo de datos y permite una manipulación personalizada. En este caso, se almacenaron y se procesaron estructuras JSON particionadas porque se probó sobre una única turbina. Para que el sistema sea escalable y sostenible, se puede continuar almacenando data cruda histórica y optimizarla a otras estructuras como Parquet para el procesamiento y visualización. En una arquitectura IoT con muchos dispositivos por monitorear, puede tener más peso en los costos el número de consultas sobre el almacenamiento en bruto, por lo que es importante trabajar con una estructura de datos adecuada.
7. La validación experimental es fundamental cuando se desea comprobar el correcto funcionamiento de la arquitectura. Mediante el proceso de pruebas y comparación analítica de costos se comprobó que en una versión 1 de la arquitectura se tenía un error ya que las consultas históricas estaban mal acotadas y se generaban cientos de operaciones de tipo

ListBucket en S3, lo que estaba aumentando los costos significativamente. En este tipo de arquitecturas, es importante tener presente que cada operación puede generar un costo adicional, por lo que es de extrema importancia delimitar muy bien el espectro de uso de cada servicio.

8. Toda arquitectura debe responder a una necesidad. En este caso base, más allá de verificar el funcionamiento del flujo de datos de extremo a extremo, se buscaba desarrollar un sistema que permita una visualización del estado del sistema cercana al tiempo real y consultar el comportamiento histórico de cada variable bajo demanda y necesidades del usuario. Se comprueba que este tipo de arquitecturas pueden ser sumamente económicas si se diseña la arquitectura con servicios serverless y se opera el dashboard con unas reglas claras.
9. La seguridad del sistema no termina en la implementación de la transmisión MQTT/TLS. Esta es una etapa sumamente importante porque es un proceso vulnerable, pero en AWS deben existir otras etapas necesarias como restringir las políticas IoT, implementar roles IAM con mínimo privilegio, control de CORS en las Function URLs de las funciones Lambda, el manejo cuidadoso de los certificados, monitoreo de servicios y eventos mediante CloudWatch y CloudTrail, entre otras, como la implementación secretos para aumentar la seguridad del dashboard.
10. El prototipo implementa una ruta completa de datos desde la captación mediante sensores hasta la visualización web, pasando por procesos de estructuración, almacenamiento local, transporte a través de varios protocolos, identificación, almacenamiento cloud, transformación, consulta y visualización. Un pipeline completo, replicable y que permite continuar experimentando con este tipo de sistemas, aunque para una implementación completa es recomendable afinar detalles en la instrumentación electrónica de manera que sea un proceso completamente estable y confiable.

7.2. Trabajos futuros

En esta etapa se comparten algunas mejoras identificadas durante el desarrollo del sistema que pueden mejorar el desempeño y eficiencia de la arquitectura.

1. Si bien previamente se realizó un proceso de comparación y calibración de sensores y circuitos de acondicionamiento, se recomienda avanzar en el proceso de calibración y

estandarización de las variables medidas en ESP1 y ESP2, de manera que cuenten con un valor o patrón de referencia más confiable.

2. En términos de la comunicación local se pueden hacer dos mejoras importantes. En ESP-NOW se puede mejorar el proceso de cifrado PMK/LMK y protección de la MAC. Además, en UART entre los nodos ESP2-ESP3 puede fortalecerse mediante protocolo de trama de longitud, delimitador y checksum o CRC para reducir el riesgo de truncamiento de mensajes o pérdida de información.
3. Aunque el sistema actualmente cumple con los requisitos mínimos de funcionamiento, se recomienda mejorar la gestión del tiempo mediante el DS3231 y documentar explícitamente el dominio de manejo (UTC, hora colombiana, etc) de los timestamps y las particiones ya que debe existir coincidencia temporal entre el firmware y los servicios de AWS. Adicionalmente, en el nodo gateway (ESP3) se puede fortalecer la validación temporal SNTP (Simple Network Time Protocol) de manera que el sistema primero haga una validación de tiempo y fecha antes de enviar a AWS IoT Core, ya que actualmente asume que está plenamente calibrado el RTC y esto podría llegar a generar fallos futuros.
4. Con el propósito de garantizar un mejor rendimiento en un proceso de escalabilidad del sistema, se recomienda optimizar el lago de datos implementado mediante otras estructuras de datos que pueden ser más óptimas para el proceso de monitoreo múltiple. Los datos en formato JSON (raw) deben conservarse, sin embargo, las consultas recientes e históricas del dashboard podrían mejorarse si se consultan sobre una capa optimizada en Parquet compactada por hora o por día, de acuerdo a las necesidades de monitoreo, de manera que se reduzca la cantidad de objetos pequeños (paquetes JSON) que deben operarse en cada consulta mediante Athena, lo que se traduce en mejoras de velocidad y costos.
5. Si bien la función `Lambda GetTimeseries` ya filtra las particiones temporales por hora y minuto, podría optimizarse aprovechando la consulta Athena en una ventana temporal seleccionada para consultar información de varias variables, actualmente solo actualiza la variable elegida en el dashboard. Adicionalmente, si son consultas repetidas por la necesidad del proceso de monitoreo, podría implementarse manejo temporal de caché para no estar cargando reiterativamente los mismos datos.
6. El dashboard que se propone es una interfaz básica, pensada únicamente para verificar el funcionamiento extremo a extremo del sistema. Por ende, este podría mejorarse implemen-

tando control de acceso mediante cognito o otras herramientas de manera que se controle quién ingresa, qué hace y cuándo. Además, puede mejorarse la experiencia de usuario con un FrontEnd más intuitivo y adaptado a una necesidad específica. Por último, para registrarse al control de costos, debe controlarse o eliminarse el refresh automático del histórico y permitir estas consultas únicamente bajo demanda.

7. Para mejorar la experiencia, los procesos de auditoría y trazabilidad del sistema, en un entorno de implementación permanente de monitoreo, se recomienda mejorar la observabilidad mediante el uso de servicios como CloudWatch y CloudTrail, de manera que puedan programarse alarmas ante eventos del sistema o aumentos de costos y medir estadísticas como invocaciones, duración, errores y logs de los diferentes servicios. Contar con el monitoreo y trazabilidad que permiten estos servicios facilita el relacionamiento y operación de la arquitectura.
8. Con el propósito de evaluar rendimiento y operación del sistema completo se propone avanzar en la implementación real a través de varias turbinas desplegadas geográficamente en diferentes lugares y conectadas a AWS IoT Core. Para ello se debe formalizar el identificador de dispositivo, el modelo de tópicos MQTT, la clave primaria en DynamoDB, la partición del histórico en S3 y el selector de turbina en el dashboard. Esta ampliación permitiría pasar de un prototipo individual a una plataforma experimental multi-dispositivo.
9. Con una base de datos consolidada en AWS a partir de la lectura de múltiples dispositivos durante un tiempo suficiente, se recomienda avanzar hacia la consolidación de modelos analíticos que permitan explorar las relaciones entre las diferentes variables del sistema, lo que permitiría fortalecer la estimación de potencial energético, detección de anomalías, curvas de desempeño de las máquinas eléctricas, predicción de la generación y mantenimiento preventivo a partir del auto reconocimiento. Estos modelos son muy interesantes pero también dependen de la calidad de los datos, por lo que en este proceso de implementación deben atenderse las recomendaciones para la mejora en el proceso de adquisición de datos.

7.3. Cierre

El trabajo desarrollado constituye una base de partida para reproducir una arquitectura de hardware y software que permite el monitoreo de un generador eólico de pequeña escala mediante

la integración de hardware IoT embebido con comunicación segura hacia AWS, consolidando un sistema de datos verificable de extremo a extremo y marcando un avance importante en la integración de un prototipo de generación de energía eléctrica con tecnologías de las industrias 4.0., lo que es aplicable y escalable en sistemas físicos reales.

A partir del prototipo propuesto se puede avanzar en la construcción de una plataforma en terreno que permita la integración de la arquitectura de hardware y cloud propuestas para recolectar datos experimentales. Este trabajo sienta las bases para implementar este camino, dejando abierta la posibilidad de continuar desarrollando y mejorando este sistema.

Anexo A — Código ESP1

Este anexo se reserva para el código completo del nodo ESP1, correspondiente a la etapa de adquisición de campo. En la versión final se debe incluir el archivo fuente principal, la ruta dentro del proyecto ESP-IDF y el hash SHA-256 de la versión compilada.

Ruta de referencia: `clase-UART/main/main7.c`

Función principal: lectura GPS, medición de voltaje, cálculo de RPM y envío ESP-NOW hacia ESP2.

Anexo B — Código ESP2

Este anexo se reserva para el código completo del nodo ESP2, correspondiente a la etapa de integración local. En la versión final se debe incluir el archivo fuente principal, la ruta dentro del proyecto ESP-IDF y el hash SHA-256 de la versión compilada.

Ruta de referencia: `Proyecto_Integrador/main/main7.c`

Función principal: recepción ESP-NOW, medición de viento y ambiente, RTC, microSD, OLED, construcción de JSON y envío UART hacia ESP3.

Anexo C — Código ESP3

Este anexo se reserva para el código completo del gateway ESP3. En la versión final se debe incluir el archivo fuente principal, la ruta dentro del proyecto ESP-IDF y el hash SHA-256 de la versión compilada.

Ruta de referencia: `AWS_IoT_Core/main/main2.c`

Función principal: recepción JSON por UART, conexión Wi-Fi, establecimiento MQTT/TLS y publicación hacia AWS IoT Core.

Anexo D — Lambdas y frontend

Este anexo se reserva para el código de las funciones Lambda y del frontend React/Vite. En la versión final se deben incluir, como mínimo, las funciones `GetLatest`, `GetTimeseries`, `ActualizarUltimoDatoDynamo`, el archivo `.env` sin secretos, `package.json` y los componentes principales del dashboard.

No se deben publicar certificados, llaves privadas, contraseñas Wi-Fi ni tokens de acceso. Cuando sea necesario documentar configuración sensible, se debe reemplazar por variables genéricas.

Referencias

- Ersin Akyuz and Batin demircan. Iot and cloud based remote monitoring of wind turbine. *Celal Bayar Üniversitesi Fen Bilimleri Dergisi*, 15, 12 2019. doi: 10.18466/cbayarfbe.540812.
- Ala Al-Fuqaha, Mohsen Guizani, Mehdi Mohammadi, Mohammed Aledhari, and Moussa Ayyash. Internet of things: A survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials*, 17(4):2347–2376, 2015. doi: 10.1109/COMST.2015.2444095.
- Abdul Hai Alami, Montaser Mahmoud, Haya Aljaghoub, Ayman Mdallal, Mohammad Ali Abdelkareem, Siti Kartom Kamarudin, and Abdul Ghani Olabi. Progress in 3d printing in wind energy and its role in achieving sustainability. *International Journal of Thermofluids*, 20: 100496, 2023. doi: 10.1016/j.ijft.2023.100496.
- Fatima Alwahedi, Alyazia Aldhaheeri, Mohamed Amine Ferrag, Ammar Battah, and Norbert Tihanyi. Machine learning techniques for iot security: Current research and future vision with generative ai and large language models. *Internet of Things and Cyber-Physical Systems*, 4: 167–185, 2024. ISSN 2667-3452. doi: <https://doi.org/10.1016/j.iotcps.2023.12.003>. URL <https://www.sciencedirect.com/science/article/pii/S2667345223000585>.
- Amazon Web Services. Querying with prepared statements in Amazon Athena. <https://docs.aws.amazon.com/athena/latest/ug/querying-with-prepared-statements.html>, 2026a. Official AWS documentation for using prepared statements in Amazon Athena.
- Amazon Web Services. What is Amazon Athena? <https://docs.aws.amazon.com/athena/latest/ug/what-is.html>, 2026b. Official AWS documentation describing Amazon Athena as an interactive query service for analyzing data directly in Amazon S3 using standard SQL.
- Amazon Web Services. Customizing error responses in Amazon CloudFront. <https://docs.aws.amazon.com/athena/latest/ug/what-is.html>.

- [amazon.com/AmazonCloudFront/latest/DeveloperGuide/GeneratingCustomErrorResponses.html](https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/GeneratingCustomErrorResponses.html), 2026c. Official AWS documentation for configuring custom error responses such as routing 403 or 404 responses to index.html.
- Amazon Web Services. Amazon CloudFront Developer Guide. <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html>, 2026d. Official AWS documentation for Amazon CloudFront, including content distribution and edge delivery.
- Amazon Web Services. Restrict access to an Amazon S3 origin using Origin Access Control. <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html>, 2026e. Official AWS documentation for using CloudFront Origin Access Control to restrict direct access to S3 origins.
- Amazon Web Services. Authoring jobs in AWS Glue. <https://docs.aws.amazon.com/glue/latest/dg/author-job.html>, 2026f. Official AWS documentation for creating and managing ETL jobs in AWS Glue.
- Amazon Web Services. Setting up IAM permissions for AWS Glue. <https://docs.aws.amazon.com/glue/latest/dg/set-up-iam.html>, 2026g. Official AWS Glue documentation for configuring IAM permissions and service roles for AWS Glue resources.
- Amazon Web Services. Apply least-privilege permissions. <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>, 2026h. Official AWS IAM documentation describing the principle of granting only the permissions required to perform a task.
- Amazon Web Services. S3 rule action in AWS IoT Core. <https://docs.aws.amazon.com/iot/latest/developerguide/s3-rule-action.html>, 2026i. Official AWS IoT Core documentation describing how an IoT rule action writes MQTT message data to an Amazon S3 bucket.
- Amazon Web Services. Defining Lambda function permissions with an execution role. <https://docs.aws.amazon.com/lambda/latest/dg/lambda-intro-execution-role.html>, 2026j. Official AWS Lambda documentation describing execution roles used by Lambda functions to access AWS services and resources.
- Amazon Web Services. AWS Pricing Calculator. <https://calculator.aws/>, 2026k. Official AWS tool for estimating service costs based on usage assumptions, architecture parameters and selected AWS services.

- Amazon Web Services. Examples of Amazon S3 bucket policies. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/example-bucket-policies.html>, 2026l. Official AWS documentation with examples of S3 bucket policies for controlling access to S3 objects and buckets.
- Amazon Web Services. Hosting a static website using Amazon S3. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteHosting.html>, 2026m. Official AWS documentation for hosting static website content using Amazon S3.
- Amazon Web Services. Cost Optimization Pillar - AWS Well-Architected Framework. <https://docs.aws.amazon.com/wellarchitected/latest/cost-optimization-pillar/welcome.html>, 2026n. Official AWS guidance for designing cost-aware architectures, including usage-based cost management and cost optimization practices.
- Analog Devices. DS3231: Extremely Accurate I2C-Integrated RTC/TCXO/Crystal Datasheet. <https://www.analog.com/media/en/technical-documentation/data-sheets/ds3231.pdf>, 2015. Datasheet for the DS3231 real-time clock with I2C interface.
- Apache Software Foundation. Apache Parquet Documentation. <https://parquet.apache.org/docs/>, 2026. Official documentation for the Apache Parquet columnar data file format.
- Luigi Atzori, Antonio Iera, and Giacomo Morabito. The internet of things: A survey. *Computer Networks*, 54(15):2787–2805, 2010. doi: 10.1016/j.comnet.2010.05.010.
- AWS. Use aws glue data catalog to connect to your data in athena, 2026a. URL <https://docs.aws.amazon.com/athena/latest/ug/data-sources-glue.html>.
- AWS. Partition your data in amazon athena, 2026b. URL <https://docs.aws.amazon.com/athena/latest/ug/partitions.html>.
- AWS. Core components of amazon dynamodb, 2026c. URL <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html>.
- AWS. Query operations in dynamodb, 2026d. URL <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Query.html>.
- AWS. Using crawlers to populate the aws glue data catalog, 2026e. URL <https://docs.aws.amazon.com/glue/latest/dg/add-crawler.html>.

- AWS. Aws iot core policies, 2026f. URL <https://docs.aws.amazon.com/iot/latest/developerguide/iot-policies.html>.
- AWS. Device communication protocols for aws iot core, 2026g. URL <https://docs.aws.amazon.com/iot/latest/developerguide/protocols.html>.
- AWS. Rules for aws iot, 2026h. URL <https://docs.aws.amazon.com/iot/latest/developerguide/iot-rules.html>.
- AWS. Security best practices in aws iot core, 2026i. URL <https://docs.aws.amazon.com/iot/latest/developerguide/security-best-practices.html>.
- AWS. X.509 client certificates for aws iot core, 2026j. URL <https://docs.aws.amazon.com/iot/latest/developerguide/x509-client-certs.html>.
- AWS. Creating and managing lambda function urls, 2026k. URL <https://docs.aws.amazon.com/lambda/latest/dg/urls-configuration.html>.
- AWS. Cors settings for lambda function urls, 2026l. URL https://docs.aws.amazon.com/lambda/latest/api/API_Cors.html.
- AWS. Organizing objects using prefixes in amazon s3, 2026m. URL <https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-prefixes.html>.
- AWS. Serverless applications lens: Aws well-architected framework, 2026n. URL <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/welcome.html>.
- Taimur Bakhshi, Bogdan Ghita, and Ievgeniia Kuzminykh. A review of iot firmware vulnerabilities and auditing techniques. *Sensors*, 24(2), 2024. ISSN 1424-8220. doi: 10.3390/s24020708. URL <https://www.mdpi.com/1424-8220/24/2/708>.
- K. Bassett, R. Carriveau, and D. S.-K. Ting. 3d printed wind turbines part 1: Design considerations and rapid manufacture potential. *Sustainable Energy Technologies and Assessments*, 11:186–193, 2015. doi: 10.1016/j.seta.2015.01.002.
- Muhammad Mahmood Aslam Bhutta, Nasir Hayat, Ahmed Uzair Farooq, Zain Ali, Sh. Rehan Jamil, and Zahid Hussain. Vertical axis wind turbine—a review of various configurations and design techniques. *Renewable and Sustainable Energy Reviews*, 16(4):1926–1939, 2012. doi: 10.1016/j.rser.2011.12.004.

- Bosch Sensortec. Bme680 low power gas, pressure, temperature and humidity sensor datasheet, 2024. URL <https://www.bosch-sensortec.com/products/environmental-sensors/gas-sensors/bme680/>.
- Alessio Botta, Walter de Donato, Valerio Persico, and Antonio Pescapè. Integration of cloud computing and internet of things: A survey. *Future Generation Computer Systems*, 56:684–700, 2016. doi: 10.1016/j.future.2015.09.021.
- Tim Bray. The JavaScript Object Notation (JSON) Data Interchange Format. Technical Report RFC 8259, Internet Engineering Task Force, 2017. URL <https://www.rfc-editor.org/rfc/rfc8259>. Defines JSON as a lightweight, text-based data interchange format.
- Howard Butler, Martin Daly, Allan Doyle, Sean Gillies, Stefan Hagen, and Tim Schaub. The GeoJSON Format. Technical Report RFC 7946, Internet Engineering Task Force, 2016. URL <https://www.rfc-editor.org/rfc/rfc7946>. Specifies the GeoJSON format for encoding geographic data structures using decimal longitude and latitude coordinates.
- Katrina Calautit and Cameron Johnstone. State-of-the-art review of micro to small-scale wind energy harvesting technologies for building integration. *Energy Conversion and Management: X*, 20:100457, 2023. doi: 10.1016/j.ecmx.2023.100457.
- Longhuan Du, Grant Ingram, and Robert G. Dominy. A review of H-Darrieus wind turbine aerodynamic research. *Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science*, 233(23–24):7590–7616, 2019. doi: 10.1177/0954406219885962.
- Wesley Eddy. Transmission control protocol (tcp). Technical Report RFC 9293, Internet Engineering Task Force, 2022a. URL <https://www.rfc-editor.org/rfc/rfc9293>.
- Wesley Eddy. Transmission Control Protocol (TCP). Technical Report RFC 9293, Internet Engineering Task Force, 2022b. URL <https://www.rfc-editor.org/rfc/rfc9293>. Current specification of the Transmission Control Protocol.
- Christos Emexidis and Panagiotis Gkonis. The integration of internet of things and machine learning for energy prediction of wind turbines. *Applied Sciences*, 14(22), 2024. ISSN 2076-3417. doi: 10.3390/app142210276. URL <https://www.mdpi.com/2076-3417/14/22/10276>.
- Sandra Eriksson, Hans Bernhoff, and Mats Leijon. Evaluation of different turbine concepts

- for wind power. *Renewable and Sustainable Energy Reviews*, 12(5):1419–1434, 2008. doi: 10.1016/j.rser.2006.05.017.
- Espressif Systems. ESP32-C6 Series Datasheet. https://documentation.espressif.com/esp32-c6_datasheet_en.pdf, 2023. Datasheet for the ESP32-C6 series, including system features, wireless connectivity and peripheral interfaces.
- Espressif Systems. Adc calibration driver, 2026a. URL https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/peripherals/adc_calibration.html.
- Espressif Systems. Gpio and rtc gpio driver for esp32-c6, 2026b. URL <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/peripherals/gpio.html>.
- Espressif Systems. General purpose timer driver, 2026c. URL <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/peripherals/gptimer.html>.
- Espressif Systems. Esp-now programming guide, 2026d. URL https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/network/esp_now.html.
- Espressif Systems. Event Loop Library: ESP32-C6. https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/system/esp_event.html, 2026e. ESP-IDF Programming Guide. Documents event loops, event handlers and asynchronous event dispatching.
- Espressif Systems. Freertos overview in esp-idf, 2026f. URL <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/system/freertos.html>.
- Espressif Systems. I2c driver, 2026g. URL <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/peripherals/i2c.html>.
- Espressif Systems. Esp-mqtt client, 2026h. URL <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/protocols/mqtt.html>.
- Espressif Systems. ESP-MQTT: ESP32-C6. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/protocols/mqtt.html>, 2026i. ESP-IDF Programming Guide. Documents the ESP-MQTT client, including MQTT client configuration and secure MQTT transport.
- Espressif Systems. NVS Encryption: ESP32-C6. https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/security/nvs_encryption.html, 2026j. ESP-IDF Programming Guide. Documents encryption for Non-Volatile Storage partitions on ESP32-C6.

- Espressif Systems. SD SPI Host Driver: ESP32-C6. https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/peripherals/sdspi_host.html, 2026k. ESP-IDF Programming Guide. Documents SD card communication using the SPI host driver on ESP32-C6.
- Espressif Systems. Uart driver, 2026l. URL <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/peripherals/uart.html>.
- Espressif Systems. Unified Provisioning: ESP32-C6. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/provisioning/index.html>, 2026m. ESP-IDF Programming Guide. Documents provisioning mechanisms for configuring Wi-Fi and device parameters.
- Espressif Systems. Wi-Fi Driver: ESP32-C6. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-guides/wifi.html>, 2026n. ESP-IDF Programming Guide. Documents Wi-Fi station mode, Wi-Fi initialization, connection handling and Wi-Fi events on ESP32-C6.
- Aoife M. Foley, Paul G. Leahy, Antonino Marvuglia, and Eamon J. McKeogh. Current methods and advances in forecasting of wind power generation. *Renewable Energy*, 37(1):1–8, 2012. doi: 10.1016/j.renene.2011.05.033.
- Lei Gong and Yanhui Chen. Machine learning-enhanced iot and wireless sensor networks for predictive analysis and maintenance in wind turbine systems. *International Journal of Intelligent Networks*, 5:133–144, 2024. ISSN 2666-6030. doi: <https://doi.org/10.1016/j.ijin.2024.02.002>. URL <https://www.sciencedirect.com/science/article/pii/S2666603024000083>.
- Jayavardhana Gubbi, Rajkumar Buyya, Slaven Marusic, and Marimuthu Palaniswami. Internet of things (iot): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7):1645–1660, 2013. ISSN 0167-739X. doi: <https://doi.org/10.1016/j.future.2013.01.010>. URL <https://www.sciencedirect.com/science/article/pii/S0167739X13000241>. Including Special sections: Cyber-enabled Distributed Computing for Ubiquitous Cloud and Network Services & Cloud Computing and Scientific Applications — Big Data, Scalable Analytics, and Beyond.
- Carlos Andrés Gómez Flórez. Prototipado y análisis comparativo de dos turbinas eólicas de eje vertical para generación eléctrica a pequeña escala. Master's thesis, Universidad Tecnológica de Pereira, Pereira, Colombia, 2024. URL <https://hdl.handle.net/11059/15277>.
- Ibrahim Abaker Targio Hashem, Ibrar Yaqoob, Nor Badrul Anuar, Salimah Mokhtar, Abdullah

- Gani, and Samee Ullah Khan. The rise of “big data” on cloud computing: Review and open research issues. *Information Systems*, 47:98–115, 2015. doi: 10.1016/j.is.2014.07.006.
- IEEE Computer Society. IEEE Standard for Configuration Management in Systems and Software Engineering. IEEE Std 828-2012, 2012. Standard for configuration management, configuration identification, configuration control, status accounting, and configuration audits.
- IEEE Standards Association. Ieee standard for information technology–telecommunications and information exchange between systems local and metropolitan area networks–specific requirements part 11: Wireless lan medium access control and physical layer specifications, 2018. URL https://standards.ieee.org/standard/802_11-2016.html.
- International Energy Agency (IEA). Power system digitalisation is crucial for clean energy transitions and security in developing markets, but investment is lagging. <https://www.iea.org/news/power-system-digitalisation-is-crucial-for-clean-energy-transitions-and-security-in-developing-markets-but-investment-is-lagging>, June 2023. Accessed: 16 sep. 2025.
- Elysa Nensy Irawan, Nuur Wachid Abdul Majid, Liptia Venica, Fahrur Aslami, and Goro Fujita. Analyzing the growth and trends of vertical axis wind turbine research: Insight from a bibliometric study. *Journal of Mechatronics, Electrical Power, and Vehicular Technology*, 14(1): 55–61, 2023.
- ISO/IEC/IEEE. ISO/IEC/IEEE 29148:2018 Systems and software engineering – Life cycle processes – Requirements engineering. International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers, 2018. Standard for requirements engineering, requirements characteristics and traceability.
- Jaesung Jung and Robert P. Broadwater. Current status and future advances for wind speed and power forecasting. *Renewable and Sustainable Energy Reviews*, 31:762–777, 2014. doi: 10.1016/j.rser.2013.12.054.
- Walt Kester, editor. *The Data Conversion Handbook*. Newnes, Burlington, MA, 2005. ISBN 9780750678414. Reference text on analog-to-digital conversion, signal conditioning, sampling, noise and practical data acquisition.
- Rafiullah Khan, Sarmad Ullah Khan, Rifaqat Zaheer, and Shahid Khan. Future internet: The

- internet of things architecture, possible applications and key challenges. In *2012 10th International Conference on Frontiers of Information Technology*, pages 257–260, 2012. doi: 10.1109/FIT.2012.53.
- Philip Koopman and Tridib Chakravarty. Cyclic Redundancy Code (CRC) Polynomial Selection for Embedded Networks. In *International Conference on Dependable Systems and Networks*, pages 145–154. IEEE, 2004. doi: 10.1109/DSN.2004.1311885. Reference on CRC-based error detection for embedded communication networks.
- Chunwen Liu, Ru Tan, Yang Wu, Yun Feng, Ze Jin, Fangjiao Zhang, Yuling Liu, and Qixu Liu. Dissecting zero trust: Research landscape and its implementation in iot. *Cybersecurity*, 7(1), May 2024. doi: 10.1186/s42400-024-00212-0.
- Zhan Liu, Limin Jia, and Yu Pang. Wind turbine operation and maintenance video supportive system based on the mobile internet of things. *Procedia Computer Science*, 107:332–340, 2017. ISSN 1877-0509. doi: <https://doi.org/10.1016/j.procs.2017.03.114>. URL <https://www.sciencedirect.com/science/article/pii/S1877050917303897>. Advances in Information and Communication Technology: Proceedings of 7th International Congress of Information and Communication Technology (ICICT2017).
- Zongxu Liu, Hui Guo, Yingshuai Zhang, and Zongliang Zuo. A comprehensive review of wind power prediction based on machine learning: Models, applications, and challenges. *Energies*, 18(2), 2025. ISSN 1996-1073. doi: 10.3390/en18020350. URL <https://www.mdpi.com/1996-1073/18/2/350>.
- Parushi Malhotra, Yashwant Singh, Pooja Anand, Deep Kumar Bangotra, Pradeep Kumar Singh, and Wei-Chiang Hong. Internet of things: Evolution, concerns and security challenges. *Sensors*, 21(5), 2021. ISSN 1424-8220. doi: 10.3390/s21051809. URL <https://www.mdpi.com/1424-8220/21/5/1809>.
- Mohsen Marjani, Fariza Nasaruddin, Abdullah Gani, Ahmad Karim, Ibrahim Abaker Targio Hashem, Aisha Siddiqa, and Ibrar Yaqoob. Big iot data analytics: Architecture, opportunities, and open research challenges. *IEEE Access*, 5:5247–5261, 2017. doi: 10.1109/ACCESS.2017.2689040.
- Peter Mell and Timothy Grance. The NIST Definition of Cloud Computing. Technical Report NIST Special Publication 800-145, National Institute of Standards and Technology, 2011.

- URL <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf>. Defines essential characteristics, deployment models and service models of cloud computing, including Software as a Service.
- Meta Open Source. React Documentation. <https://react.dev/>, 2026. Official React documentation for building user interfaces with components.
- Mozilla Developer Network. Reason: Multiple CORS header 'Access-Control-Allow-Origin' not allowed. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS/Errors/CORSMultipleAllowOriginNotAllowed>, 2026. MDN Web Docs explanation of browser CORS errors caused by multiple Access-Control-Allow-Origin headers.
- National Institute of Standards and Technology. FIPS 180-4: Secure Hash Standard (SHS). <https://csrc.nist.gov/pubs/fips/180-4/upd1/final>, 2015. Federal Information Processing Standards Publication. Specifies secure hash algorithms including SHA-256.
- National Institute of Standards and Technology. Networks of things. Technical Report Special Publication 800-183, NIST, 2016. URL <https://csrc.nist.gov/publications/detail/sp/800-183/final>.
- National Institute of Standards and Technology. Considerations for managing internet of things (iot) cybersecurity and privacy risks. Technical Report Internal Report 8228, NIST, 2019. URL <https://csrc.nist.gov/publications/detail/nistir/8228/final>.
- National Marine Electronics Association. Nmea 0183 standard for interfacing marine electronic devices, 2018. URL <https://www.nmea.org/>.
- OASIS. Mqtt version 5.0 oasis standard, 2019. URL <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>.
- OWASP Foundation. SQL Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html, 2026. Security guidance for preventing SQL injection through parameterized queries, input validation and allow-listing.
- Charith Perera, Arkady Zaslavsky, Peter Christen, and Dimitrios Georgakopoulos. Sensing as a Service Model for Smart Cities Supported by Internet of Things. In *Transactions on Emerging Telecommunications Technologies*, volume 25, pages 81–93, 2014. doi: 10.1002/ett.2704. Presents the Sensing as a Service model supported by IoT technologies.

- S. C. Pryor, F.W. Letson, and R. J. Barthelmie. Variability in wind energy generation across the contiguous united states. *Journal of Applied Meteorology and Climatology*, 59(12):2021 – 2039, 2020. doi: 10.1175/JAMC-D-20-0162.1. URL <https://journals.ametsoc.org/view/journals/apme/59/12/jamc-d-20-0162.1.xml>.
- Silvio Quincozes, Tubino Emilio, and Juliano Kazienko. MQTT protocol: fundamentals, tools and future directions. *IEEE Latin America Transactions*, 17(9):1439–1448, 2019. doi: 10.1109/TLA.2019.8931137.
- R., Preetha, S., Ramesh Kumar, R., Srisainath, and Divya, P. Backiya. Integrating renewable energy sources with micro grid using iot and machine learning. *E3S Web Conf.*, 387: 02004, 2023. doi: 10.1051/e3sconf/202338702004. URL <https://doi.org/10.1051/e3sconf/202338702004>.
- Recharts. Recharts Documentation. <https://recharts.org/en-US/>, 2026. Official documentation for building charts in React applications.
- Eric Rescorla. The transport layer security (tls) protocol version 1.3. Technical Report RFC 8446, Internet Engineering Task Force, 2018. URL <https://www.rfc-editor.org/rfc/rfc8446>.
- Tinshu Sasi, Arash Habibi Lashkari, Rongxing Lu, Pulei Xiong, and Shahrear Iqbal. A comprehensive survey on iot attacks: Taxonomy, detection mechanisms and challenges. *Journal of Information and Intelligence*, 2(6):455–513, 2024. ISSN 2949-7159. doi: <https://doi.org/10.1016/j.jiixd.2023.12.001>. URL <https://www.sciencedirect.com/science/article/pii/S2949715923000793>.
- Emiliano Sisinni, Abusayeed Saifullah, Song Han, Ulf Jennehag, and Mikael Gidlund. Industrial internet of things: Challenges, opportunities, and directions. *IEEE Transactions on Industrial Informatics*, 14(11):4724–4734, 2018. doi: 10.1109/TII.2018.2852491.
- Solomon Systech. SSD1306: 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller. <https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>, 2008. Datasheet for the SSD1306 OLED/PLED display controller.
- Pierre Tchakoua, René Wamkeue, Mohand Ouhrouche, Fouad Slaoui-Hasnaoui, Tommy Andy Tameghe, and Gabriel Ekemb. Wind turbine condition monitoring: State-of-the-art review, new trends, and future challenges. *Energies*, 7(4):2595–2630, 2014. ISSN 1996-1073. doi: 10.3390/en7042595. URL <https://www.mdpi.com/1996-1073/7/4/2595>.

- W. Tjiu, T. Marnoto, S. Mat, M. H. Ruslan, and K. Sopian. Darrieus vertical axis wind turbine for power generation i: Assessment of darrieus VAWT configurations. *Renewable Energy*, 75: 50–67, 2015. doi: 10.1016/j.renene.2014.09.038.
- Farhan Ullah, Xuexia Zhang, Mansoor Khan, Muhammad Shahid Mastoi, Hafiz Mudassir Munir, Aymen Flah, and Yahia Said. A comprehensive review of wind power integration and energy storage technologies for modern grid frequency regulation. *Heliyon*, 10(9):e30466, 2024. ISSN 2405-8440. doi: <https://doi.org/10.1016/j.heliyon.2024.e30466>. URL <https://www.sciencedirect.com/science/article/pii/S2405844024064971>.
- UPME. Plan indicativo de expansión de la generación 2023–2037. https://www1.upme.gov.co/siel/Plan_expansin_generacion_transmision/Plan_de_Expansion_Generacion_2023-2037_a_comentarios.pdf, April 2024. Accedido: 16 sep. 2025.
- Vite. Vite Guide. <https://vite.dev/guide/>, 2026. Official Vite documentation for frontend tooling and static application builds.
- Hao Wang, Bendong Xiong, Zutao Zhang, Hexiang Zhang, and Ali Azam. Small wind turbines and their potential for internet of things applications. *iScience*, 26(9):107674, 2023. doi: 10.1016/j.isci.2023.107674. URL <https://www.sciencedirect.com/science/article/pii/S2589004223017510>.
- David Wafula Wekesa, Churchill Otieno Saoke, and Joseph Ngugi Kamau. An experimental investigation into performance characteristics of H-shaped and Savonius-type VAWT rotors. *Scientific African*, 10:e00603, 2020. doi: 10.1016/j.sciaf.2020.e00603.
- Taolue Yang, Tao Tao, Xinran Guo, Yi Yang, and Shi Liu. Preliminary test for 3d surface strain measurement in the tower and foundation of offshore wind turbines using dofs. *Sensors*, 23 (15), 2023. ISSN 1424-8220. doi: 10.3390/s23156734. URL <https://www.mdpi.com/1424-8220/23/15/6734>.